1. Parte 1. Diseño físico y DDL

1.1. Definición del modelo relacional

El sistema de gestión de reservas para un hotel se estructura mediante cuatro tablas principales: cliente, habitacion, reserva y pago. Estas permiten almacenar y relacionar la información necesaria para administrar los clientes, las habitaciones disponibles, las reservas realizadas y los pagos asociados.

El modelo relacional se define de la siguiente manera:

Tabla 1. cliente

Atributo	Tipo de dato	Clave
id_cliente	SERIAL	PK
nombre	VARCHAR(100)	—
correo	VARCHAR(120)	—
telefono	VARCHAR(20)	—
nacionalidad	VARCHAR(60)	—

Tabla 2. habitacion

Atributo	Tipo de dato	Clave
id_habitacion	SERIAL	PK
numero	VARCHAR(10)	UNIQUE
tipo	VARCHAR(20)	—
precio_noche	NUMERIC(10,2)	—
estado	VARCHAR(20)	—

Tabla 3. reserva

Atributo	Tipo de dato	Clave
id_reserva	SERIAL	PK
id_cliente	INTEGER	FK
id_habitacion	INTEGER	FK
fecha_entrada	DATE	—
fecha_salida	DATE	—

Tabla 4. pago

Atributo	Tipo de dato	Clave
id_pago	SERIAL	PK

id_reserva	INTEGER	FK
fecha_pago	DATE	—
monto	NUMERIC(10,2)	—
metodo	VARCHAR(20)	—

Relaciones entre las tablas

- **cliente — reserva (1):** un cliente puede realizar múltiples reservas, pero cada reserva pertenece a un único cliente.
- **habitacion — reserva (1):** una habitación puede tener múltiples reservas en diferentes períodos, pero cada reserva corresponde a una sola habitación.
- **reserva — pago (1):** una reserva puede tener varios pagos asociados, aunque inicialmente puede no registrar ninguno. Cada pago corresponde a una única reserva.

Las claves primarias (PK) identifican de manera única cada registro, mientras que las claves foráneas (FK) permiten establecer las relaciones entre las tablas y mantener la integridad referencial.

El atributo estado_reserva se incorporará posteriormente mediante una sentencia ALTER TABLE, para representar una modificación controlada del modelo físico y permitir la gestión de cancelaciones.

1.2. Selección y justificación de los tipos de datos

Para implementar el modelo relacional del sistema de gestión de reservas del hotel en PostgreSQL, se seleccionaron tipos de datos apropiados para las características de cada atributo. Esta elección busca garantizar un almacenamiento adecuado de la información, facilitar las operaciones SQL y contribuir a la integridad de los datos.

1.2.1. Tabla cliente

Atributo	Tipo de dato	Justificación
id_cliente	SERIAL	Permite generar automáticamente un identificador numérico único para cada cliente mediante una secuencia de PostgreSQL.
nombre	VARCHAR(100)	Permite almacenar nombres y apellidos de longitud variable, con un máximo de 100 caracteres.
correo	VARCHAR(120)	Almacena direcciones de correo electrónico de diferentes longitudes.

telefono	VARCHAR(20)	Permite almacenar números telefónicos como texto, conservando prefijos internacionales, el símbolo + y posibles ceros iniciales.
nacionalidad	VARCHAR(60)	Permite registrar la nacionalidad del cliente mediante un texto de longitud variable.

1.2.2. Tabla habitacion

Atributo	Tipo de dato	Justificación
id_habitacion	SERIAL	Genera automáticamente un identificador numérico para cada habitación.
numero	VARCHAR(10)	Permite identificar las habitaciones mediante números o combinaciones alfanuméricas, como 101 o A102.
tipo	VARCHAR(20)	Almacena los tipos de habitación establecidos: Sencilla, Doble y Suite.
precio_noche	NUMERIC(10,2)	Permite almacenar el precio por noche con precisión decimal, evitando errores de aproximación propios de los tipos de punto flotante.
estado	VARCHAR(20)	Almacena los estados definidos: Disponible, Ocupada y Mantenimiento.

1.2.3. Tabla reserva

Atributo	Tipo de dato	Justificación
id_reserva	SERIAL	Genera automáticamente un identificador numérico para cada reserva.
id_cliente	INTEGER	Almacena el identificador del cliente relacionado con la reserva y es compatible con la clave primaria de la tabla cliente.
id_habitacion	INTEGER	Almacena el identificador de la habitación reservada y es compatible con la clave primaria de la tabla habitacion.
fecha_entrada	DATE	Permite almacenar la fecha de ingreso del cliente sin registrar una hora específica.
fecha_salida	DATE	Permite almacenar la fecha de salida y calcular la duración de la estancia mediante la diferencia entre fechas.

El valor total de la reserva se calculará multiplicando el número de noches por el precio correspondiente de la habitación, conforme a lo establecido en el enunciado.

Posteriormente, se incorporará el atributo estado_reserva mediante una sentencia ALTER TABLE, con el propósito de permitir la gestión de cancelaciones y documentar la evolución del modelo físico.

1.2.4. Tabla pago

Atributo	Tipo de dato	Justificación
id_pago	SERIAL	Genera automáticamente un identificador numérico para cada pago registrado.
id_reserva	INTEGER	Almacena el identificador de la reserva a la que corresponde el pago, manteniendo compatibilidad con su clave primaria.
fecha_pago	DATE	Permite registrar la fecha en la que se realiza el pago, sin necesidad de almacenar la hora.
monto	NUMERIC(10,2)	Almacena los valores monetarios de los pagos con precisión decimal.
metodo	VARCHAR(20)	Permite registrar los métodos de pago autorizados: Tarjeta, Efectivo y Transferencia.

Consideraciones sobre el almacenamiento y la presentación de fechas

Para los atributos fecha_entrada, fecha_salida y fecha_pago, se utilizará el tipo de dato DATE de PostgreSQL, debido a que permite almacenar fechas válidas y realizar operaciones como calcular la duración de una estancia o identificar reservas dentro de un período determinado.

Se establece el formato **DD/MM/AAAA (día/mes/año)** para la lectura, presentación y documentación de las fechas, considerando que es una convención ampliamente utilizada en Colombia y resulta familiar para los usuarios del sistema. Esta elección facilita la interpretación cronológica de la información por parte del personal administrativo del hotel, especialmente al consultar fechas de ingreso, salida y pagos.

Adicionalmente, mantener un formato uniforme contribuye a prevenir confusiones entre el día y el mes, particularmente cuando ambos valores son inferiores o iguales a 12. Por ejemplo, la fecha 05/11/2026 representa el 5 de noviembre de 2026 bajo la convención seleccionada, mientras que en sistemas que utilizan el formato mes/día/año podría interpretarse como el 11 de mayo de 2026. En un sistema de reservas hoteleras, este tipo de confusión podría ocasionar errores al registrar las estadías, consultar la disponibilidad de habitaciones o calcular los valores correspondientes.

Es importante aclarar que esta decisión corresponde a la forma en que se ingresan y presentan las fechas, sin modificar su almacenamiento interno. PostgreSQL conserva los valores mediante el tipo DATE, independientemente del formato utilizado para visualizarlos. Para evitar interpretaciones ambiguas, se utilizará la función TO_DATE con el patrón DD/MM/YYYY cuando sea necesario convertir una fecha escrita como texto, y TO_CHAR con el mismo patrón para mostrar las fechas en las consultas y reportes.

De esta manera, se mantiene la precisión y funcionalidad del tipo de dato DATE, al mismo tiempo que se garantiza una presentación consistente, comprensible y adaptada al contexto de utilización del sistema hotelero.

Finalmente, las claves primarias, claves foráneas y demás restricciones de integridad se implementarán mediante las sentencias SQL correspondientes, garantizando el cumplimiento de las reglas del sistema hotelero.

1.4. Modificación del modelo físico mediante ALTER TABLE

Durante la implementación del sistema de gestión de reservas del hotel, se identificó la necesidad de incorporar un atributo que permitiera conocer el estado de cada reserva.

Inicialmente, la tabla reserva contaba con cinco atributos: id_reserva, id_cliente, id_habitacion, fecha_entrada y fecha_salida. Sin embargo, esta estructura no permitía diferenciar las reservas confirmadas de las canceladas.

Para solucionar esta necesidad, se utilizó la sentencia ALTER TABLE, mediante la cual se agregó la columna estado_reserva de tipo VARCHAR(20), con la restricción NOT NULL y el valor predeterminado Confirmada.

Esta modificación permite registrar el estado de una reserva sin eliminar su información histórica. Además, representa una evolución del modelo físico, ya que se incorporó una nueva funcionalidad después de la creación inicial de las tablas.

La modificación se realizó mediante el archivo 02_modificacion_estado_reserva.sql, elaborado en Visual Studio Code y ejecutado en PostgreSQL 18 mediante pgAdmin 9.19.

Figura. Incorporación del atributo estado_reserva mediante ALTER TABLE en PostgreSQL.

La ejecución fue exitosa y se verificó que la tabla reserva pasó de cinco a seis columnas. Posteriormente, se implementarán las restricciones necesarias para limitar los estados permitidos y controlar las reservas superpuestas.

1.5. Diferencias entre DROP TABLE y TRUNCATE TABLE

Las sentencias DROP TABLE y TRUNCATE TABLE permiten realizar operaciones sobre las tablas de una base de datos, pero tienen finalidades diferentes.

DROP TABLE

Se utiliza cuando es necesario eliminar completamente una tabla de la base de datos, incluyendo su estructura y registros.

En el contexto del sistema hotelero, podría utilizarse si durante las pruebas se creara una tabla auxiliar que posteriormente dejara de ser necesaria.

Por ejemplo, para eliminar una tabla hipotética llamada registro_prueba:

DROP TABLE registro_prueba;

Después de ejecutar esta instrucción, la tabla dejaría de existir en la base de datos.

TRUNCATE TABLE

Se utiliza cuando se necesita eliminar todos los registros de una tabla, pero conservar su estructura, columnas y restricciones para utilizarla nuevamente.

En el sistema hotelero, podría emplearse durante la etapa de pruebas si se necesitara vaciar una tabla auxiliar antes de cargar un nuevo conjunto de datos sintéticos.

Por ejemplo:

TRUNCATE TABLE registro_prueba;

Después de ejecutar esta instrucción, la tabla permanecería disponible, pero sin registros.

Tabla comparativa

Característica	DROP TABLE	TRUNCATE TABLE
Elimina registros	Sí	Sí
Elimina la estructura	Sí	No
Conserva las columnas	No	Sí
Conserva las restricciones de la tabla	No	Sí
Uso principal	Eliminar tablas innecesarias	Vaciar tablas para reutilizarlas

Conclusión

La elección entre estas sentencias depende del objetivo de la operación. Si una tabla ya no es necesaria, puede utilizarse DROP TABLE. En cambio, si se requiere conservar su estructura para almacenar nuevos registros, puede utilizarse TRUNCATE TABLE.

En el presente proyecto, ambos comandos se presentan mediante ejemplos explicativos y no se ejecutan sobre las tablas principales, con el propósito de proteger la información y conservar la integridad del modelo físico implementado.

2. Pruebas de integridad del sistema hotelero

Con el propósito de verificar el correcto funcionamiento de las restricciones implementadas en PostgreSQL, se realizaron seis pruebas de integridad: tres positivas y tres negativas.

Las pruebas positivas permitieron comprobar que el sistema acepta información válida, mientras que las negativas verificaron que PostgreSQL rechaza operaciones que incumplen las reglas establecidas para el hotel.

Todas las pruebas se desarrollaron mediante el archivo 05_pruebas_integridad.sql, elaborado en Visual Studio Code y ejecutado en pgAdmin 9.19 sobre la base de datos hotel_reservas.

Se utilizaron transacciones mediante BEGIN y ROLLBACK para evitar que los registros temporales permanecieran almacenados después de las verificaciones.

2.1. Pruebas positivas

1. Primera prueba positiva: Registro de un cliente válido

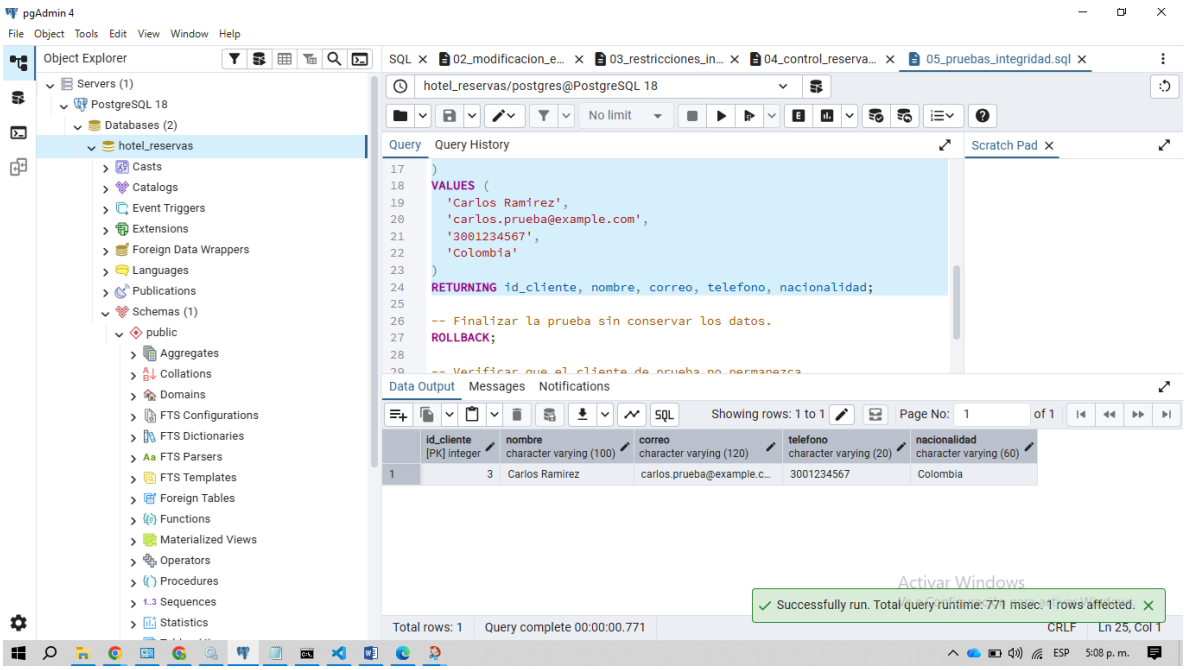
Descripción: Se realizó la inserción de un cliente ficticio llamado Carlos Ramírez, proporcionando su nombre, correo electrónico, número telefónico y nacionalidad.

Objetivo: Comprobar que PostgreSQL permite registrar correctamente un cliente cuando la información suministrada cumple con los tipos de datos y las restricciones definidas en la tabla cliente.

Resultado obtenido: La inserción se ejecutó correctamente y PostgreSQL generó el identificador id_cliente = 3, mostrando los datos registrados. Posteriormente, se ejecutó ROLLBACK y se realizó una consulta de verificación que devolvió cero registros asociados al correo utilizado en la prueba.

Por lo tanto, se confirmó que el sistema permite registrar clientes válidos y que la transacción puede revertirse sin conservar los datos temporales.

Resultado final: Prueba exitosa.



---

2. Segunda prueba positiva: Registro de una habitación válida

Descripción: Se realizó la inserción de una habitación ficticia identificada con el número PR901, de tipo Doble, con un precio por noche de $220.000 y estado Disponible.

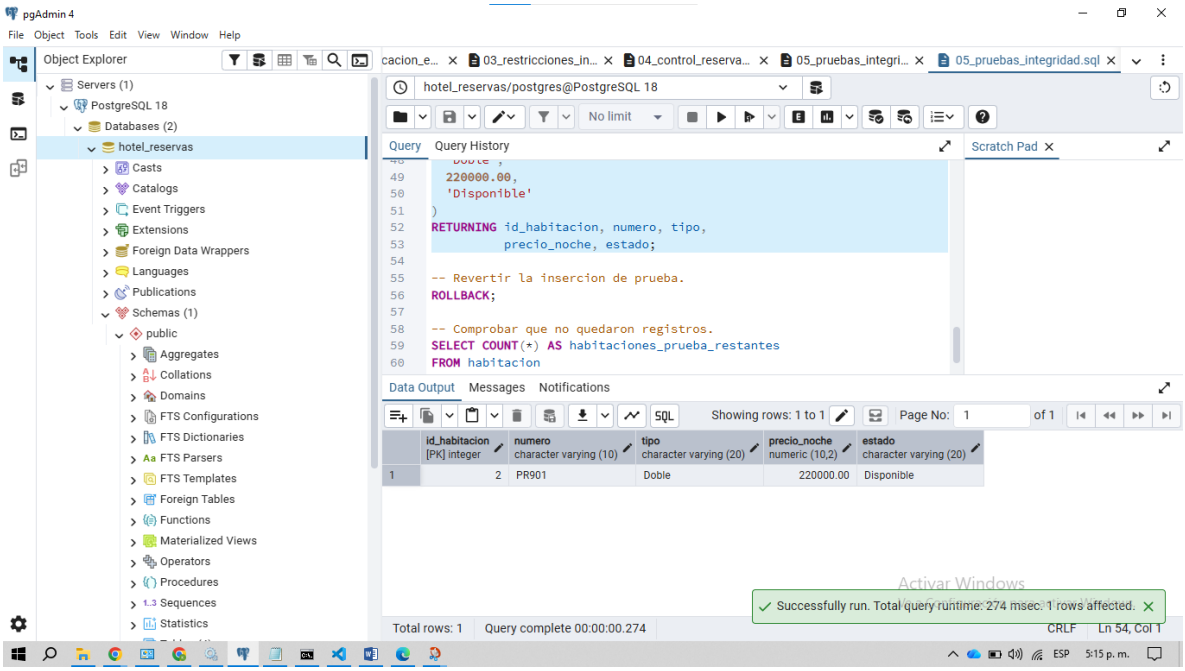
Objetivo: Verificar que PostgreSQL permite registrar habitaciones cuyos valores cumplen con las restricciones CHECK, especialmente las relacionadas con el tipo de habitación, el precio por noche y el estado.

Resultado obtenido: PostgreSQL aceptó la inserción y asignó a la habitación el identificador id_habitacion = 2. Esto permitió comprobar que los valores ingresados cumplían las condiciones establecidas.

Posteriormente, se ejecutó ROLLBACK y se consultó nuevamente la tabla, obteniendo cero registros asociados a la habitación PR901.

De esta manera, se comprobó que el sistema acepta habitaciones con información válida y permite revertir los registros utilizados durante las pruebas.

Resultado final: Prueba exitosa.



---

3. Tercera prueba positiva: Registro de una reserva válida

Descripción: Se crearon temporalmente un cliente ficticio llamado Laura Torres y una habitación identificada como PR902, de tipo Suite, con un precio de $350.000 por noche.

Posteriormente, se registró una reserva con fecha de entrada del 20/11/2026 y fecha de salida del 23/11/2026, equivalente a tres noches de alojamiento.

Objetivo: Comprobar que PostgreSQL permite registrar una reserva cuando el cliente y la habitación existen, las fechas son coherentes y no se presentan conflictos de superposición con otras reservas confirmadas.

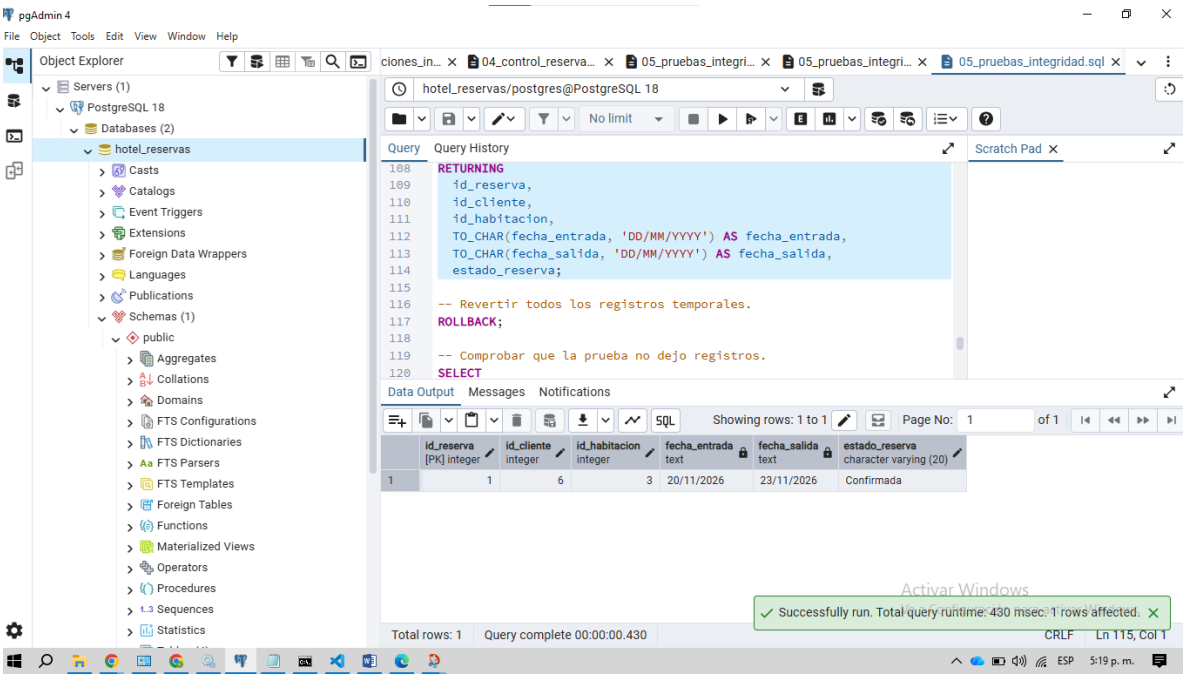
Resultado obtenido: La reserva fue registrada correctamente con el identificador id_reserva = 1, asociada al cliente y a la habitación creados temporalmente. PostgreSQL mostró las fechas establecidas y el estado Confirmada.

El valor calculado de la estancia corresponde a $1.050.000, obtenido mediante la multiplicación de tres noches por $350.000.

Posteriormente, se ejecutó ROLLBACK y se verificó que los conteos de clientes, habitaciones y reservas temporales fueran iguales a cero.

Esta prueba permitió comprobar tanto el funcionamiento de las relaciones entre las tablas como la aceptación de fechas válidas y la reversión de operaciones relacionadas.

Resultado final: Prueba exitosa.



2.2. Pruebas negativas

4. Primera prueba negativa: Rechazo de una reserva con fechas incorrectas

Descripción: Se intentó registrar una reserva para un cliente y una habitación creados temporalmente, estableciendo como fecha de entrada el 25/11/2026 y como fecha de salida el 23/11/2026.

La operación es incorrecta porque la fecha de salida es anterior a la fecha de entrada.

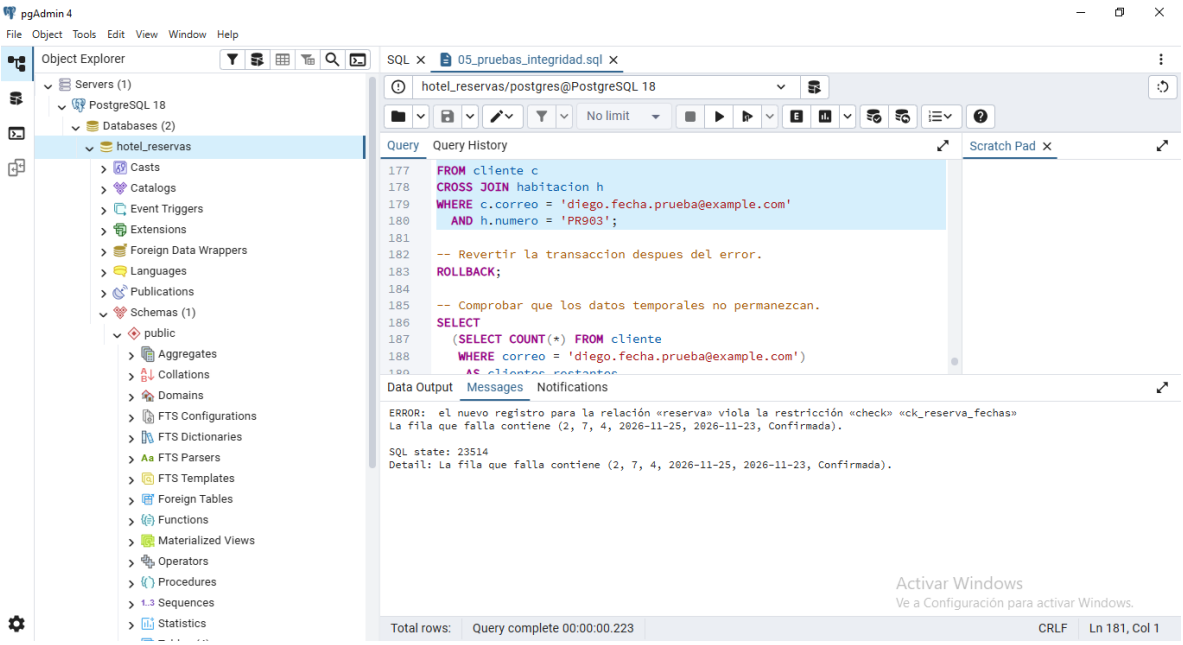
Objetivo: Verificar que la restricción ck_reserva_fechas impide almacenar reservas con períodos de alojamiento inválidos, garantizando que la fecha de salida siempre sea posterior a la fecha de entrada.

Resultado obtenido: PostgreSQL rechazó la operación y presentó el código de error SQL 23514, indicando que el registro incumplía la restricción CHECK denominada ck_reserva_fechas.

Posteriormente, se ejecutó ROLLBACK y se verificó que los registros temporales del cliente y la habitación no permanecieran almacenados, obteniendo cero en ambos conteos.

Este resultado demuestra que el sistema identifica y rechaza fechas inconsistentes, evitando registrar estadías con duraciones inválidas.

Resultado final: Prueba exitosa, operación rechazada correctamente.



---

5. Segunda prueba negativa: Rechazo de un pago con monto negativo

Descripción: Se creó temporalmente una reserva válida y posteriormente se intentó registrar un pago mediante Transferencia por un valor de -$50.000.

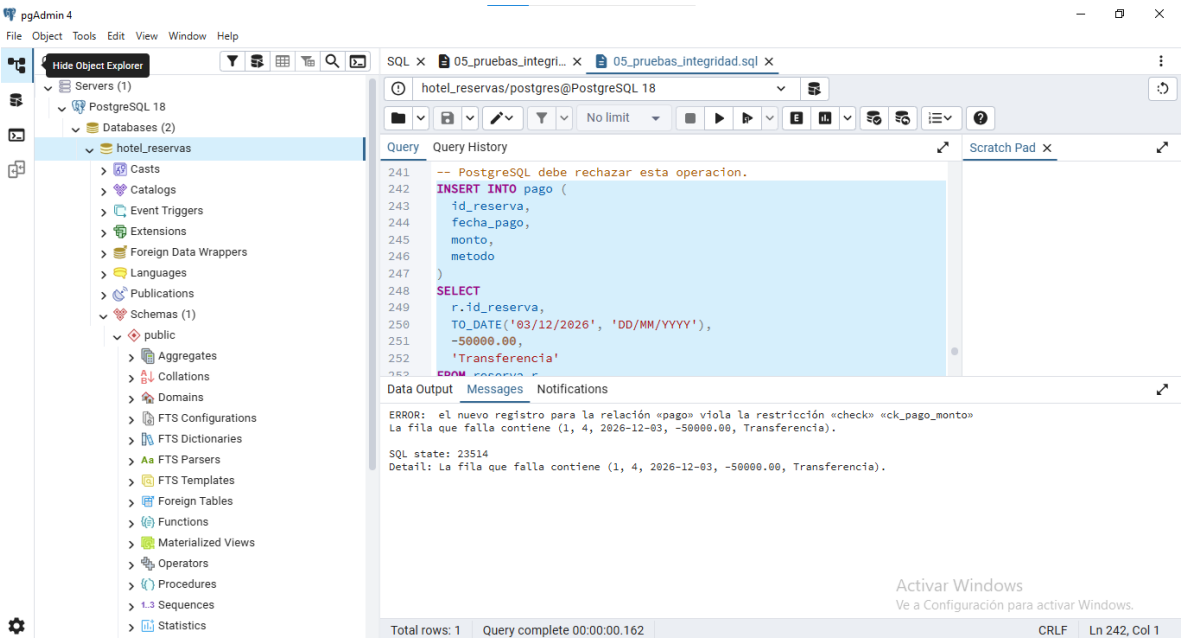
Objetivo: Comprobar que la restricción ck_pago_monto impide registrar pagos con valores iguales o inferiores a cero, protegiendo la integridad de la información financiera del hotel.

Resultado obtenido: PostgreSQL rechazó la inserción del pago y generó el código de error SQL 23514, correspondiente al incumplimiento de la restricción CHECK denominada ck_pago_monto.

Después del error, se realizó la reversión de la transacción y se verificó que no permanecieran registros temporales de clientes, habitaciones ni pagos inválidos. Los tres conteos devolvieron cero.

Esta prueba permitió demostrar que el motor de base de datos valida los montos antes de aceptar un pago, evitando el almacenamiento de valores que incumplen las reglas establecidas.

Resultado final: Prueba exitosa, operación rechazada correctamente.



---

6. Tercera prueba negativa: Rechazo de reservas superpuestas

Descripción: Se creó temporalmente un cliente y una habitación identificada como PR905, de tipo Doble.

Posteriormente, se registró una primera reserva confirmada con fecha de entrada del 10/12/2026 y salida del 13/12/2026.

Después, se intentó registrar una segunda reserva confirmada para la misma habitación, con fecha de entrada del 12/12/2026 y salida del 15/12/2026.

Los períodos presentan una superposición, debido a que ambas reservas incluyen la noche del 12/12/2026.

Objetivo: Verificar que la restricción ex_reserva_sin_superposicion, implementada mediante EXCLUDE, daterange y la extensión btree_gist, impide que una habitación tenga dos reservas confirmadas cuyos períodos coincidan.

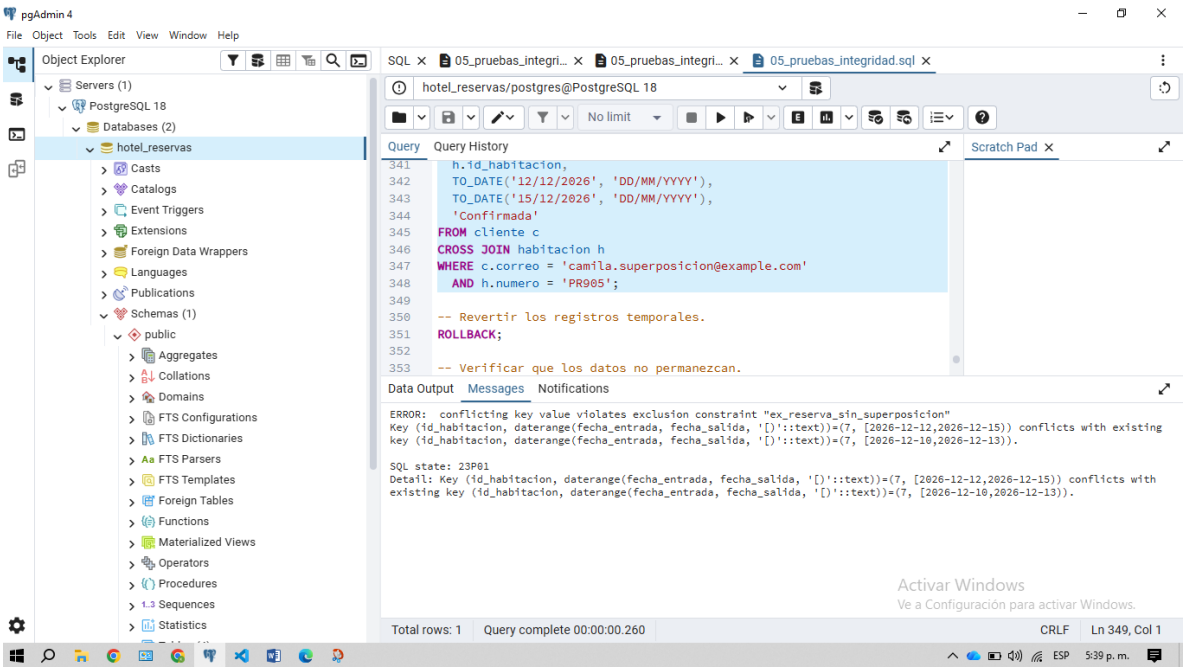
Resultado obtenido: PostgreSQL rechazó la segunda reserva y generó el código de error SQL 23P01, indicando un conflicto con la restricción de exclusión ex_reserva_sin_superposicion.

El mensaje identificó que la habitación con id_habitacion = 7 presentaba una coincidencia entre los períodos de las dos reservas.

Posteriormente, se revirtió la transacción y se comprobó que los registros temporales de clientes, habitaciones y reservas no permanecieran almacenados. Las tres consultas de verificación devolvieron cero.

Con esta prueba se demostró que el sistema evita automáticamente la superposición de reservas confirmadas para una misma habitación, contribuyendo a mantener la disponibilidad y consistencia de las operaciones del hotel.

Resultado final: Prueba exitosa, superposición rechazada correctamente.



2.3. Conclusión de las pruebas de integridad

Las seis pruebas desarrolladas permitieron verificar el funcionamiento de las restricciones de integridad implementadas en la base de datos del sistema hotelero.

Las tres pruebas positivas demostraron que PostgreSQL acepta operaciones válidas relacionadas con el registro de clientes, habitaciones y reservas. Por otra parte, las tres pruebas negativas comprobaron que el sistema rechaza fechas inconsistentes, pagos negativos y reservas confirmadas con períodos superpuestos.

Asimismo, la utilización de transacciones y de la instrucción ROLLBACK permitió ejecutar las verificaciones sin conservar los datos temporales.

En conclusión, los resultados obtenidos evidencian que las restricciones evaluadas contribuyen a proteger la consistencia de la información y al cumplimiento de las principales reglas de negocio del hotel.

2.4. Definición y justificación de las acciones referenciales

Las acciones referenciales permiten establecer el comportamiento de las relaciones entre las tablas cuando se actualizan o eliminan registros que contienen claves primarias referenciadas mediante claves foráneas.

En el sistema de gestión de reservas del hotel se implementaron las acciones ON UPDATE CASCADE y ON DELETE RESTRICT, con el propósito de mantener la integridad referencial y evitar la pérdida accidental de información relacionada.

Estas acciones fueron definidas desde la creación inicial del modelo físico, mediante el archivo 01_creacion_modelo_fisico.sql.

2.4.1. Acciones referenciales implementadas

Relación	Clave foránea	Actualización	Eliminación
cliente → reserva	reserva.id_cliente	CASCADE	RESTRICT
habitacion → reserva	reserva.id_habitacion	CASCADE	RESTRICT
reserva → pago	pago.id_reserva	CASCADE	RESTRICT

2.4.2. Justificación de las acciones referenciales

1. Relación entre cliente y reserva

Se implementó ON UPDATE CASCADE para garantizar que, si el identificador de un cliente se modifica, sus reservas asociadas actualicen automáticamente la referencia correspondiente.

Por otra parte, se estableció ON DELETE RESTRICT para impedir que un cliente sea eliminado mientras tenga reservas registradas.

Esta decisión permite proteger la relación entre los clientes y sus reservas, evitando que existan registros asociados a clientes inexistentes.

Ejemplo: Si un cliente tiene tres reservas registradas y se intenta eliminar su información, PostgreSQL rechazará la operación mientras existan dichas reservas.

2. Relación entre habitación y reserva

Se utilizó ON UPDATE CASCADE para mantener actualizadas las referencias de las reservas cuando se modifica el identificador de una habitación.

Asimismo, se implementó ON DELETE RESTRICT para impedir que una habitación sea eliminada si existen reservas relacionadas con ella.

Esta decisión permite conservar la información de las habitaciones utilizadas en las reservas y evita eliminar accidentalmente registros que forman parte del historial del hotel.

Ejemplo: Si una habitación tiene reservas registradas, PostgreSQL impedirá eliminarla hasta que se resuelva la existencia de esas relaciones.

3. Relación entre reserva y pago

Se implementó ON UPDATE CASCADE para garantizar que los pagos asociados permanezcan relacionados correctamente si se modifica el identificador de una reserva.

También se estableció ON DELETE RESTRICT para impedir que se elimine una reserva cuando existen pagos asociados.

Esta decisión es especialmente importante porque los pagos representan información financiera del hotel y deben conservar su relación con la reserva correspondiente.

Ejemplo: Si una reserva tiene dos pagos registrados, PostgreSQL rechazará cualquier intento de eliminarla directamente mientras dichos pagos continúen asociados.

En situaciones de cancelación, el sistema permitirá modificar el atributo estado_reserva a Cancelada, evitando eliminar innecesariamente los registros históricos.

2.4.3. Conclusión

La implementación de las acciones referenciales ON UPDATE CASCADE y ON DELETE RESTRICT permite mantener la coherencia de las relaciones entre clientes, habitaciones, reservas y pagos.

Aunque los identificadores primarios normalmente no deberían modificarse durante la operación habitual del hotel, ON UPDATE CASCADE proporciona un mecanismo para mantener las referencias cuando se presentan modificaciones autorizadas.

Por su parte, ON DELETE RESTRICT evita eliminar registros que tienen información dependiente, contribuyendo a proteger los datos históricos y financieros del sistema.

De esta manera, las acciones referenciales complementan las restricciones de integridad implementadas y fortalecen la consistencia del modelo físico desarrollado en PostgreSQL.

3. Datos de prueba y datos sintéticos generados con inteligencia artificial

3.1. Generación de datos sintéticos

Para comprobar el funcionamiento del sistema de gestión de reservas del hotel, se utilizó inteligencia artificial generativa como apoyo para elaborar un conjunto de datos ficticios que representara diferentes situaciones de la operación hotelera.

Los registros se organizaron en cuatro tablas: cliente, habitacion, reserva y pago, siguiendo las cantidades mínimas solicitadas en el taller.

La generación se realizó considerando los atributos, tipos de datos, relaciones y restricciones implementadas previamente en PostgreSQL.

Tabla. Distribución de los datos sintéticos generados

Tabla	Registros generados
cliente	20
habitacion	10
reserva	30
pago	30
Total	**90**

Los datos fueron preparados mediante instrucciones SQL almacenadas en el archivo 06_carga_datos_sinteticos.sql, elaborado en Visual Studio Code.

3.2. Generación e inserción de clientes

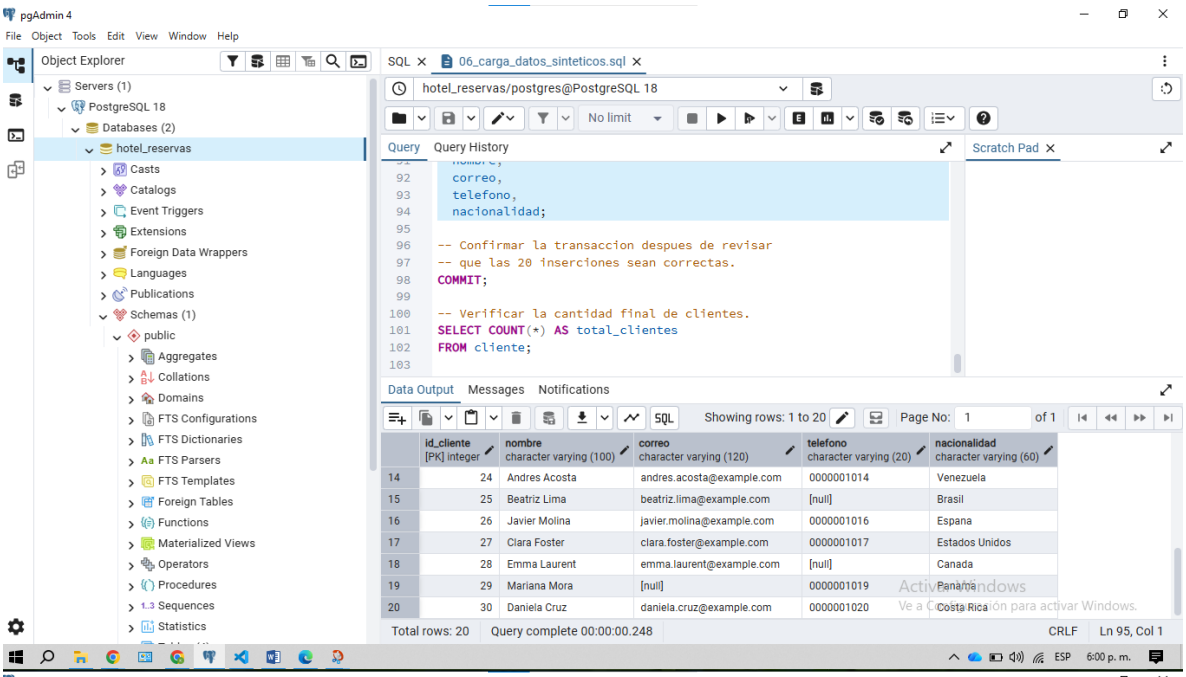
Se generaron 20 clientes ficticios con nombres, correos electrónicos, números telefónicos y diferentes nacionalidades.

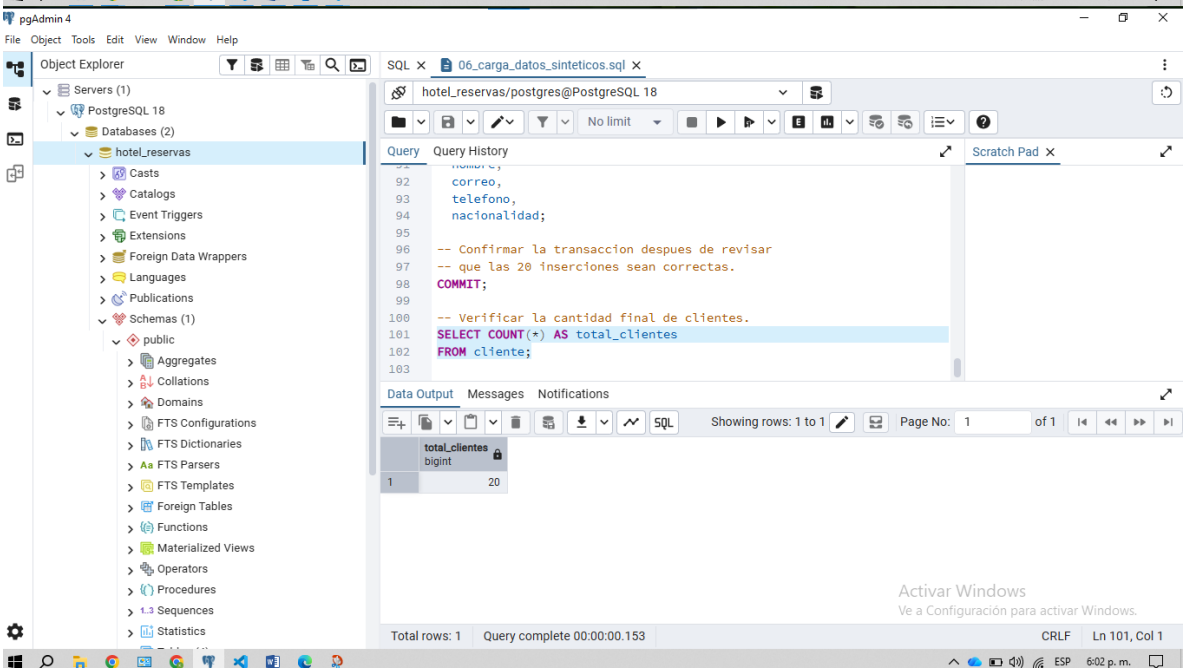
Para evitar utilizar información personal real, se emplearon direcciones del dominio example.com y números telefónicos ficticios.

Además, algunos registros contienen valores NULL en los atributos opcionales, permitiendo representar situaciones en las que no se dispone de toda la información de contacto.

La inserción fue realizada correctamente mediante la sentencia INSERT INTO, y PostgreSQL asignó automáticamente los identificadores utilizando el tipo SERIAL.

Posteriormente, se verificó que la tabla contenía exactamente 20 clientes.





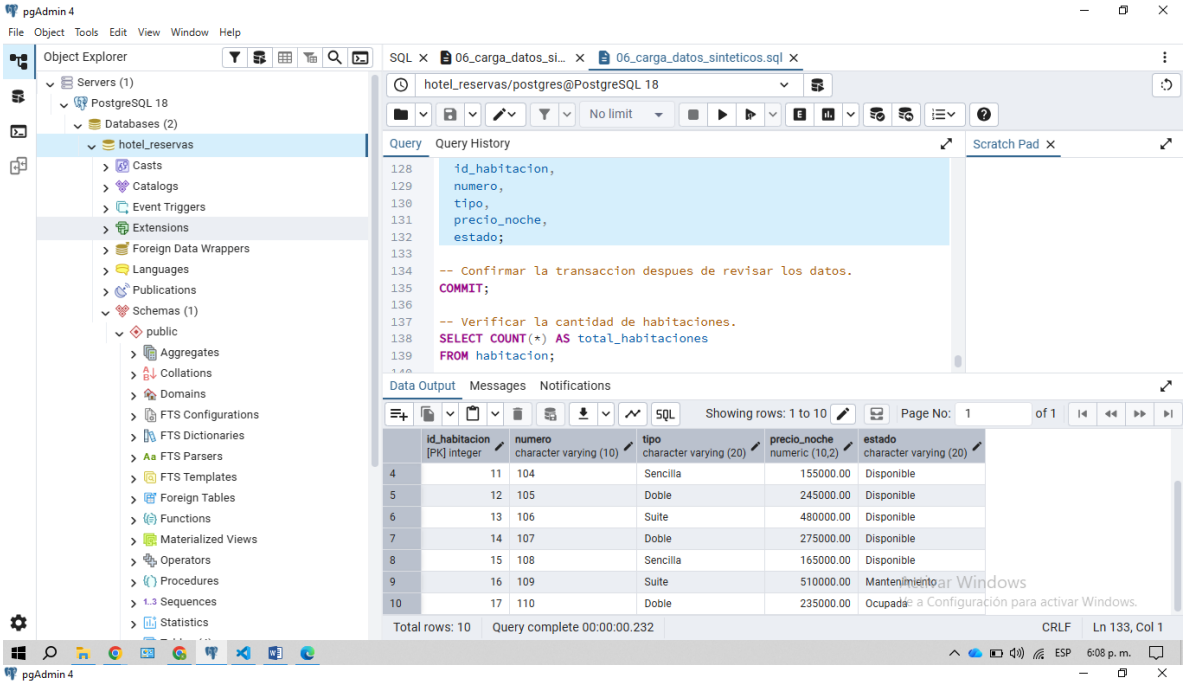
3.3. Generación e inserción de habitaciones

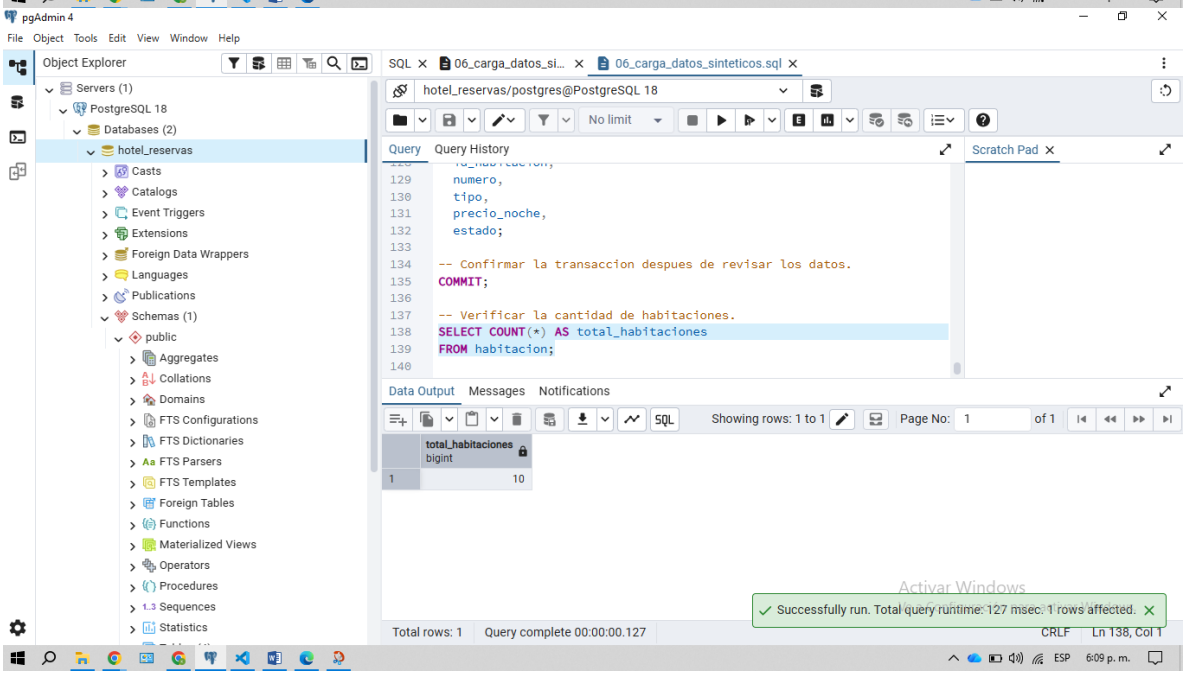
Se generaron diez habitaciones ficticias identificadas con números del 101 al 110.

Los registros incluyen los tres tipos de habitación establecidos: Sencilla, Doble y Suite. Se asignaron diferentes precios por noche y estados de disponibilidad, ocupación o mantenimiento.

Los datos se prepararon respetando las restricciones CHECK implementadas para el tipo de habitación, el precio por noche y el estado.

La inserción se realizó correctamente y la consulta de verificación confirmó un total de diez habitaciones almacenadas.





3.4. Generación e inserción de reservas

Se generaron treinta reservas distribuidas entre octubre, noviembre y diciembre de 2026.

Para establecer las relaciones con las tablas existentes, se utilizaron consultas JOIN que permitieron obtener los identificadores de clientes y habitaciones correspondientes.

Se prepararon las fechas siguiendo el formato DD/MM/AAAA y se utilizó la función TO_DATE para convertirlas al tipo DATE de PostgreSQL.

Las reservas se distribuyeron en diferentes habitaciones y períodos, procurando representar situaciones útiles para los reportes y consultas del hotel.

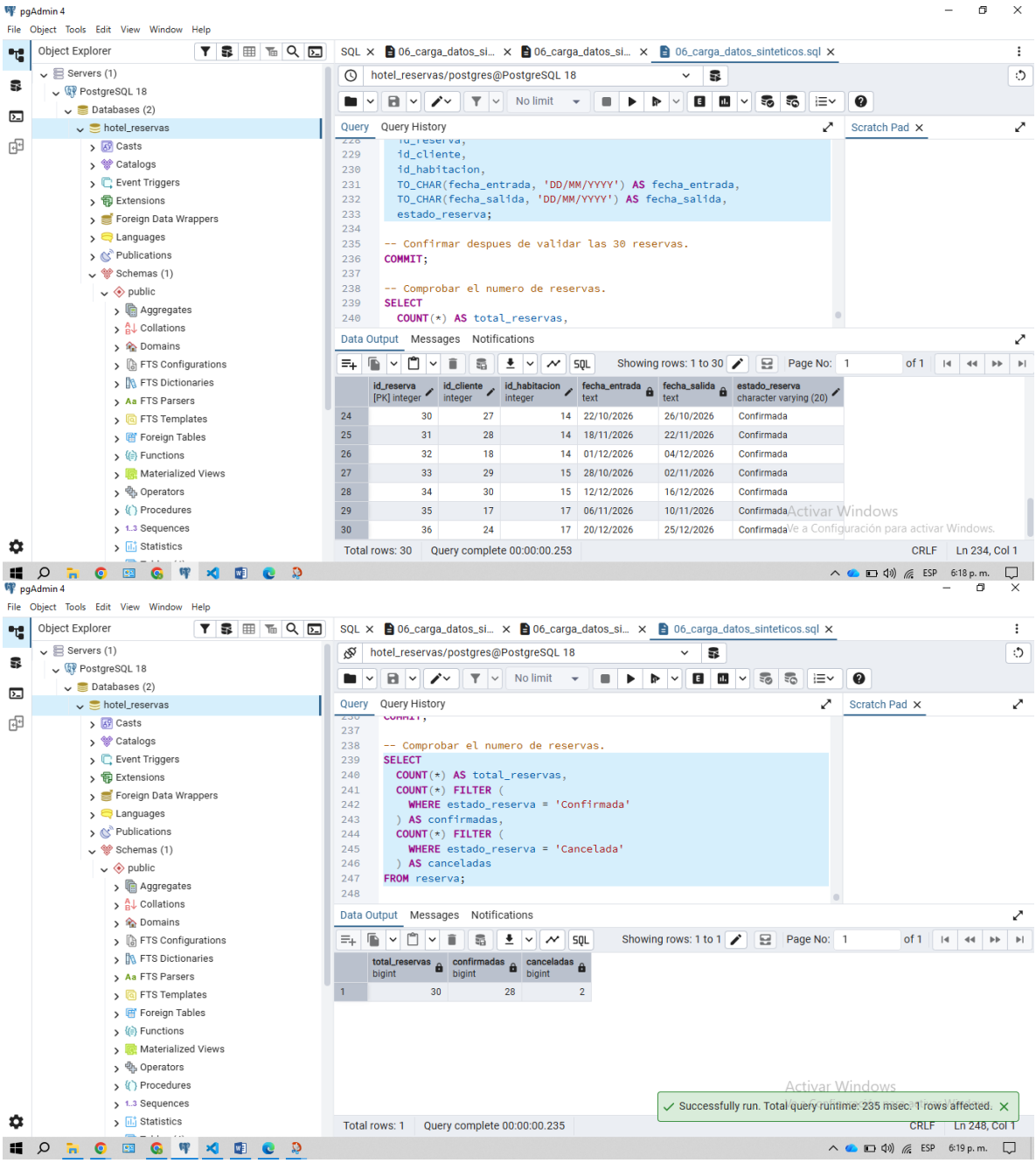
La habitación 101 recibió siete reservas, mientras que la habitación 109 se dejó sin reservas para permitir su identificación mediante consultas posteriores.

Se incluyeron reservas confirmadas y canceladas, respetando las restricciones del modelo.

La ejecución fue exitosa y PostgreSQL confirmó los siguientes resultados:

Estado	Cantidad
Confirmada	28
Cancelada	2
Total	**30**

La correcta ejecución de las inserciones permitió verificar que las reservas propuestas fueron aceptadas por las restricciones de integridad implementadas.



3.5. Generación e inserción de pagos

Se generaron treinta pagos ficticios asociados con reservas previamente registradas.

Los pagos fueron distribuidos entre los métodos Tarjeta, Efectivo y Transferencia, utilizando montos positivos y diferentes fechas.

Se incluyeron reservas con varios pagos parciales y otras sin pagos, con el propósito de representar diferentes situaciones financieras del hotel.

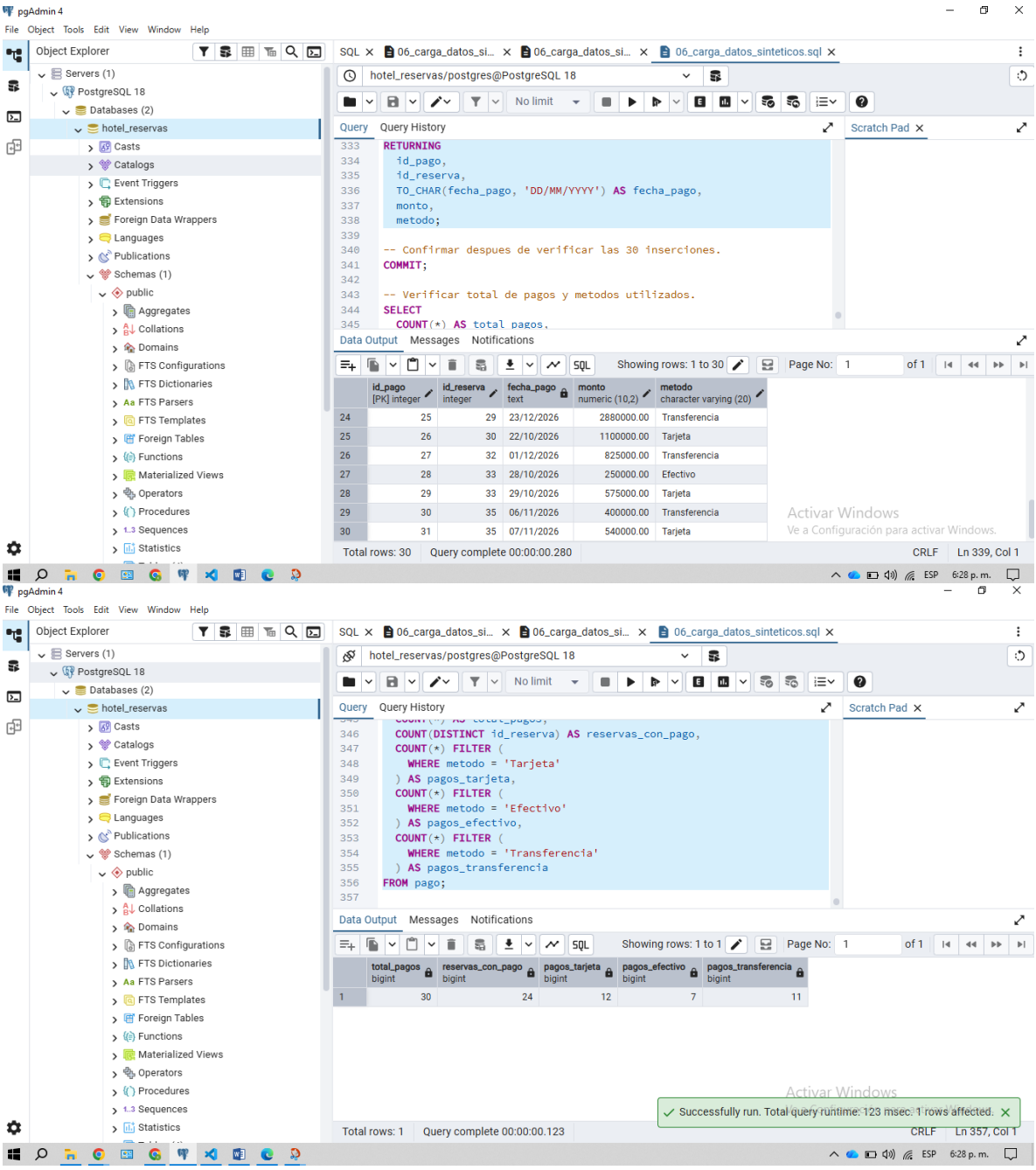
Para relacionar cada pago con su reserva correspondiente, se utilizaron consultas JOIN entre las tablas habitacion y reserva.

Una vez ejecutadas las inserciones, se verificó que PostgreSQL almacenó correctamente los treinta pagos.

Los resultados fueron:

Método de pago	**Cantidad**
Tarjeta	12
Efectivo	7
Transferencia	11
Total	**30**

Adicionalmente, se comprobó que 24 reservas tenían al menos un pago asociado.



3.6. Validación de la integridad de los datos generados

Después de generar los datos con apoyo de inteligencia artificial, se revisó su compatibilidad con la estructura del sistema hotelero.

La validación contempló la coherencia de las fechas, la existencia de los registros referenciados, el cumplimiento de las restricciones de integridad y la ausencia de conflictos entre reservas confirmadas.

Adicionalmente, se ejecutó una consulta SQL para identificar reservas sin pagos y pagos acumulados superiores al valor calculado de cada estancia.

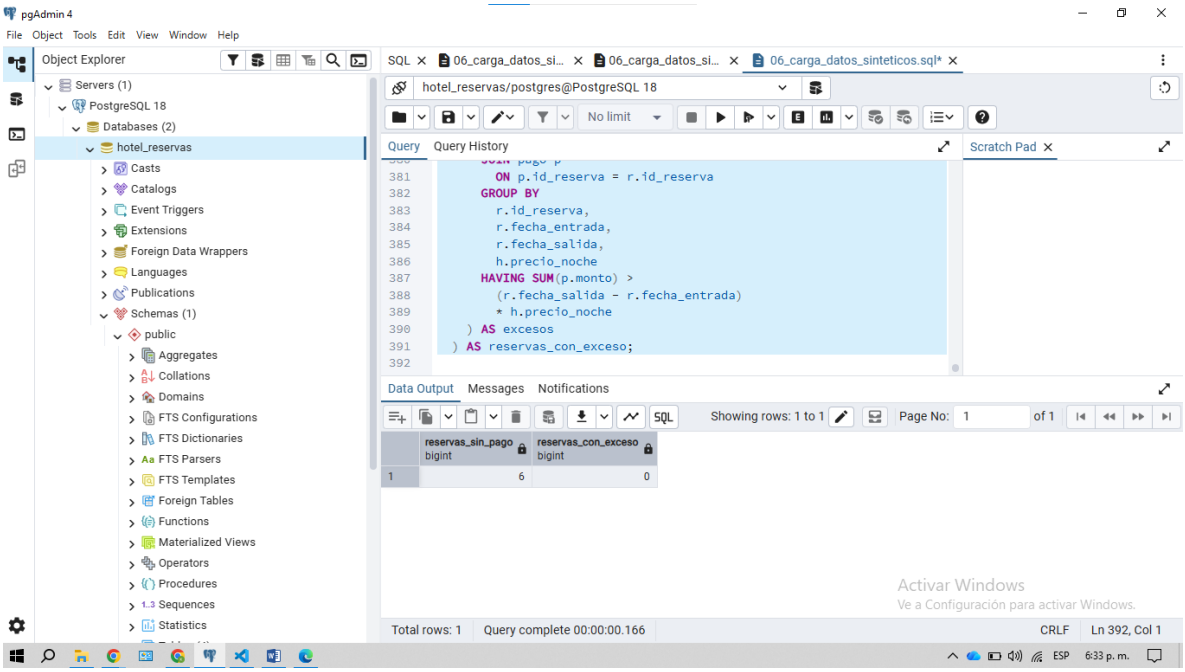
Los resultados fueron los siguientes:

Validación	Resultado
Reservas sin pagos	6
Reservas con exceso de pago	0

Las seis reservas sin pagos representan situaciones que podrán analizarse en las consultas posteriores.

Por otra parte, no se identificaron reservas cuyos pagos acumulados superaran el valor correspondiente a las noches de alojamiento.

Durante las verificaciones de carga presentadas no se evidenciaron errores de integridad que exigieran modificar los registros generados.



3.7. Conclusión

La generación y carga de los datos sintéticos permitió comprobar el funcionamiento del modelo físico implementado en PostgreSQL.

Se almacenaron 90 registros ficticios distribuidos entre clientes, habitaciones, reservas y pagos, respetando las relaciones y restricciones previamente definidas.

Las verificaciones realizadas confirmaron las cantidades esperadas y permitieron comprobar que los pagos registrados no superan los valores calculados de las estancias.

Finalmente, este conjunto de datos constituye la base para desarrollar las consultas SQL, reportes estadísticos, vistas y transacciones solicitadas en las siguientes partes del taller.

4. Parte 4. Manipulación de datos y consultas SQL

4.1. Implementación de operaciones DML

Con el objetivo de comprobar la manipulación de información en el sistema de gestión de reservas del hotel, se desarrollaron tres escenarios mediante instrucciones del lenguaje de manipulación de datos (DML).

Estas operaciones permitieron modificar el estado de una habitación, registrar un nuevo pago y cancelar una reserva, utilizando las sentencias UPDATE, INSERT y SELECT.

Las pruebas se implementaron en el archivo 07_operaciones_dml.sql, elaborado en Visual Studio Code y ejecutado en PostgreSQL 18 mediante pgAdmin 9.19.

Para conservar los 90 registros sintéticos originales, las operaciones se desarrollaron mediante transacciones con BEGIN y ROLLBACK.

4.1.1. Primer escenario: Actualización del estado de una habitación

Se simuló la finalización del mantenimiento de la habitación 109, de tipo Suite, que inicialmente se encontraba en estado Mantenimiento.

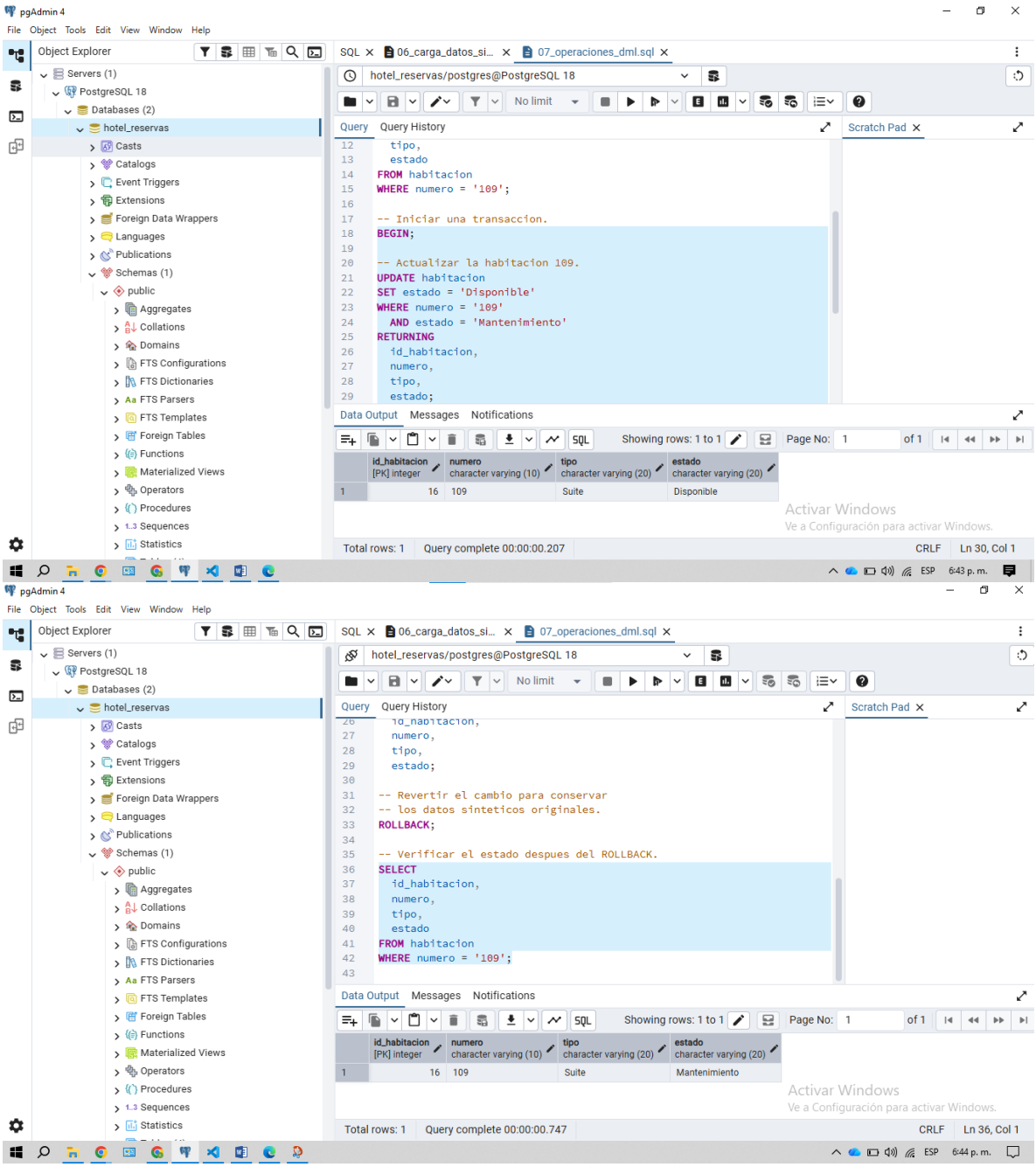
Objetivo: Verificar que PostgreSQL permite modificar el estado de una habitación existente utilizando la sentencia UPDATE, respetando los valores permitidos por las restricciones de integridad.

Procedimiento: Se consultó el estado inicial de la habitación y posteriormente se ejecutó una actualización para cambiarlo a Disponible. La instrucción utilizó la cláusula WHERE para identificar específicamente la habitación y evitar modificaciones sobre otros registros.

Resultado obtenido: PostgreSQL ejecutó correctamente la actualización y mostró la habitación 109 en estado Disponible.

Posteriormente, se ejecutó ROLLBACK y se realizó una nueva consulta, comprobando que la habitación recuperó su estado original Mantenimiento.

Resultado final: Operación exitosa y cambio revertido correctamente.



4.1.2. Segundo escenario: Registro de un nuevo pago

Se simuló el registro de un pago parcial asociado con la reserva 19, correspondiente a la habitación 103, con fecha de entrada del 13/11/2026 y salida del 18/11/2026.

La estancia tenía una duración de cinco noches, con un costo total calculado de $2.100.000.

Objetivo: Comprobar que PostgreSQL permite registrar un nuevo pago asociado con una reserva existente, manteniendo la integridad referencial entre las tablas reserva y pago.

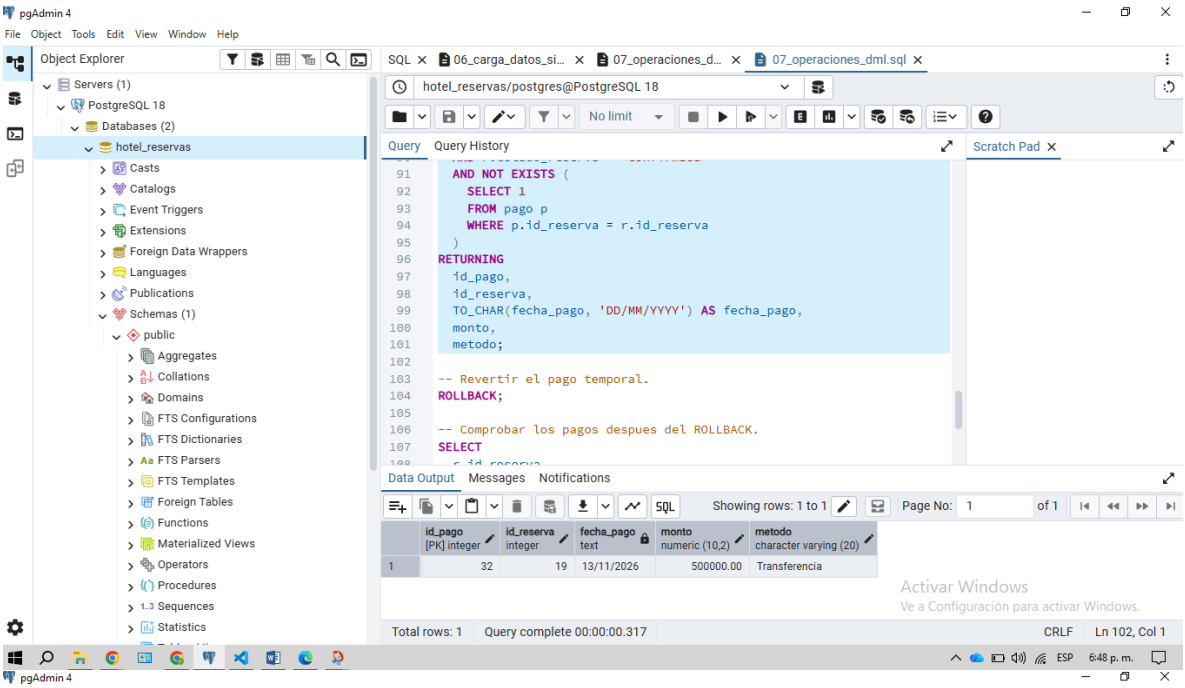
Procedimiento: Inicialmente se consultó la reserva para verificar su existencia y confirmar que no tenía pagos registrados. Posteriormente, se ejecutó una sentencia INSERT INTO para registrar un pago de $500.000 mediante Transferencia.

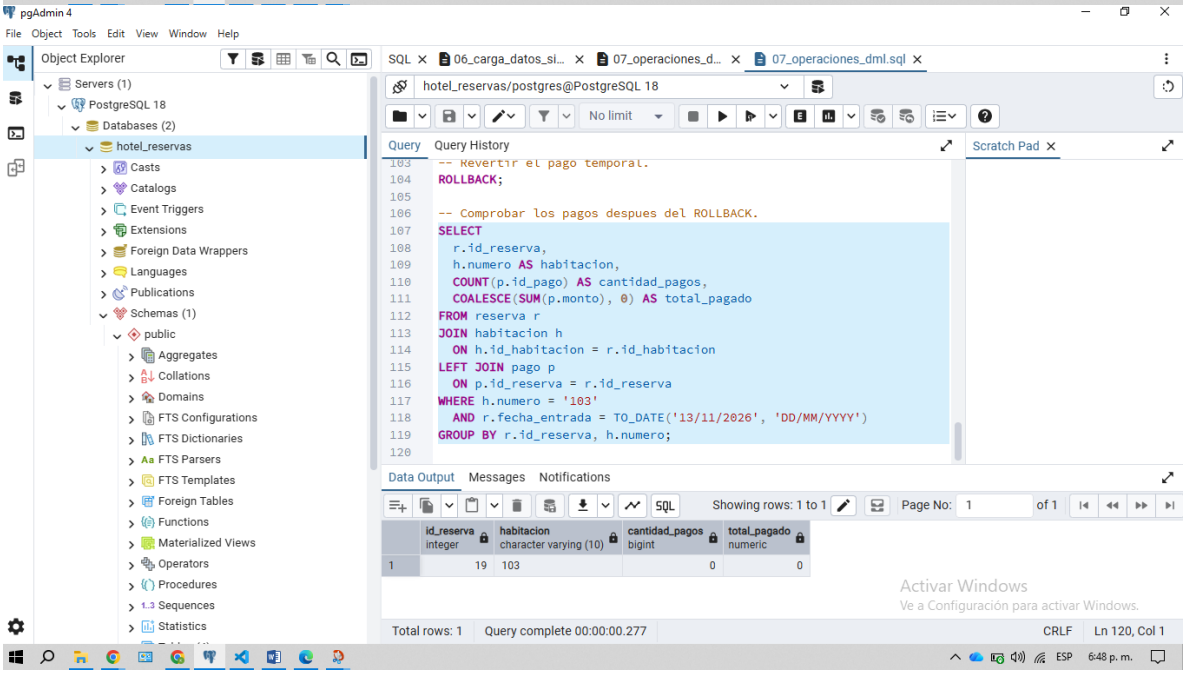
Resultado obtenido: PostgreSQL aceptó correctamente la operación y generó el identificador id_pago = 32, relacionado con la reserva 19.

El pago fue registrado con fecha del 13/11/2026 y un monto de $500.000.

Posteriormente, se ejecutó ROLLBACK y se comprobó mediante una consulta que la reserva volvió a tener cero pagos registrados y un total pagado de $0.

Resultado final: Operación exitosa y pago temporal revertido correctamente.





4.1.3. Tercer escenario: Cancelación de una reserva

Se simuló la cancelación de la reserva 19, correspondiente a la habitación 103, cuya fecha de entrada era el 13/11/2026 y la fecha de salida el 18/11/2026.

Inicialmente, la reserva se encontraba en estado Confirmada y no tenía pagos registrados.

Objetivo: Verificar que PostgreSQL permite cancelar una reserva mediante la actualización del atributo estado_reserva, sin eliminar el registro ni perder su información histórica.

Procedimiento: Primero se consultó la reserva para comprobar su estado y la ausencia de pagos. Después, se ejecutó una sentencia UPDATE para modificar el estado de Confirmada a Cancelada.

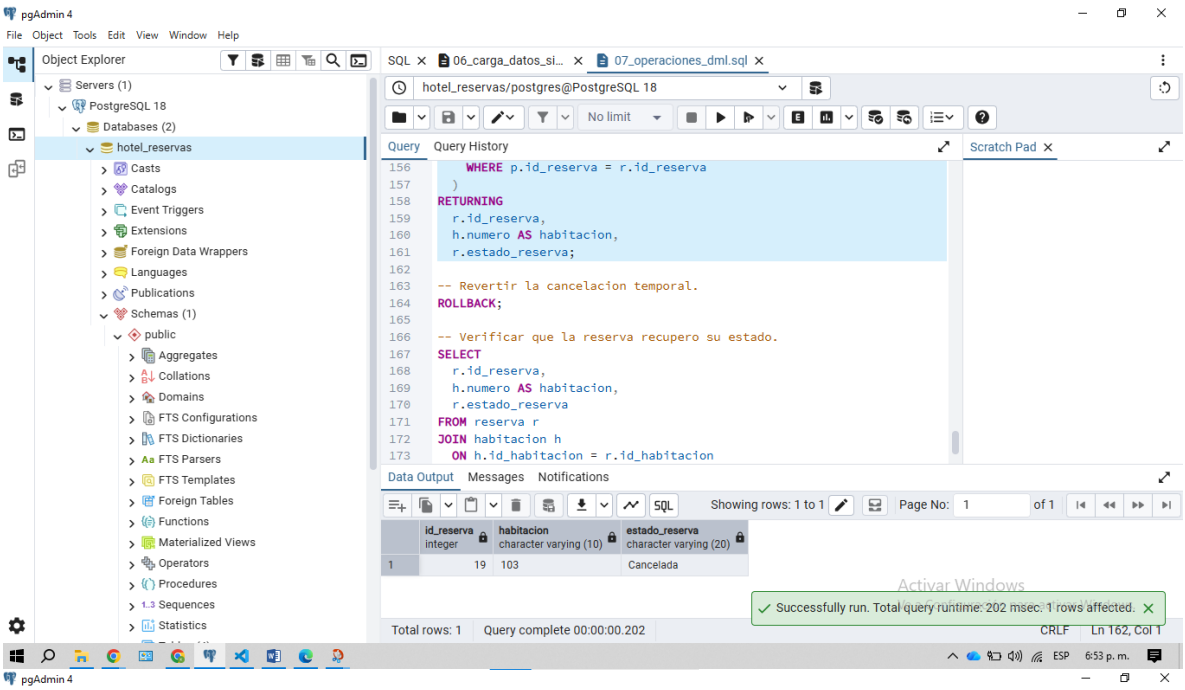
La operación incluyó condiciones para identificar la reserva y verificar que no tuviera pagos asociados.

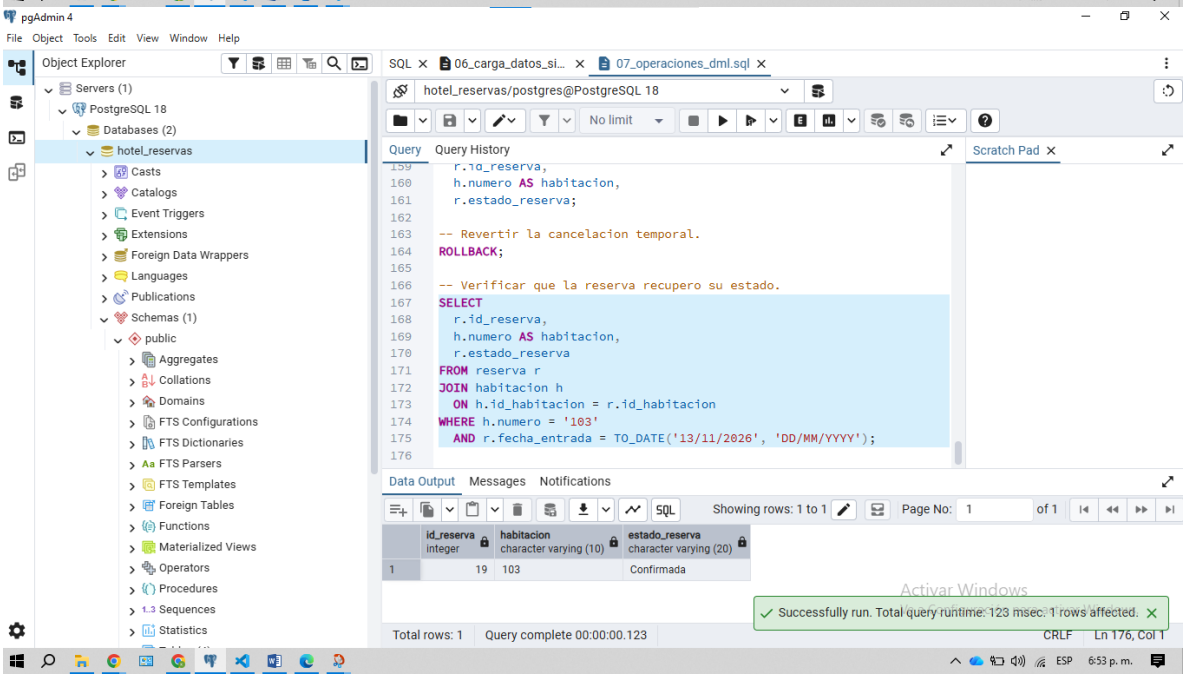
Resultado obtenido: PostgreSQL ejecutó correctamente la actualización y mostró la reserva 19 en estado Cancelada.

Posteriormente, se ejecutó ROLLBACK y se consultó nuevamente la reserva, comprobando que recuperó el estado original Confirmada.

Esta operación demuestra que la cancelación puede representarse mediante un cambio de estado, conservando las relaciones y la información histórica del sistema.

Resultado final: Operación exitosa y cancelación temporal revertida correctamente.





4.1.4. Conclusión de las operaciones DML

Los tres escenarios desarrollados permitieron verificar el funcionamiento de las instrucciones DML en la base de datos del sistema hotelero.

Se comprobó que PostgreSQL permite actualizar el estado de una habitación, registrar pagos relacionados con reservas existentes y modificar el estado de una reserva sin eliminar su información.

La utilización de transacciones permitió realizar las operaciones de manera controlada y revertir los cambios mediante ROLLBACK, evitando modificaciones permanentes en los datos sintéticos.

En conclusión, los resultados obtenidos demuestran que el modelo implementado permite realizar operaciones básicas de gestión hotelera, respetando las relaciones y restricciones de integridad establecidas.

4.2. Consultas básicas, filtros y ordenamiento

Con el propósito de verificar la recuperación de información en el sistema de gestión de reservas para un hotel, se desarrollaron tres consultas básicas utilizando la sentencia SELECT y diferentes condiciones de filtrado.

Estas consultas permitieron identificar las habitaciones disponibles, las reservas que se encontraban activas y las estadías cuya duración superaba tres noches.

Los códigos fueron organizados en el archivo 08_consultas_basicas.sql, elaborado en Visual Studio Code y ejecutado en PostgreSQL 18 mediante pgAdmin.

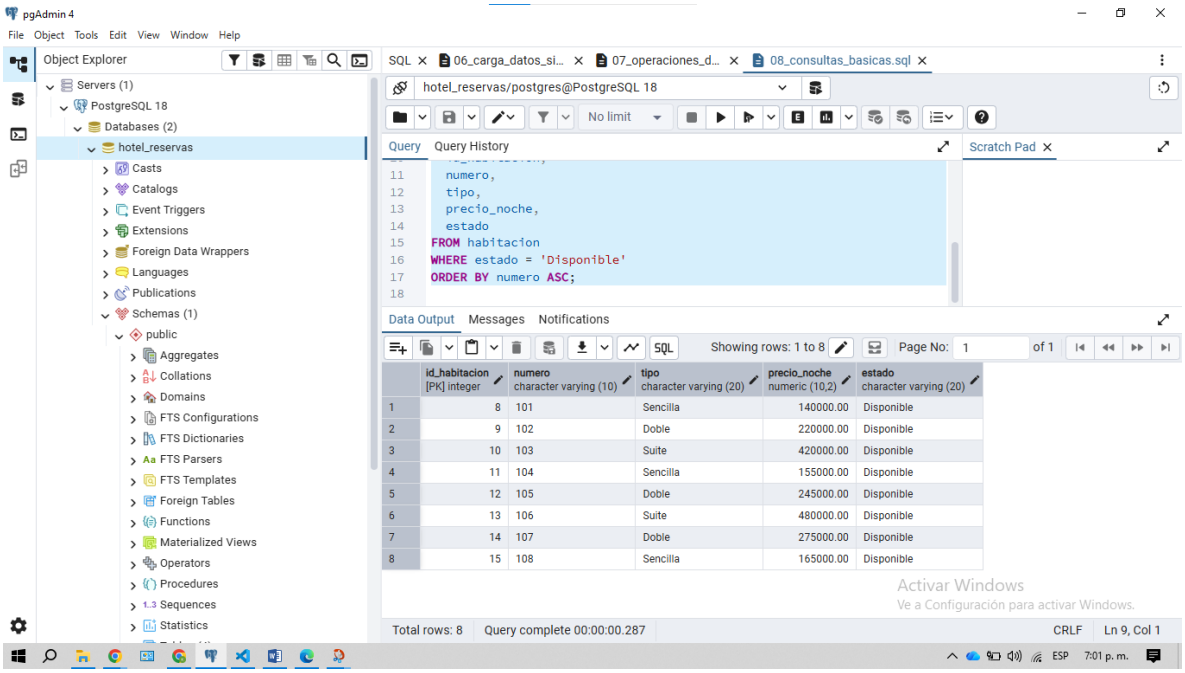
4.2.1. Consulta de habitaciones disponibles

Se desarrolló una consulta para identificar las habitaciones cuyo estado se encuentra registrado como Disponible.

Para resolverla se utilizó la tabla habitacion, aplicando una condición mediante WHERE y organizando los resultados por número de habitación.

La ejecución permitió identificar ocho habitaciones disponibles, correspondientes a los números 101 al 108. Este resultado confirmó que el sistema permite consultar las habitaciones según su estado, facilitando la identificación de aquellas que se encuentran habilitadas para su utilización.

Figura. Consulta de habitaciones disponibles mediante SELECT y WHERE.



4.2.2. Consulta de reservas activas

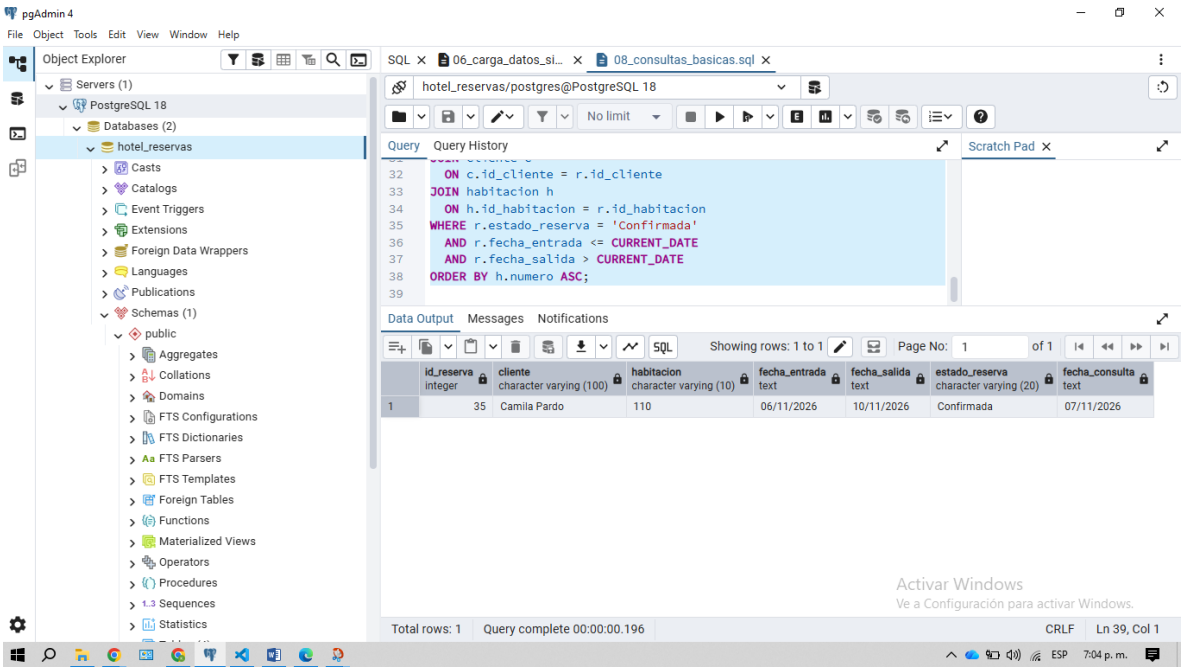
Se implementó una consulta para identificar las reservas que se encontraban vigentes en la fecha de ejecución.

La operación relacionó las tablas reserva, cliente y habitacion mediante JOIN, utilizando condiciones de fecha y estado para identificar las reservas confirmadas cuyo periodo de estancia incluía la fecha actual.

Durante la verificación realizada en noviembre de 2026, la consulta identificó la reserva de Camila Pardo para la habitación 110, con fecha de entrada 06/11/2026 y fecha de salida 10/11/2026.

El resultado confirmó que el sistema permite identificar las reservas vigentes de acuerdo con sus fechas de entrada y salida. Debido al uso de CURRENT_DATE, la información obtenida puede variar según el día de ejecución.

Figura. Consulta de reservas activas mediante JOIN y filtros de fecha.



4.2.3. Consulta de reservas con más de tres noches

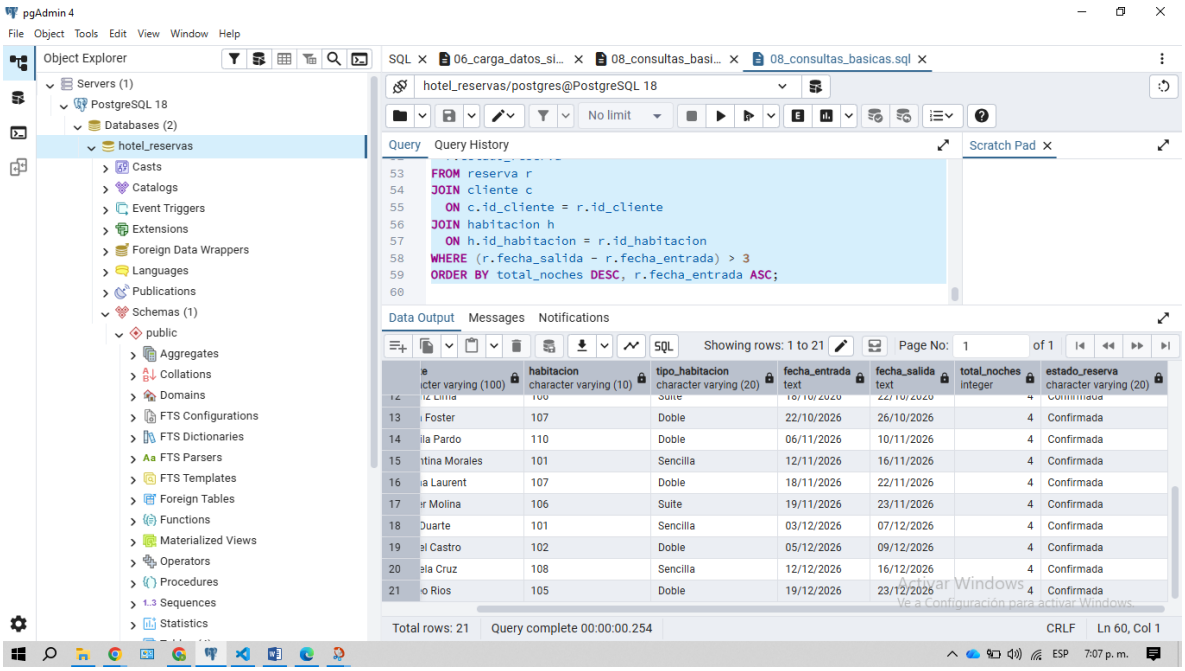
Se desarrolló una consulta para identificar las reservas cuya duración supera tres noches de alojamiento.

Para calcular la duración se utilizó la diferencia entre fecha_salida y fecha_entrada, aplicando posteriormente una condición para seleccionar únicamente las reservas superiores a tres noches.

La ejecución permitió identificar 21 reservas que cumplían esta condición.

El resultado demostró que PostgreSQL permite realizar operaciones con fechas y utilizarlas para filtrar registros según la duración de las estancias, proporcionando información útil para el análisis de los periodos de alojamiento.

Figura. Consulta de reservas con más de tres noches de estadía.



4.3. Consultas mediante JOIN

Se desarrollaron dos consultas para comprobar el funcionamiento de las relaciones entre las tablas del modelo físico.

Mediante operaciones JOIN, fue posible combinar la información de clientes, habitaciones y reservas, obteniendo resultados que no podrían consultarse directamente desde una sola tabla.

Los códigos quedaron registrados en el archivo 09_consultas_join.sql.

4.3.1. Consulta de clientes que reservaron habitaciones Suite

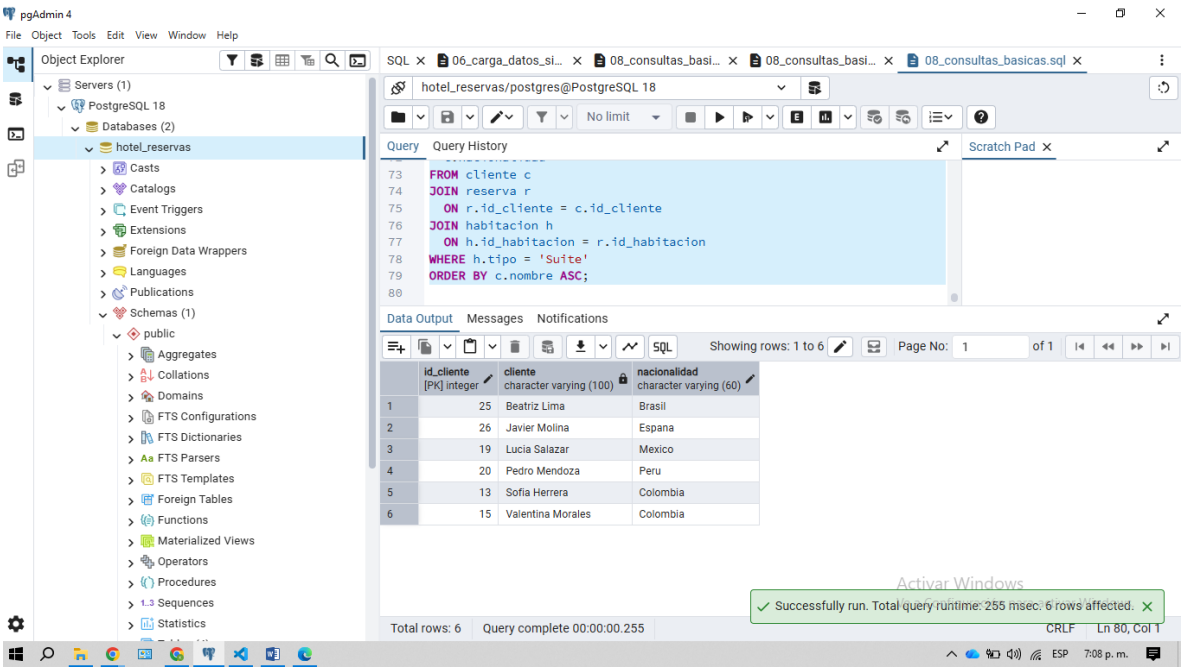
Se implementó una consulta para identificar los clientes que han realizado reservas de habitaciones tipo Suite.

La operación relacionó las tablas cliente, reserva y habitacion, aplicando una condición sobre el atributo tipo.

Durante la ejecución se identificaron seis clientes: Beatriz Lima, Javier Molina, Lucia Salazar, Pedro Mendoza, Sofia Herrera y Valentina Morales.

Este resultado permitió comprobar que las relaciones entre las tablas funcionan correctamente y que es posible consultar clientes según las características de las habitaciones reservadas.

Figura. Consulta de clientes que han reservado habitaciones tipo Suite mediante JOIN.



4.3.2. Consulta de clientes colombianos con reservas

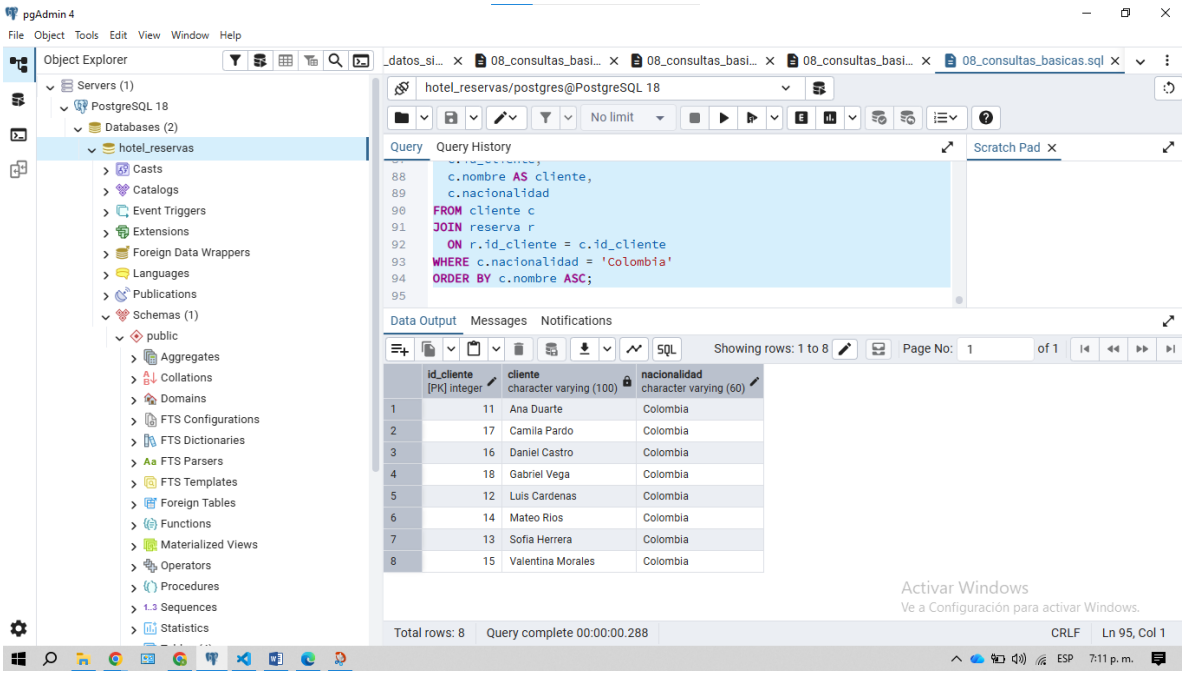
Se desarrolló una consulta para identificar los clientes de nacionalidad colombiana que tienen reservas registradas en el sistema.

Para resolverla se relacionaron las tablas cliente y reserva, aplicando un filtro sobre el atributo nacionalidad.

La ejecución permitió identificar ocho clientes colombianos con reservas registradas.

El resultado demostró que las operaciones JOIN pueden combinarse con condiciones de filtrado para consultar información específica de los clientes y su relación con las reservas.

Figura. Consulta de clientes colombianos con reservas mediante JOIN y WHERE.



4.4. Consultas de agregación y agrupamiento

Se desarrollaron diez consultas utilizando funciones de agregación como COUNT, SUM y AVG, junto con las cláusulas GROUP BY, HAVING y ORDER BY.

Estas consultas permitieron obtener información estadística y financiera relacionada con la frecuencia de las reservas, los ingresos registrados, los métodos de pago y el comportamiento de los clientes.

Los códigos fueron organizados en el archivo 10_consultas_agrupacion.sql, elaborado en Visual Studio Code y ejecutado mediante Query Tool de pgAdmin.

4.4.1. Consulta de habitaciones reservadas más de cinco veces

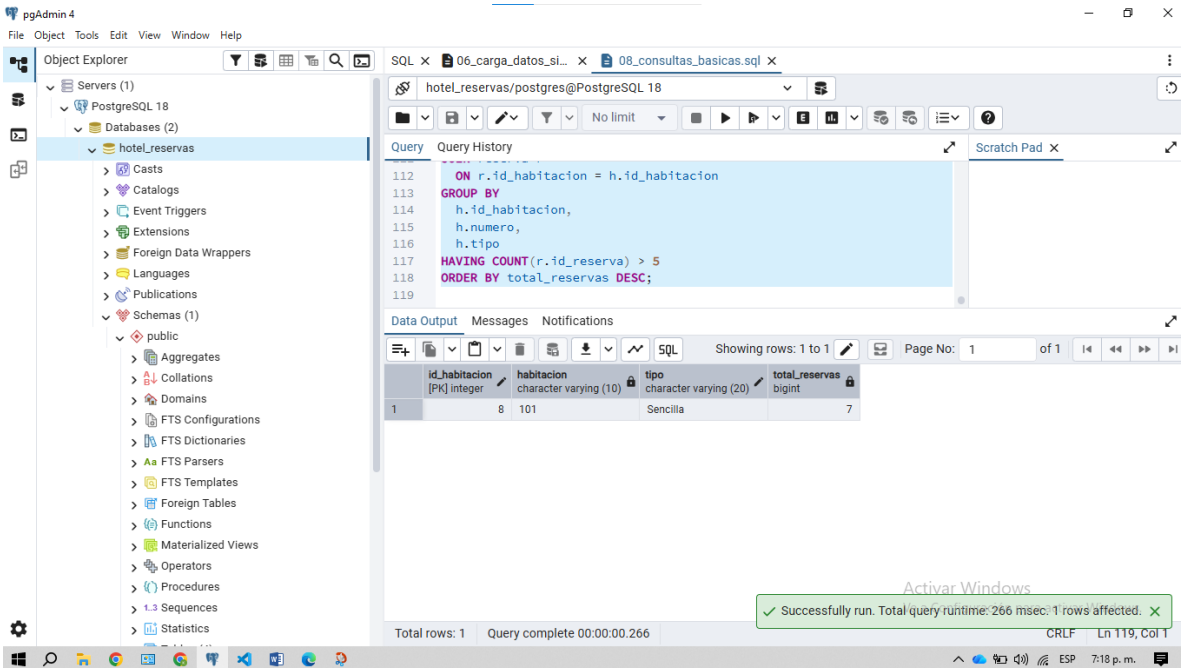
Se implementó una consulta para identificar las habitaciones que han registrado más de cinco reservas.

Se utilizó la función COUNT para calcular la cantidad de reservas por habitación y la cláusula HAVING para filtrar los resultados después del agrupamiento.

La ejecución permitió identificar la habitación 101, con siete reservas registradas.

Este resultado confirmó el funcionamiento de las cláusulas GROUP BY y HAVING para analizar la frecuencia de utilización de las habitaciones, permitiendo identificar aquellas con mayor cantidad de reservas.

Figura. Consulta de habitaciones reservadas más de cinco veces mediante GROUP BY y HAVING.



4.4.2. Consulta del total de reservas y monto por tipo de habitación

Se desarrolló una consulta para calcular la cantidad de reservas y el valor económico total de las estancias correspondientes a cada tipo de habitación.

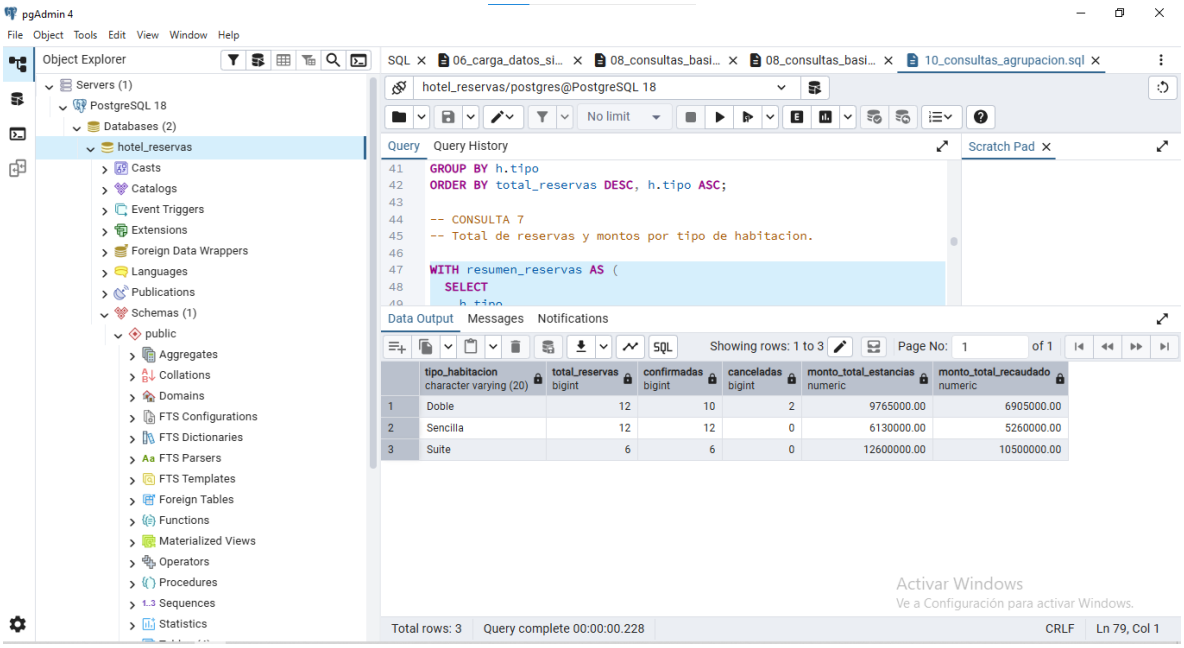
La operación agrupó las reservas según las categorías Sencilla, Doble y Suite, permitiendo obtener la cantidad de registros y los valores económicos asociados.

Los resultados mostraron doce reservas de habitaciones Sencilla, doce de habitaciones Doble y seis de habitaciones Suite, para un total de treinta reservas.

El valor total calculado de las estancias fue de $28.495.000, mientras que el dinero efectivamente registrado en pagos ascendió a $22.665.000.

Esta consulta permitió distinguir entre el valor económico de las reservas y los ingresos efectivamente recaudados por el hotel, considerando que algunas reservas no tenían pagos asociados y otras contaban con pagos parciales.

Figura. Consulta del total de reservas y valores económicos por tipo de habitación.



4.4.3. Consulta de ingresos generados por mes

Se implementó una consulta para obtener el total de ingresos mensuales registrados en el sistema.

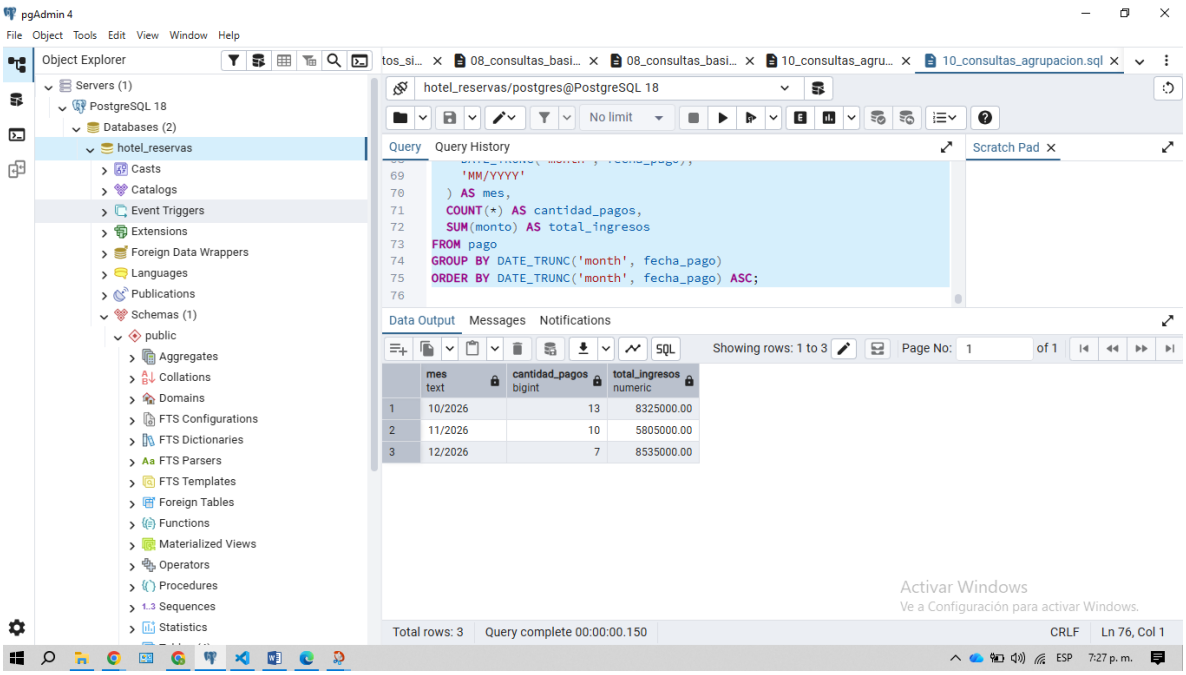
Mediante las funciones DATE_TRUNC, COUNT y SUM, junto con GROUP BY, se agruparon los pagos correspondientes a cada mes.

La ejecución permitió identificar ingresos de $8.325.000 en octubre, $5.805.000 en noviembre y $8.535.000 en diciembre de 2026.

En total se contabilizaron treinta pagos por $22.665.000, siendo diciembre el mes con mayores ingresos registrados.

El resultado permitió comprobar que PostgreSQL facilita el análisis de información financiera por periodos y permite obtener reportes mensuales a partir de los pagos almacenados.

Figura. Consulta de ingresos mensuales mediante SUM y GROUP BY.



4.4.4. Consulta del número de reservas por cliente

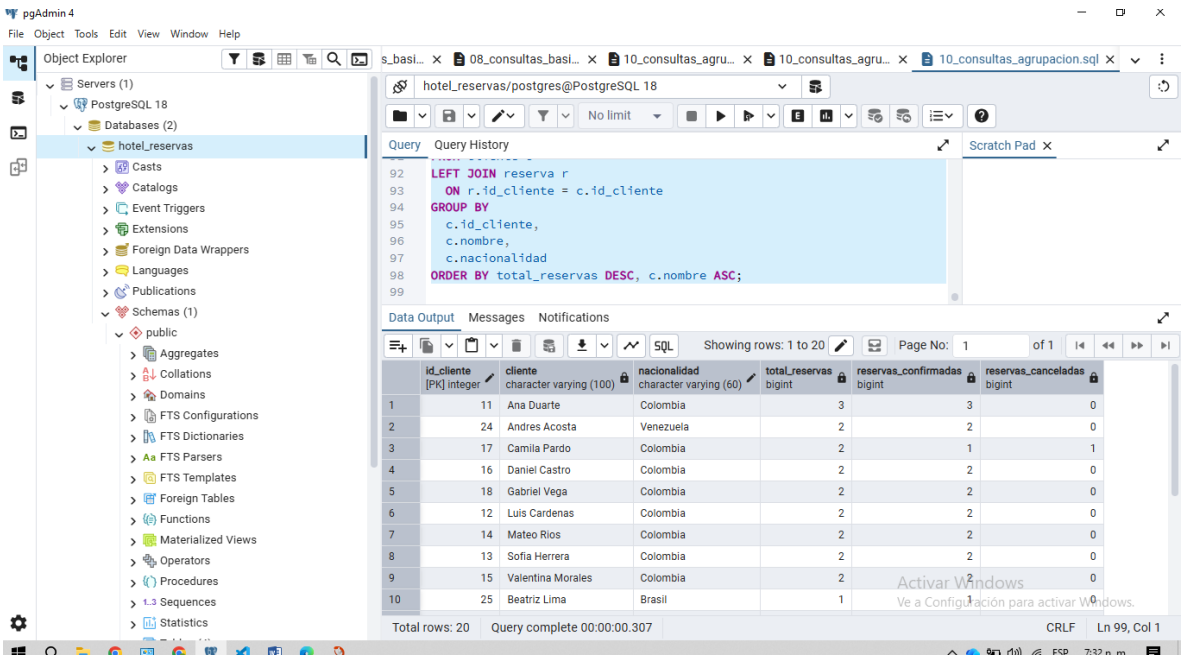
Se desarrolló una consulta para conocer cuántas reservas ha realizado cada cliente registrado en el hotel.

Se utilizó LEFT JOIN para incluir también a los clientes sin reservas y COUNT para calcular la cantidad de reservas de cada uno. Adicionalmente, se diferenciaron las reservas confirmadas y canceladas mediante condiciones de filtrado aplicadas a los resultados agrupados.

La consulta mostró los veinte clientes registrados y permitió identificar que Ana Duarte tenía tres reservas, siendo una de las clientes con mayor cantidad de registros.

El resultado permitió analizar la frecuencia de reservas y comprobar el funcionamiento del agrupamiento por cliente, conservando información sobre el estado de las reservas realizadas.

Figura. Consulta del número de reservas por cliente mediante COUNT y GROUP BY.



4.4.5. Consulta de clientes con mayor gasto acumulado

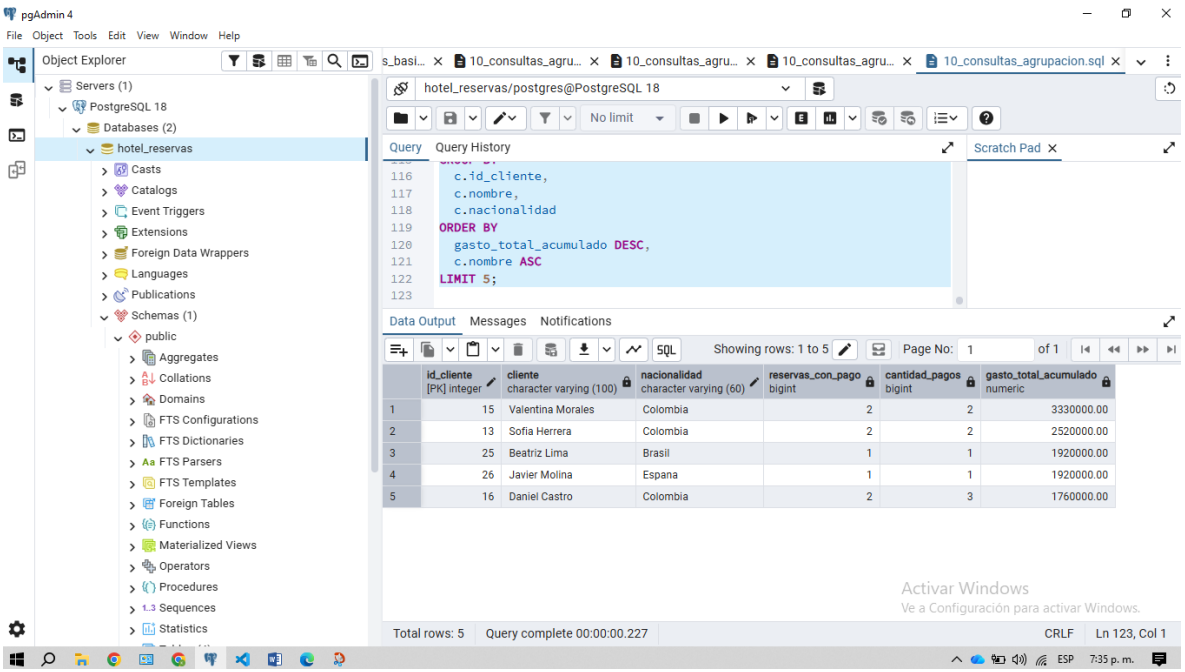
Se implementó una consulta para identificar los cinco clientes con mayor cantidad de dinero pagado al hotel.

La operación relacionó las tablas cliente, reserva y pago, utilizando la función SUM para calcular el gasto acumulado por cliente y ORDER BY para presentar los resultados de mayor a menor.

Los clientes con mayores pagos registrados fueron Valentina Morales, con $3.330.000; Sofia Herrera, con $2.520.000; Beatriz Lima y Javier Molina, con $1.920.000 cada uno; y Daniel Castro, con $1.760.000.

Esta consulta permitió identificar los clientes que han generado mayores ingresos mediante sus pagos registrados, diferenciando el dinero efectivamente pagado del valor total de las reservas.

Figura. Consulta de clientes con mayor gasto acumulado mediante SUM y ORDER BY.



4.4.6. Consulta de clientes que utilizaron más de un método de pago

Se desarrolló una consulta para identificar los clientes que han utilizado diferentes métodos de pago.

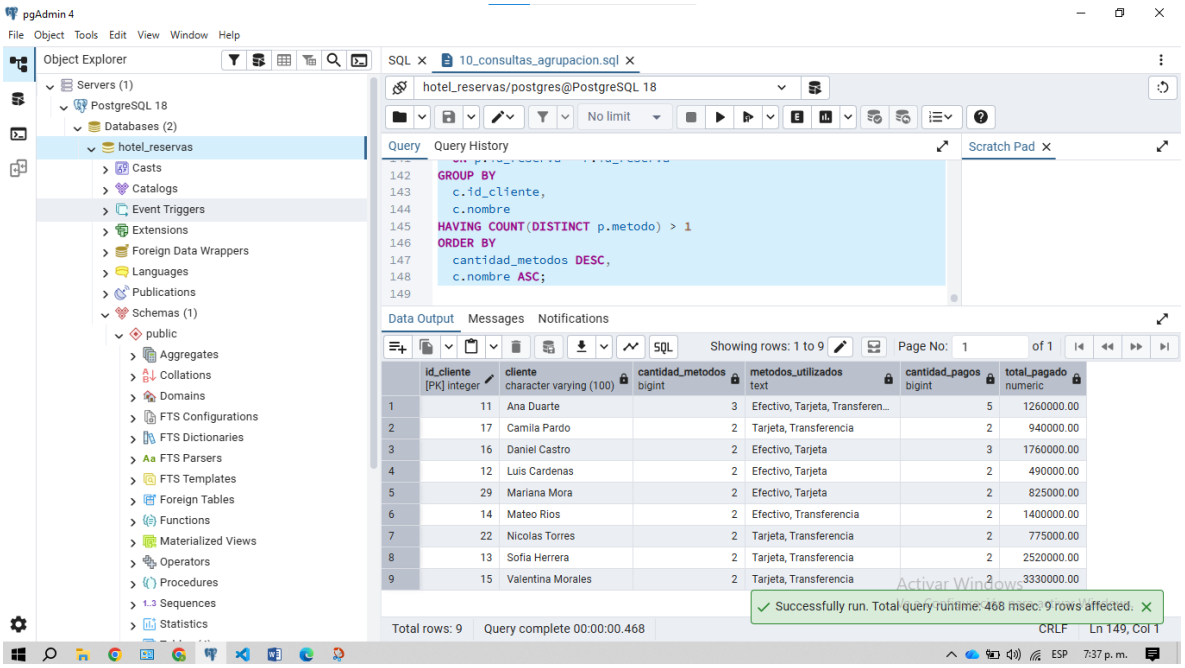
Para ello se empleó COUNT(DISTINCT metodo) y la cláusula HAVING, permitiendo seleccionar únicamente los clientes que habían utilizado más de un método.

La ejecución identificó nueve clientes que cumplían esta condición. Entre ellos, Ana Duarte registró tres métodos de pago diferentes: Efectivo, Tarjeta y Transferencia.

El resultado demostró la utilidad de DISTINCT y HAVING para analizar características no repetidas dentro de un conjunto de registros.

Adicionalmente, esta consulta permitió conocer la diversidad de métodos de pago utilizados por los huéspedes del hotel.

Figura. Consulta de clientes con diferentes métodos de pago mediante HAVING y COUNT DISTINCT.



4.4.7. Consulta del promedio de noches reservadas por tipo de habitación

Se implementó una consulta para calcular el promedio de noches registradas en las reservas de cada tipo de habitación.

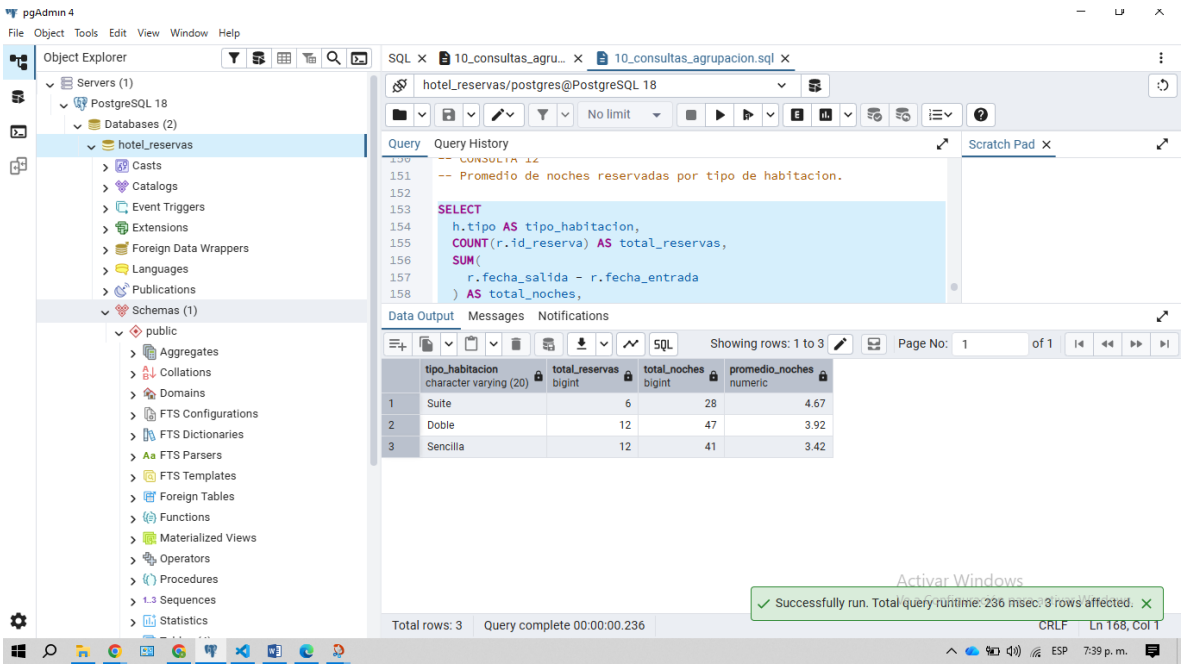
La duración se obtuvo mediante la diferencia entre las fechas de salida y entrada, utilizando posteriormente AVG y ROUND para calcular y presentar los promedios.

Los resultados mostraron un promedio de 4,67 noches para habitaciones Suite, 3,92 noches para habitaciones Doble y 3,42 noches para habitaciones Sencilla.

En total, las treinta reservas representaron 116 noches registradas.

La consulta permitió identificar que las reservas de habitaciones Suite presentaron la mayor duración promedio, proporcionando información útil sobre el comportamiento de las estancias según el tipo de habitación.

Figura. Consulta del promedio de noches por tipo de habitación mediante AVG y GROUP BY.



4.4.8. Consulta de los tres clientes con mayor cantidad de noches reservadas

Se desarrolló una consulta para identificar los tres clientes con mayor cantidad acumulada de noches correspondientes a reservas confirmadas.

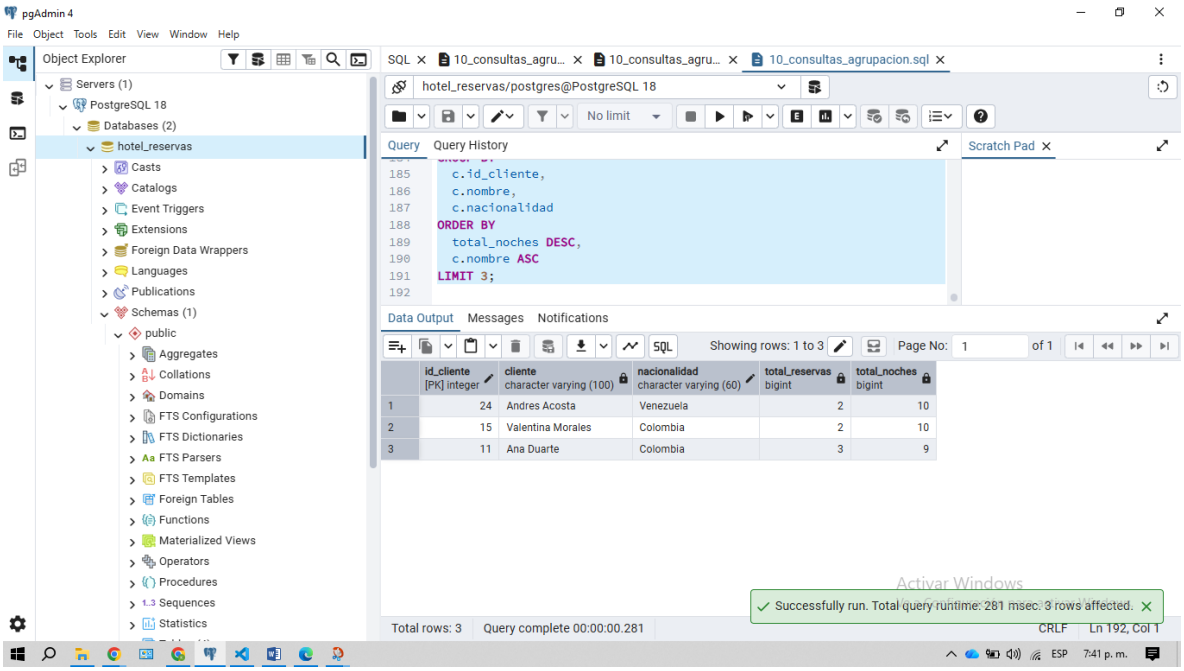
La operación calculó la duración de cada estancia mediante la diferencia entre las fechas de entrada y salida y utilizó SUM para acumular las noches de cada cliente.

Finalmente, mediante ORDER BY y LIMIT 3, se seleccionaron los tres registros con los valores más altos.

La consulta permitió identificar a Andres Acosta y Valentina Morales, con diez noches confirmadas cada uno, seguidos de Ana Duarte, con nueve noches.

Este resultado permitió distinguir entre la cantidad de reservas realizadas y la duración acumulada de las estancias, proporcionando información sobre los clientes con mayor permanencia registrada en el hotel.

Figura. Consulta de los tres clientes con mayor cantidad de noches reservadas.



4.4.9. Consulta de ingresos por método de pago

Se implementó una consulta para conocer el dinero recaudado mediante cada método de pago registrado en el sistema.

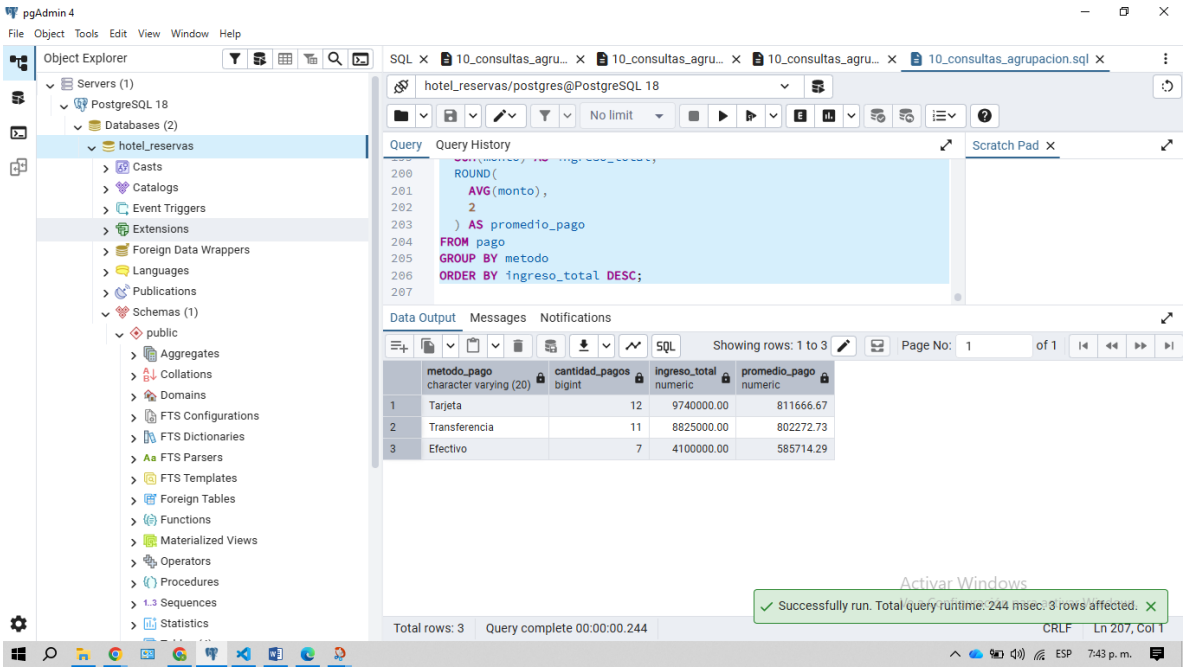
Mediante las funciones SUM, AVG y COUNT, junto con GROUP BY, se calcularon la cantidad de pagos, los ingresos acumulados y el valor promedio de las transacciones.

Los resultados mostraron doce pagos mediante Tarjeta por $9.740.000, once mediante Transferencia por $8.825.000 y siete mediante Efectivo por $4.100.000.

El ingreso total obtenido fue de $22.665.000, coincidiendo con las consultas financieras anteriores.

Este resultado permitió comprobar que el método Tarjeta concentró el mayor valor de ingresos registrados, seguido de Transferencia y Efectivo.

Figura. Consulta del ingreso total por método de pago mediante funciones de agregación.



4.4.10. Consulta de clientes que reservaron diferentes habitaciones

Se desarrolló una consulta para identificar los clientes que han reservado más de una habitación diferente.

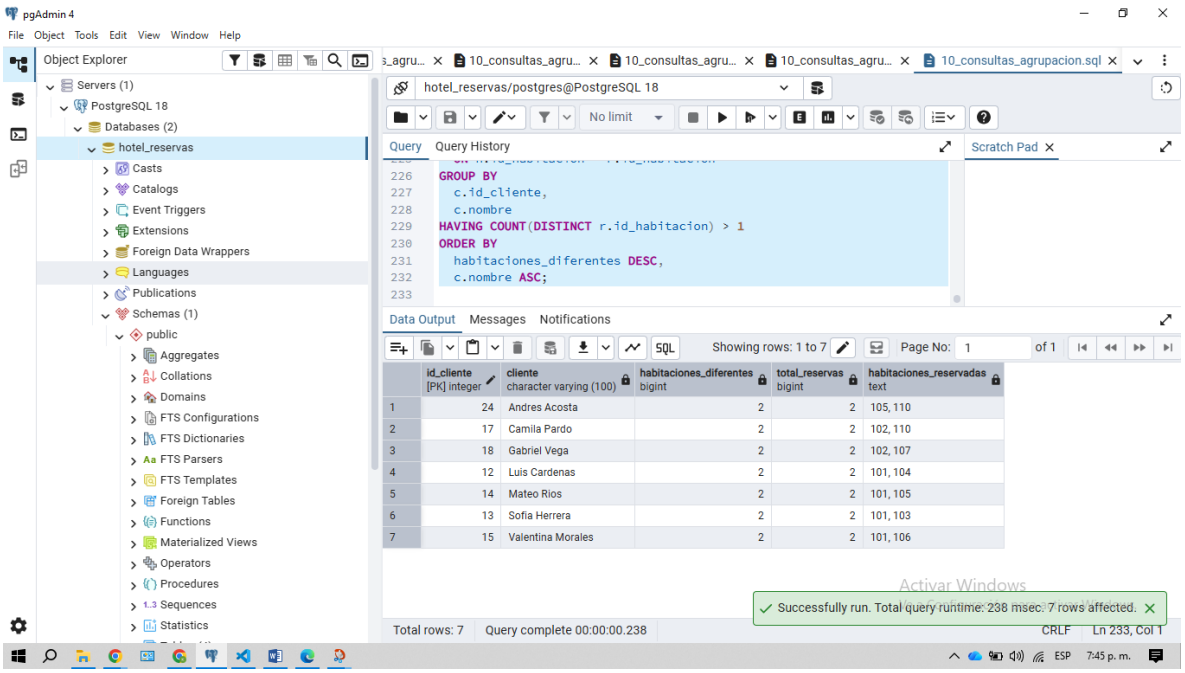
Se utilizó COUNT(DISTINCT id_habitacion) para evitar contabilizar repetidamente una habitación cuando el mismo cliente la reservó en varias ocasiones.

La consulta también utilizó GROUP BY para agrupar los registros por cliente y HAVING para seleccionar únicamente aquellos que habían reservado más de una habitación diferente.

Durante la ejecución se identificaron siete clientes que cumplían esta condición: Andres Acosta, Camila Pardo, Gabriel Vega, Luis Cardenas, Mateo Rios, Sofia Herrera y Valentina Morales. Cada uno de estos clientes registró reservas en dos habitaciones diferentes.

Los resultados permitieron comprobar el funcionamiento de `COUNT(DISTINCT id_habitacion)`, al contabilizar las habitaciones diferentes asociadas a cada cliente, independientemente de la cantidad total de reservas realizadas .La consulta permitió analizar la variedad de habitaciones reservadas por cada cliente, diferenciándola del número total de reservas realizadas.

Figura. Consulta de clientes que reservaron más de una habitación diferente mediante COUNT DISTINCT.



4.5. Subconsultas y comparaciones SQL

Se desarrollaron cinco consultas utilizando subconsultas para resolver operaciones de comparación y búsqueda de información relacionada.

Estas consultas permitieron identificar habitaciones con precios superiores al promedio, reservas de mayor valor, habitaciones sin reservas, reservas sin pagos y la habitación más costosa reservada por cada cliente.

Los códigos fueron organizados en el archivo 11_consultas_subconsultas.sql, elaborado en Visual Studio Code y ejecutado mediante pgAdmin.

4.5.1. Consulta de habitaciones con precio superior al promedio

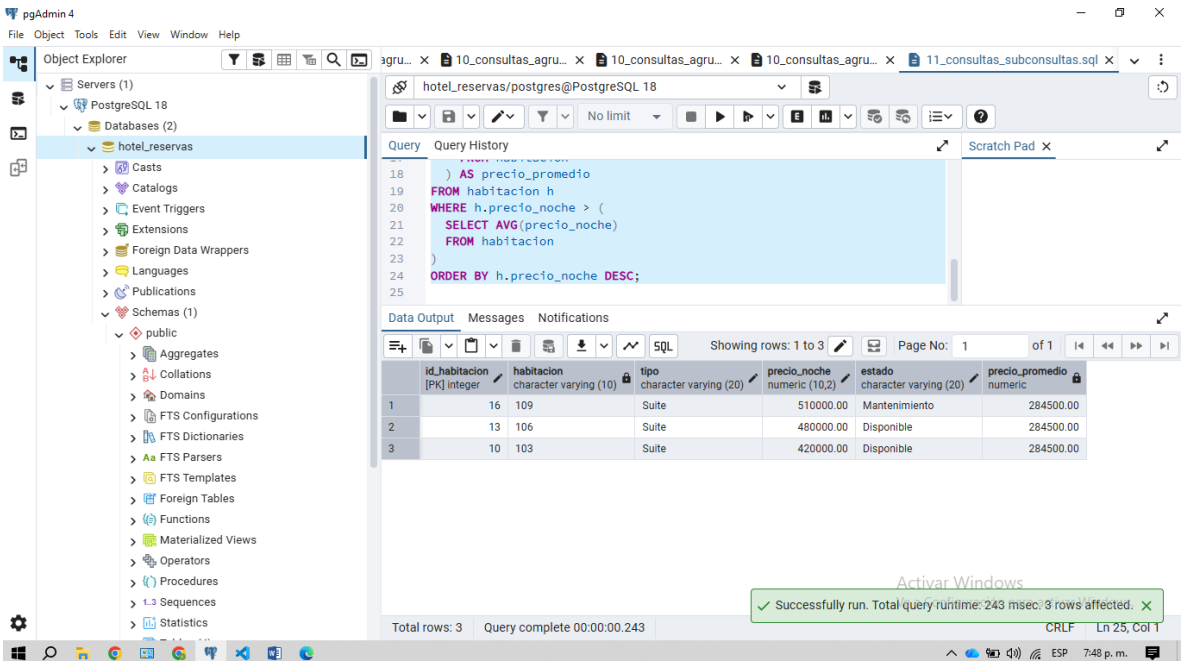
Se implementó una consulta para identificar las habitaciones cuyo precio por noche supera el promedio general del hotel.

Mediante una subconsulta con AVG, se calculó un precio promedio de $284.500 por noche.

La ejecución permitió identificar tres habitaciones que superaban este valor: la 109, con $510.000; la 106, con $480.000; y la 103, con $420.000, todas de tipo Suite.

El resultado demostró que las subconsultas pueden utilizarse para comparar los valores individuales de una tabla con un resultado agregado, sin necesidad de establecer manualmente un valor fijo de comparación.

Figura. Consulta de habitaciones con precio superior al promedio mediante una subconsulta con AVG.



4.5.2. Consulta del cliente que realizó la reserva más costosa

Se desarrolló una consulta para identificar la reserva con el mayor valor económico registrado en el hotel.

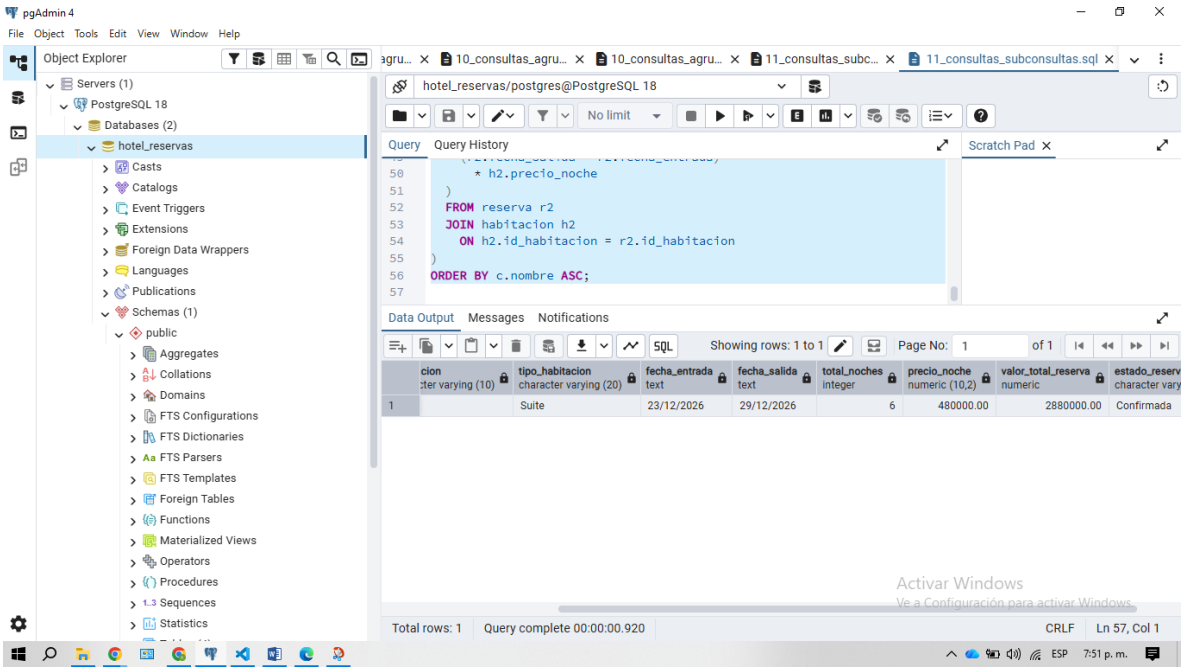
El valor de cada reserva se calculó multiplicando la cantidad de noches por el precio de la habitación. Posteriormente, se utilizó una subconsulta con MAX para encontrar el valor más alto.

La ejecución identificó a Valentina Morales, quien tenía registrada una reserva confirmada para la habitación 106, tipo Suite, desde el 23/12/2026 hasta el 29/12/2026.

La estancia correspondía a seis noches, con un precio de $480.000 por noche y un valor total de $2.880.000.

El resultado permitió identificar correctamente la reserva de mayor valor económico y comprobar el funcionamiento de las subconsultas utilizadas para comparar valores calculados.

Figura. Consulta del cliente que realizó la reserva más costosa mediante MAX.



4.5.3. Consulta de habitaciones que nunca han sido reservadas

Se implementó una consulta para identificar habitaciones sin registros asociados en la tabla reserva.

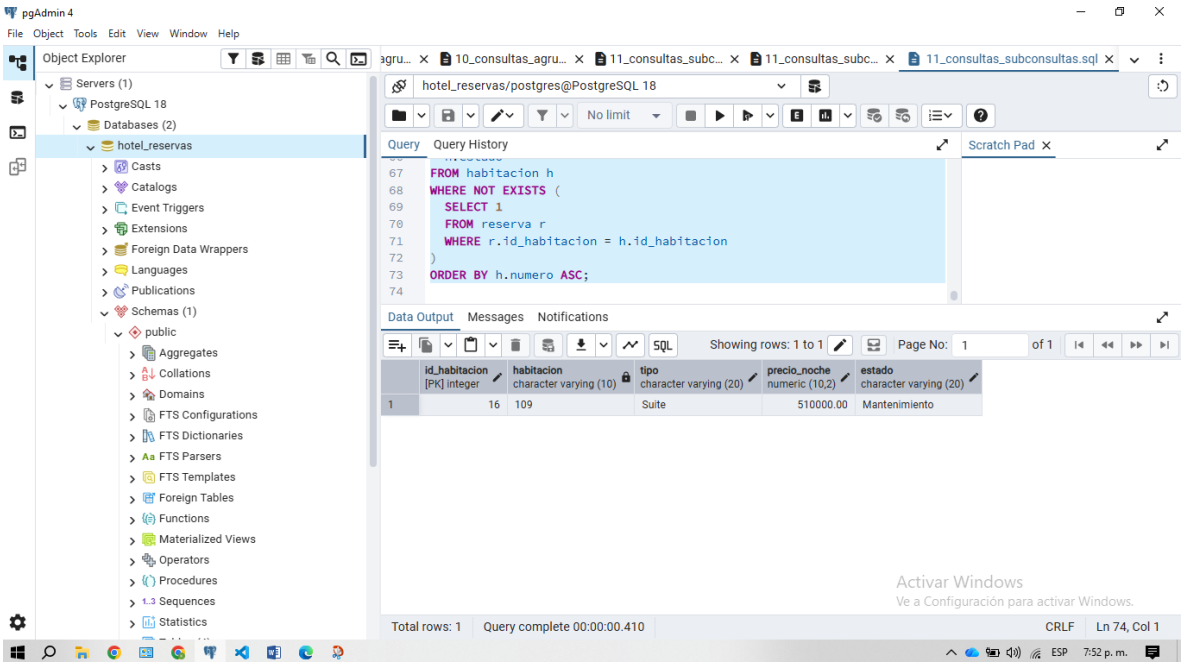
Para resolverla se utilizó una subconsulta con NOT EXISTS, que permite seleccionar habitaciones para las cuales no existe ninguna reserva registrada.

La ejecución identificó únicamente la habitación 109, de tipo Suite, con un precio de $510.000 por noche y estado Mantenimiento.

Este resultado confirmó que la habitación no tenía reservas históricas, independientemente de su estado de disponibilidad.

La consulta permitió comprobar la utilidad de NOT EXISTS para identificar registros de una tabla que no poseen relaciones con registros de otra.

Figura. Consulta de habitaciones sin reservas mediante NOT EXISTS.



4.5.4. Consulta de reservas que no tienen pagos registrados

Se desarrolló una consulta para identificar reservas que no tienen registros asociados en la tabla pago.

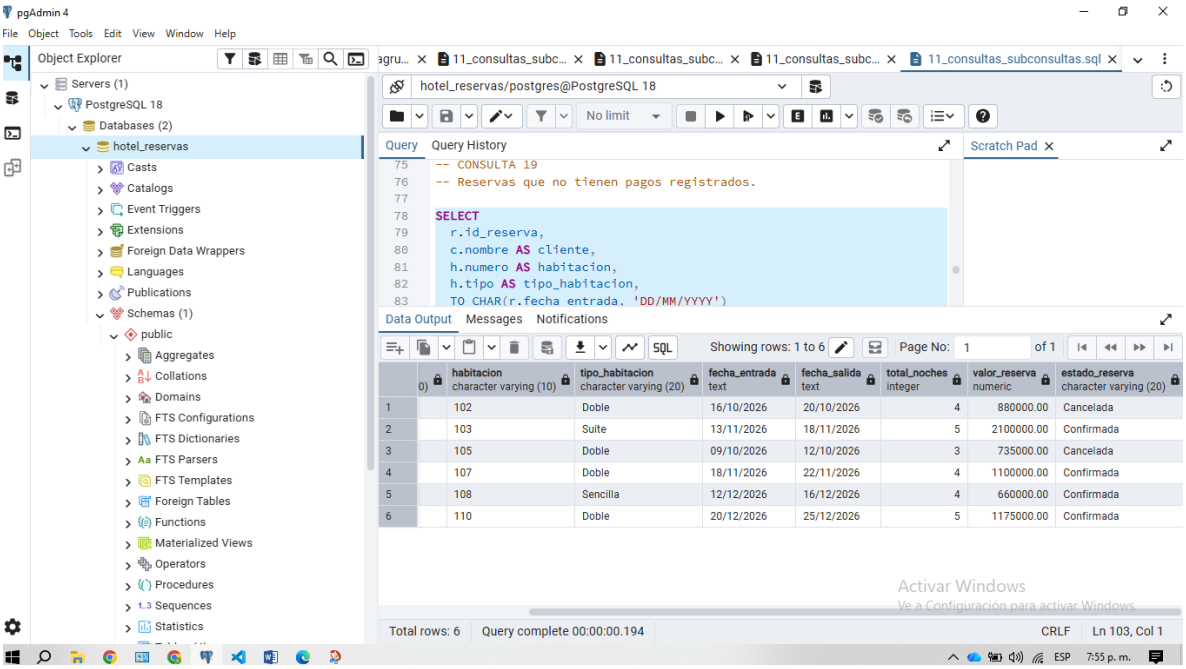
Mediante una subconsulta con NOT EXISTS, se verificó la ausencia de pagos para cada reserva.

La ejecución identificó seis reservas sin pagos registrados, de las cuales cuatro se encontraban confirmadas y dos canceladas.

El resultado demostró que la ausencia de pagos no implica necesariamente una obligación de cobro vigente, ya que también deben considerarse los estados de las reservas.

Adicionalmente, permitió identificar situaciones que podrían requerir seguimiento administrativo por parte del hotel, especialmente en las reservas confirmadas que no registran pagos.

Figura. Consulta de reservas sin pagos registrados mediante NOT EXISTS.



4.5.5. Consulta de la habitación más costosa reservada por cada cliente

Se implementó una consulta para identificar la habitación con el mayor precio por noche que ha reservado cada cliente.

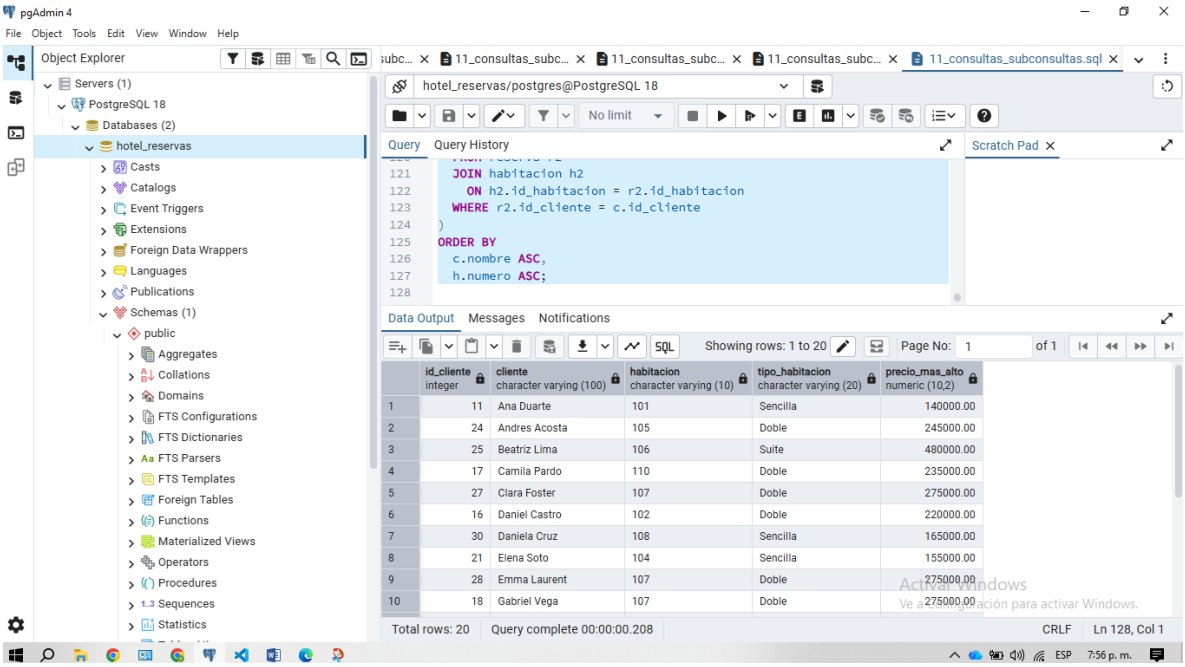
La operación utilizó una subconsulta correlacionada con MAX, comparando el precio de las habitaciones correspondientes a las reservas de un mismo cliente.

La ejecución mostró veinte registros, uno por cada cliente con reservas, identificando la habitación de mayor precio asociada a su historial.

Entre los resultados se encontró la habitación 106, de tipo Suite, con un precio de $480.000 por noche, como la más costosa reservada por Beatriz Lima, Javier Molina y Valentina Morales.

Esta consulta permitió comprobar el funcionamiento de las subconsultas correlacionadas y diferenciar el precio individual de una habitación del valor económico total de una estancia.

Figura. Consulta de la habitación más costosa reservada por cada cliente mediante una subconsulta correlacionada.



4.6. Conclusión de las consultas SQL

El desarrollo de las veinte consultas permitió comprobar el funcionamiento del sistema de gestión de reservas para un hotel mediante diferentes técnicas de recuperación y procesamiento de información en PostgreSQL 18.

Las consultas básicas facilitaron la identificación de habitaciones disponibles y reservas según sus fechas y duración. Las operaciones JOIN permitieron relacionar información de clientes, reservas y habitaciones, mientras que las funciones de agregación y agrupamiento proporcionaron resultados estadísticos sobre ingresos, frecuencia de reservas y métodos de pago.

Por otra parte, las subconsultas permitieron realizar comparaciones avanzadas para identificar habitaciones de mayor precio, reservas de mayor valor económico, habitaciones sin reservas y registros sin pagos asociados.

Los resultados obtenidos fueron coherentes con el conjunto de datos sintéticos previamente cargado y permitieron comprobar la utilidad de las consultas para apoyar la gestión operativa y financiera del hotel.

Todos los scripts se organizaron en archivos SQL independientes y se registraron mediante Git, permitiendo conservar el historial de desarrollo y las modificaciones realizadas en el proyecto.

La implementación también permitió fortalecer la comprensión de instrucciones como SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY y las funciones de agregación, así como de las subconsultas utilizadas para resolver requerimientos específicos del sistema.

De esta manera, se desarrollaron las consultas correspondientes a la Parte 4 del taller, verificando su funcionamiento mediante PostgreSQL y conservando evidencias de los resultados obtenidos.

4.7. Análisis del rendimiento de consultas SQL mediante EXPLAIN

Con el propósito de complementar las verificaciones funcionales realizadas en PostgreSQL 18, se desarrolló un análisis del rendimiento mediante la instrucción EXPLAIN ANALYZE, permitiendo examinar el comportamiento interno del motor de la base de datos durante la ejecución de una consulta.

Para este análisis se seleccionó la consulta de clientes con mayor gasto acumulado, debido a que relaciona las tablas cliente, reserva y pago mediante operaciones JOIN, utiliza funciones de agregación y organiza los resultados para identificar los cinco clientes con mayores pagos registrados.

Se empleó la instrucción EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT), con el objetivo de identificar el plan de ejecución, los tiempos de planificación y ejecución, así como información relacionada con los recursos utilizados durante el procesamiento de los datos.

La prueba se ejecutó en PostgreSQL 18 mediante pgAdmin, utilizando la base de datos hotel_reservas y conservando la consulta SQL original sin modificaciones.

4.7.1. Resultados del análisis de rendimiento

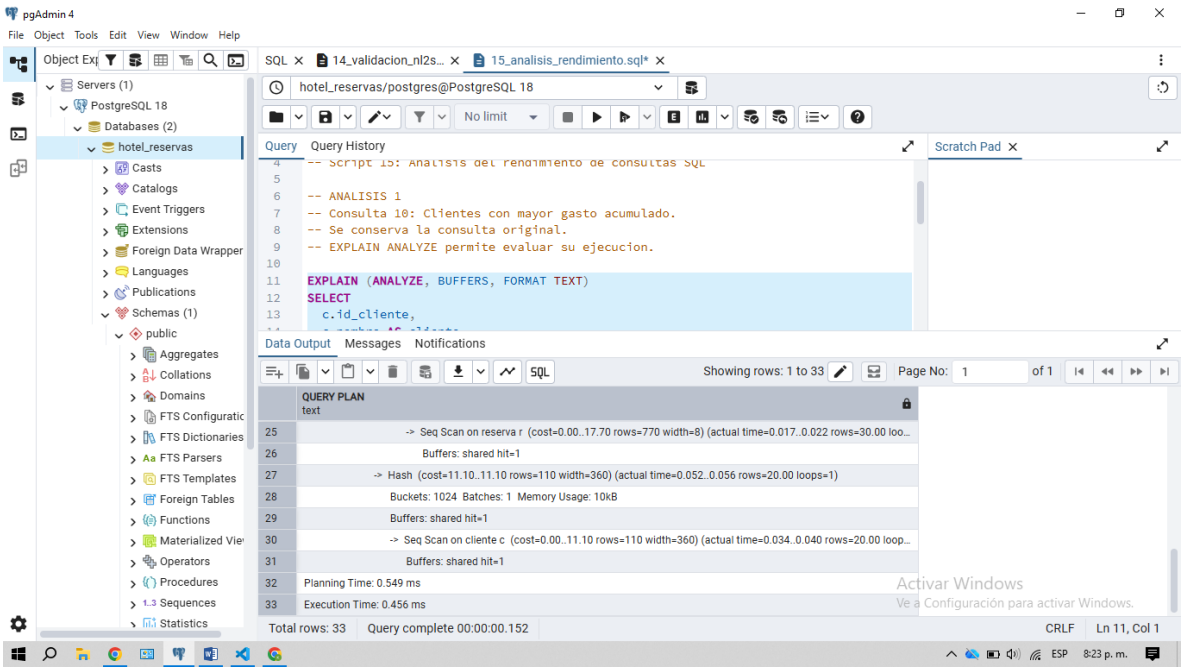
La ejecución permitió obtener un plan de consulta de 33 líneas, en el que se identificaron operaciones de lectura secuencial (Seq Scan) y procesamiento mediante Hash.

PostgreSQL reportó un tiempo de planificación de **0,549 milisegundos** y un tiempo de ejecución de **0,456 milisegundos**.

Adicionalmente, en los nodos visibles del plan se registraron accesos a bloques de memoria mediante Buffers: shared hit=1, así como un consumo de memoria de 10 kB para una operación Hash.

Estos resultados indican que la consulta presentó un tiempo de procesamiento bajo sobre el conjunto de datos sintéticos utilizado.

Figura. Análisis del rendimiento de la consulta de clientes con mayor gasto acumulado mediante EXPLAIN ANALYZE.



4.7.2. Interpretación de los resultados

El análisis permitió comprobar que PostgreSQL ejecutó la consulta sin presentar tiempos elevados de procesamiento.

Las operaciones de lectura secuencial identificadas son coherentes con el tamaño reducido de las tablas utilizadas en el proyecto, puesto que estas contienen únicamente veinte clientes, treinta reservas y treinta pagos.

La información del plan de ejecución permitió complementar la validación funcional realizada anteriormente, diferenciando la obtención correcta de resultados del análisis del rendimiento interno de la consulta.

Sin embargo, los tiempos registrados corresponden al conjunto de datos empleado durante el taller y no garantizan el mismo comportamiento ante un volumen considerablemente mayor de información.

En conclusión, la utilización de EXPLAIN ANALYZE permitió evaluar el rendimiento de una consulta representativa del sistema hotelero, identificando sus tiempos de ejecución y algunas de las operaciones utilizadas por PostgreSQL para procesar la información.

Parte 5. Implementación de vistas y transacciones en PostgreSQL

En esta parte del proyecto se implementaron vistas y transacciones SQL con el propósito de facilitar la consulta de información y garantizar la integridad de las operaciones realizadas en el Sistema de Gestión de Reservas para un Hotel.

Para su desarrollo se utilizó PostgreSQL 18 como sistema gestor de bases de datos, pgAdmin como herramienta de ejecución y verificación, y Visual Studio Code para la organización de los scripts SQL. Adicionalmente, se empleó Git y GitHub para mantener el control de versiones de los archivos desarrollados.

Se crearon dos vistas: una para consultar las reservas activas del hotel y otra para obtener los ingresos mensuales. Posteriormente, se implementaron transacciones que involucraron las tablas reserva y pago, utilizando las instrucciones BEGIN, COMMIT y ROLLBACK. Estas operaciones permitieron analizar el funcionamiento de las transacciones y su relación con las propiedades ACID: atomicidad, consistencia, aislamiento y durabilidad.

5.1. Implementación de la vista de reservas activas

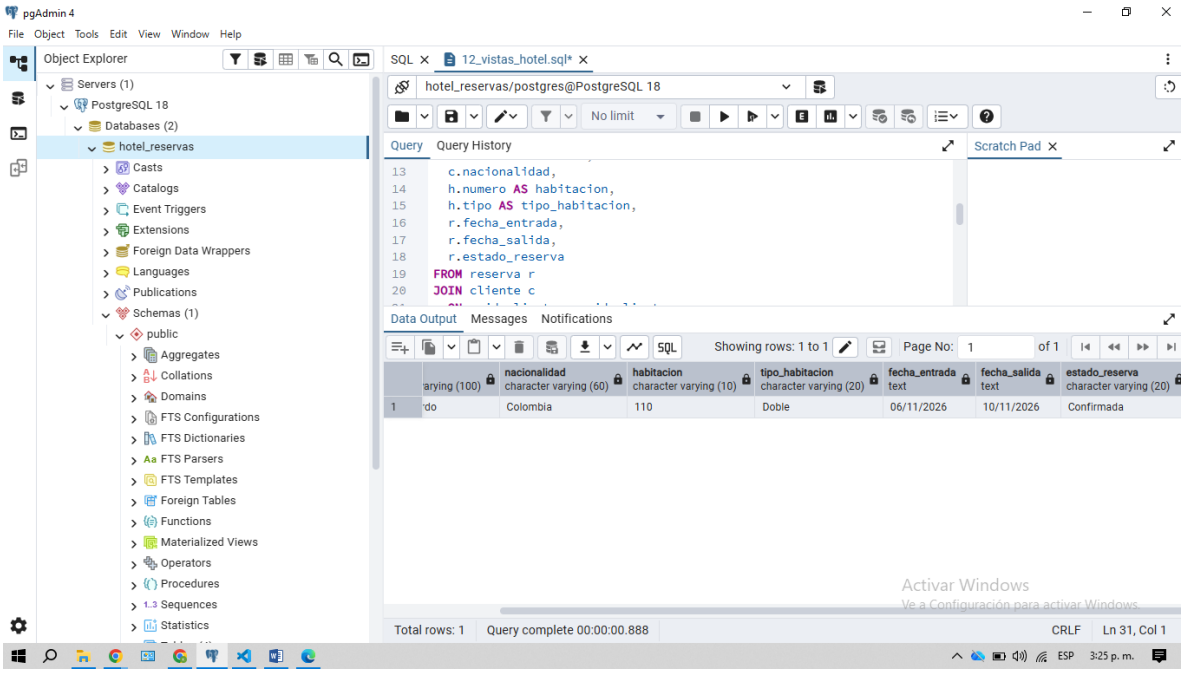
Se creó la vista vista_reservas_activas mediante la sentencia CREATE OR REPLACE VIEW, con el objetivo de facilitar la identificación de las reservas que se encuentran vigentes en el hotel.

Esta vista relaciona las tablas reserva, cliente y habitacion mediante operaciones JOIN, permitiendo obtener información como el identificador de la reserva, el nombre y la nacionalidad del cliente, el número y tipo de habitación, las fechas de entrada y salida, y el estado correspondiente.

Para identificar las reservas activas se establecieron condiciones que verifican que su estado sea Confirmada y que la fecha actual, obtenida mediante CURRENT_DATE, se encuentre dentro del periodo comprendido entre la fecha de entrada y la fecha de salida. De esta manera, se excluyen las reservas canceladas, las que todavía no han comenzado y aquellas cuya estancia ya finalizó.

La creación de esta vista permite reutilizar una consulta operativa del hotel sin necesidad de escribir nuevamente las relaciones entre las tablas involucradas. Su definición y consulta de verificación quedaron registradas en el archivo 12_vistas_hotel.sql.

Figura. Creación de la vista de reservas activas mediante CREATE OR REPLACE VIEW.



Posteriormente, se ejecutó una consulta SELECT sobre la vista creada para comprobar su funcionamiento. Durante la verificación realizada el 08/11/2026 se obtuvo la reserva con identificador 35, correspondiente a Camila Pardo, de nacionalidad colombiana, quien tenía una reserva confirmada para la habitación 110, tipo Doble, con fecha de entrada 06/11/2026 y fecha de salida 10/11/2026.

El resultado coincidió con las condiciones definidas en la vista, demostrando que era posible identificar las reservas vigentes de acuerdo con la fecha de ejecución.

Es importante señalar que, al utilizar CURRENT_DATE, los resultados pueden variar cuando se consulte nuevamente la vista en una fecha diferente, sin necesidad de modificar su definición SQL.

5.2. Implementación de la vista de ingresos mensuales

Se implementó una segunda vista denominada vista_ingresos_mensuales, cuyo propósito es facilitar la consulta de los ingresos registrados en el hotel durante cada mes.

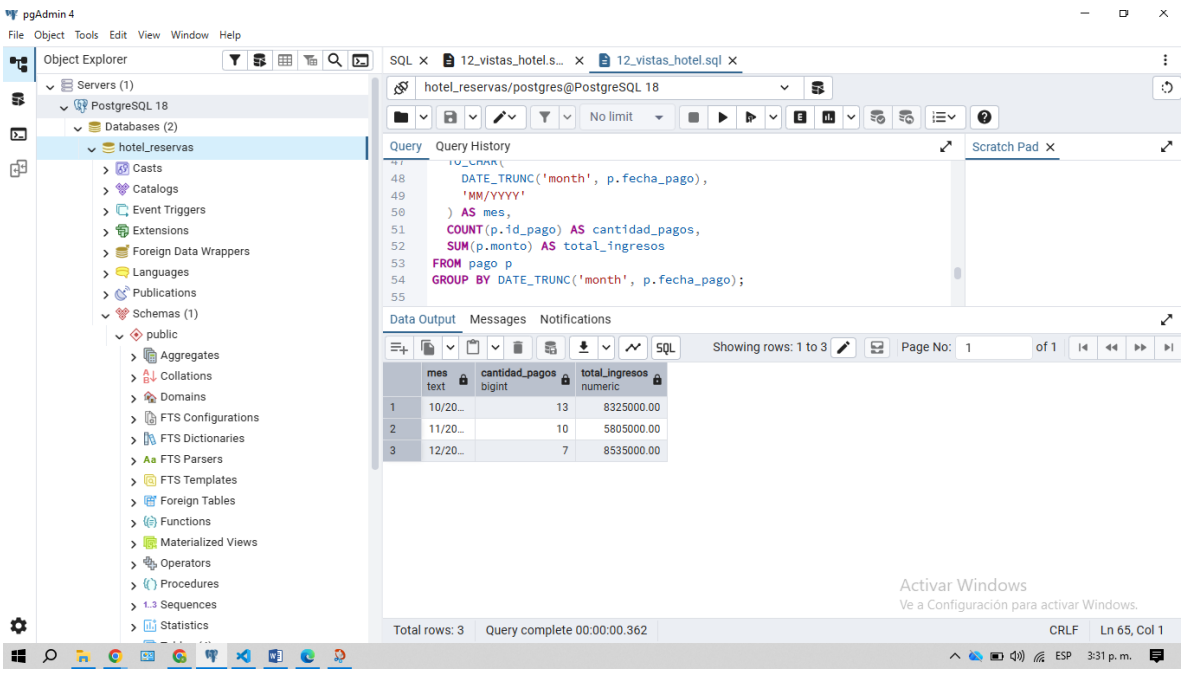
Para su construcción se utilizó la tabla pago, agrupando los registros según el mes y año de la fecha de pago. Se empleó la función DATE_TRUNC para identificar el periodo mensual, COUNT para contabilizar los pagos y SUM para calcular los ingresos acumulados. Adicionalmente, se utilizó TO_CHAR para presentar el periodo en el formato MM/AAAA.

Esta vista permite obtener información financiera de forma organizada y reutilizable. Además, los resultados reflejan los cambios que se realicen

posteriormente en la tabla de pagos, ya que una vista convencional de PostgreSQL conserva la definición de la consulta y obtiene sus datos de las tablas relacionadas.

El código correspondiente a esta segunda vista también se incorporó en el archivo 12_vistas_hotel.sql, manteniendo organizadas las dos vistas requeridas para el proyecto.

Figura. Creación de la vista de ingresos mensuales mediante CREATE OR REPLACE VIEW.



Para verificar su funcionamiento se ejecutó una consulta SELECT sobre la vista, obteniendo los ingresos correspondientes a octubre, noviembre y diciembre de 2026.

Los resultados fueron los siguientes:

Mes	Cantidad de pagos	Ingresos registrados
Octubre de 2026	13	$8.325.000
Noviembre de 2026	10	$5.805.000
Diciembre de 2026	7	$8.535.000
Total	**30**	**$22.665.000**

Se identificó que diciembre fue el mes con mayores ingresos registrados, con $8.535.000, seguido de octubre y noviembre.

Adicionalmente, se comprobó que la suma de los ingresos de los tres meses correspondía a los $22.665.000 obtenidos previamente en las consultas de agregación de la Parte 4. Esta coincidencia permitió verificar que la vista presentaba correctamente la información registrada en la tabla pago.

5.3. Implementación de transacciones SQL

Una transacción permite agrupar varias operaciones SQL para que se ejecuten como una sola unidad lógica de trabajo. Esta característica resulta importante en un sistema hotelero, debido a que algunas operaciones requieren modificar información relacionada en varias tablas.

Para demostrar su funcionamiento se desarrolló un escenario en el que un cliente realiza una reserva y registra un pago inicial. La operación involucró las tablas reserva y pago, mediante sentencias INSERT ejecutadas dentro de una transacción iniciada con BEGIN.

Se utilizó el cliente Ana Duarte, identificado con el código 11, y la habitación 101, de tipo Sencilla, con un precio de $140.000 por noche. La estancia de prueba comprendió desde el 15/01/2027 hasta el 18/01/2027, para un total de tres noches y un valor de $420.000. Como pago inicial se registraron $200.000 mediante el método Tarjeta.

Las pruebas se organizaron en el archivo 13_transaccion_acid.sql y se ejecutaron por bloques en pgAdmin para observar el comportamiento de las instrucciones ROLLBACK y COMMIT.

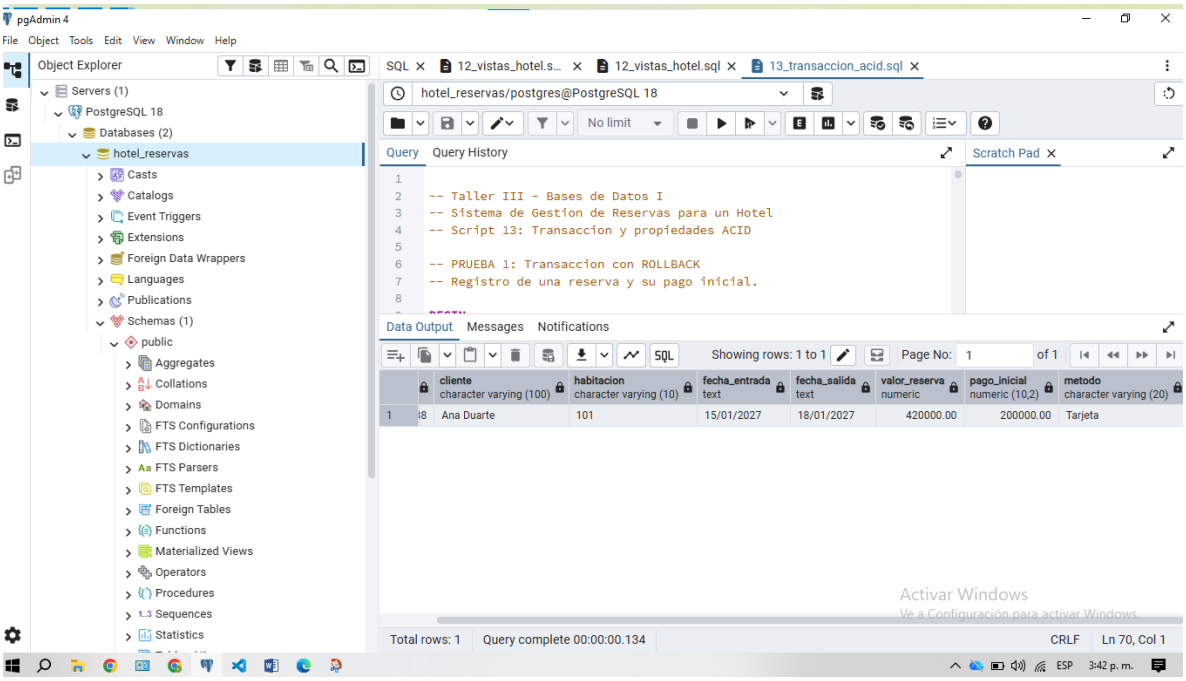
5.3.1. Prueba de transacción mediante ROLLBACK

En la primera prueba se utilizó la instrucción BEGIN para iniciar una transacción y posteriormente se insertaron una reserva y su pago inicial. Para asociar correctamente ambos registros se utilizó una expresión WITH junto con RETURNING id_reserva, que permitió obtener el identificador de la reserva recién creada y utilizarlo en la inserción del pago.

Durante la ejecución se generó temporalmente la reserva con identificador 38, correspondiente a Ana Duarte, para la habitación 101, con fechas de entrada 15/01/2027 y salida 18/01/2027.

La inserción del pago inicial por $200.000 también se realizó correctamente, manteniendo la relación entre ambas tablas mediante la clave foránea correspondiente.

Figura. Ejecución de la transacción de prueba para registrar una reserva y su pago inicial.



Antes de finalizar la transacción, se ejecutó una consulta de verificación para comprobar que los registros se encontraban disponibles dentro de la misma transacción.

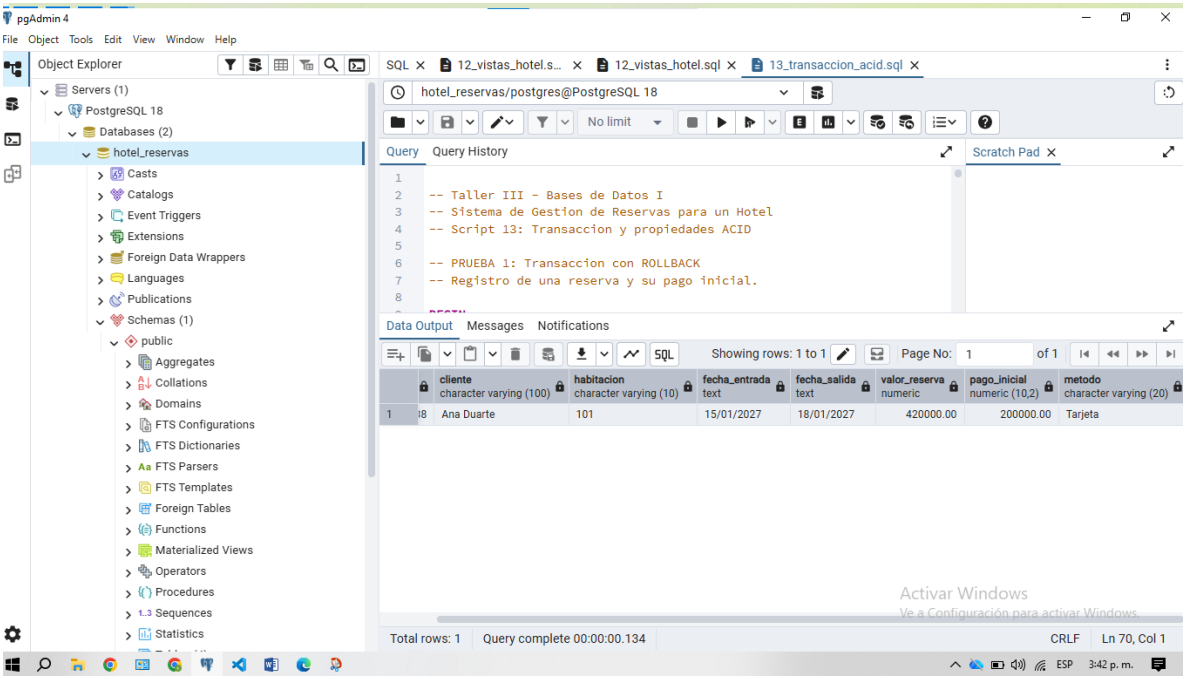
El resultado mostró la reserva 38, el nombre del cliente, la habitación 101, el valor total de $420.000 y el pago inicial de $200.000 mediante Tarjeta.

Posteriormente, se ejecutó la instrucción ROLLBACK para deshacer las operaciones realizadas desde el inicio de la transacción.

Para verificar su efecto se realizó una nueva consulta, obteniendo como resultado cero reservas y cero pagos asociados a la operación de prueba.

Esto permitió comprobar que los dos registros fueron deshechos conjuntamente, sin alterar los registros originales del sistema.

Figura. Ejecución de ROLLBACK y verificación de la eliminación de los cambios temporales.



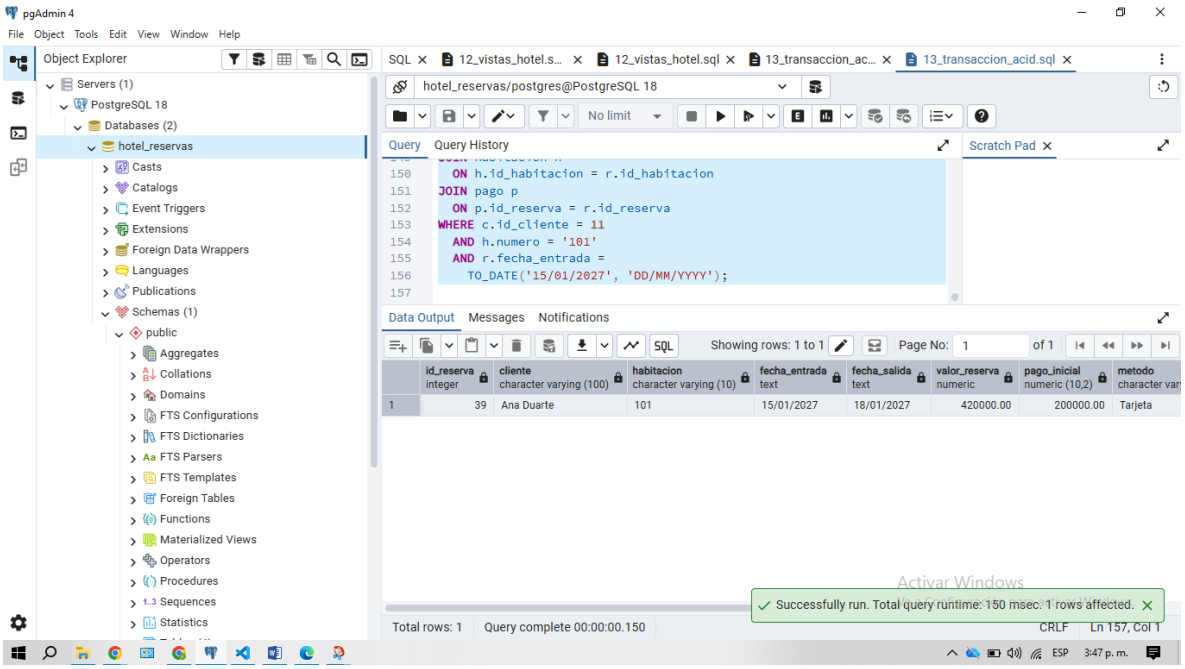
La prueba permitió evidenciar el comportamiento de la atomicidad, ya que ambas inserciones formaban parte de una misma transacción y fueron revertidas mediante una única instrucción.

5.3.2. Prueba de transacción mediante COMMIT

En la segunda prueba se implementó nuevamente el registro de una reserva y su pago inicial, utilizando los mismos datos del escenario anterior. Sin embargo, en esta ocasión se utilizó la instrucción COMMIT para confirmar la transacción.

La ejecución comenzó con BEGIN y continuó con la inserción de la reserva y el pago correspondiente. Una vez completadas ambas operaciones, se ejecutó COMMIT para confirmar los cambios.

Figura. Ejecución de la transacción para registrar una reserva y su pago inicial mediante COMMIT.



Posteriormente, se ejecutó una consulta para verificar la permanencia de los registros confirmados.

El resultado mostró la reserva con identificador 39, correspondiente a Ana Duarte, habitación 101, con fecha de entrada 15/01/2027 y salida 18/01/2027, por un valor de $420.000 y un pago inicial registrado de $200.000 mediante Tarjeta.

La diferencia entre ambas pruebas permitió observar que ROLLBACK revierte los cambios no confirmados, mientras que COMMIT confirma las operaciones y permite que permanezcan almacenadas en la base de datos.

También se identificó que PostgreSQL asignó un nuevo identificador a la segunda reserva, debido a que las secuencias utilizadas por los campos SERIAL no retroceden automáticamente cuando una transacción es revertida.

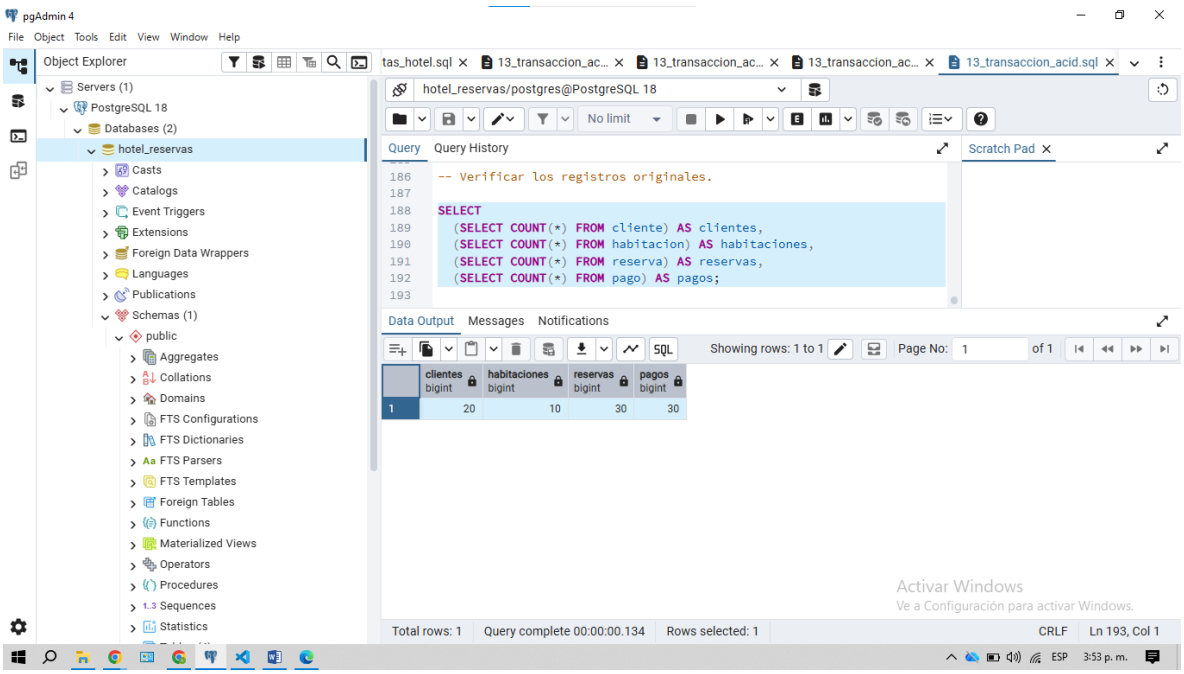
5.3.3. Limpieza controlada de los registros de prueba

Después de comprobar el funcionamiento de COMMIT, se realizó una limpieza controlada para retirar los registros utilizados exclusivamente durante las pruebas de transacciones.

La operación se ejecutó mediante sentencias DELETE, eliminando primero el pago relacionado con la reserva 39 y posteriormente la reserva correspondiente. Este orden permitió respetar las restricciones de clave foránea definidas en el modelo físico.

La limpieza se realizó dentro de una nueva transacción y se confirmó mediante COMMIT después de verificar que las eliminaciones correspondían a los registros de prueba.

Figura. Eliminación controlada de la reserva y el pago utilizados en las pruebas ACID.



Finalmente, se ejecutó una consulta de conteo sobre las cuatro tablas principales del sistema para comprobar que se conservaran las cantidades originales de datos sintéticos.

Los resultados obtenidos fueron:

Tabla	Registros verificados
cliente	20
habitacion	10
reserva	30
pago	30

La verificación confirmó que los registros de prueba fueron retirados correctamente y que el sistema conservó las cantidades originales de información utilizadas durante el desarrollo de las partes anteriores del taller.

5.4. Análisis de las propiedades ACID

Las propiedades ACID constituyen un conjunto de principios fundamentales para el manejo confiable de transacciones en sistemas de bases de datos. Estas propiedades permiten mantener la integridad de la información cuando se realizan operaciones relacionadas.

A continuación, se explica su aplicación en el Sistema de Gestión de Reservas para un Hotel.

Propiedad	Descripción y aplicación en el proyecto
Atomicidad	Garantiza que las operaciones de una transacción se confirmen o se reviertan como una unidad. En el proyecto se comprobó mediante ROLLBACK, que permitió deshacer conjuntamente la reserva 38 y su pago inicial.
Consistencia	Busca que la base de datos pase de un estado válido a otro, respetando las restricciones definidas. En el proyecto, las transacciones estuvieron sujetas a las claves primarias, claves foráneas y restricciones CHECK, que controlan aspectos como las fechas válidas y los montos positivos de los pagos.
Aislamiento	Establece mecanismos para controlar la interacción entre transacciones ejecutadas simultáneamente. PostgreSQL administra la visibilidad de los cambios según el nivel de aislamiento utilizado. Las pruebas del proyecto se realizaron en una misma conexión, por lo cual esta propiedad se analizó conceptualmente y no mediante una prueba de concurrencia entre sesiones.
Durabilidad	Garantiza que los cambios confirmados mediante COMMIT permanezcan almacenados. En el proyecto se verificó que la reserva 39 y su pago continuaron disponibles después de confirmar la transacción. Su posterior eliminación fue una operación independiente de limpieza controlada.

El desarrollo de las pruebas permitió observar directamente el comportamiento de la atomicidad y la confirmación de los cambios mediante COMMIT. Asimismo, se relacionó la consistencia con las restricciones de integridad implementadas previamente y se explicó el papel del aislamiento en escenarios de concurrencia.

Estas propiedades son especialmente importantes en el sistema hotelero, ya que el registro de reservas y pagos requiere mantener relaciones válidas entre las tablas y evitar operaciones incompletas que puedan generar inconsistencias.

5.5. Control de versiones de las transacciones SQL

Una vez finalizadas las pruebas y documentados sus resultados, se organizó el código de las transacciones en el archivo 13_transaccion_acid.sql, manteniendo separados los bloques correspondientes a las pruebas con ROLLBACK, COMMIT y la limpieza controlada.

El archivo contiene las sentencias utilizadas y las consultas de verificación necesarias para comprobar los resultados de cada operación. También se documentó la importancia de ejecutar los bloques de manera individual y en el orden correspondiente, debido a que las pruebas modifican temporalmente los datos de la base de datos.

Para conservar el historial del desarrollo, el script se incorporó al repositorio Git del proyecto mediante un commit específico y su posterior publicación en GitHub.

El uso del control de versiones permitió conservar los archivos SQL utilizados durante el desarrollo y mantener evidencia de las modificaciones realizadas en el proyecto.

5.6. Conclusión de la Parte 5

La implementación de vistas y transacciones permitió fortalecer el funcionamiento del Sistema de Gestión de Reservas para un Hotel mediante herramientas que facilitan la consulta de información y el manejo controlado de operaciones relacionadas.

Las vistas implementadas permitieron consultar las reservas activas y analizar los ingresos mensuales a partir de los datos almacenados. Su verificación confirmó que los resultados coincidían con la información registrada previamente en las tablas del sistema.

Por otra parte, las pruebas con BEGIN, ROLLBACK y COMMIT permitieron comprender cómo PostgreSQL administra las operaciones realizadas dentro de una transacción. La reversión conjunta de una reserva y su pago evidenció el comportamiento de la atomicidad, mientras que la confirmación de los registros permitió observar su permanencia después de ejecutar COMMIT.

Finalmente, la limpieza controlada y la verificación de las cantidades originales confirmaron que las pruebas finalizaron sin alterar el conjunto de datos sintéticos establecido para el proyecto.

De esta manera, se cumplieron los objetivos de implementación de vistas y transacciones de la Parte 5, aplicando conceptos fundamentales de PostgreSQL y documentando su funcionamiento mediante consultas, resultados y evidencias de control de versiones.

Parte 6. Validación de consultas NL2SQL con controles de seguridad

En esta última parte del proyecto se utilizó inteligencia artificial generativa para transformar preguntas de negocio formuladas en lenguaje natural en consultas SQL, mediante el enfoque conocido como NL2SQL (*Natural Language to SQL*).

El objetivo fue evaluar la capacidad de una herramienta de inteligencia artificial para generar consultas compatibles con PostgreSQL 18 y verificar si los resultados obtenidos respondían correctamente a las necesidades de información del Sistema de Gestión de Reservas para un Hotel.

Para realizar el ejercicio se formularon tres preguntas de negocio diferentes de las desarrolladas en la Parte 4. Cada pregunta fue enviada a una conversación independiente de ChatGPT, solicitando la generación de una consulta SQL de solo lectura.

Posteriormente, los códigos generados se conservaron sin modificaciones en el archivo 14_validacion_nl2sql.sql, elaborado en Visual Studio Code, y se ejecutaron en PostgreSQL mediante pgAdmin.

La validación contempló tanto la correcta ejecución de las consultas como la revisión de los resultados obtenidos, considerando las relaciones entre las tablas y las reglas del sistema hotelero.

6.1. Metodología de generación y validación

El proceso se desarrolló mediante las siguientes actividades:

1. **Formulación de preguntas:** se plantearon tres requerimientos de información relacionados con la gestión financiera y operativa del hotel.
2. **Generación mediante inteligencia artificial:** se solicitaron consultas SQL a ChatGPT, proporcionando la estructura de las tablas y las condiciones necesarias para responder cada pregunta.
3. **Conservación del código original:** los códigos generados se guardaron sin modificaciones para permitir su revisión y comparación.
4. **Ejecución controlada:** las consultas se ejecutaron en PostgreSQL 18 utilizando transacciones de solo lectura, evitando modificaciones sobre los registros almacenados.
5. **Validación de resultados:** se comprobaron la ejecución sintáctica y la coherencia de los resultados visibles con las preguntas formuladas.

Esta metodología permitió evaluar las respuestas de la inteligencia artificial sin aceptar automáticamente los códigos como correctos, aplicando una revisión humana de su estructura y funcionamiento.

6.2. Primera pregunta NL2SQL: Reservas con saldo pendiente

6.2.1. Planteamiento de la pregunta

La primera pregunta formulada a la inteligencia artificial fue:

¿Cuáles son las reservas confirmadas del hotel que tienen dinero pendiente por pagar, considerando tanto las reservas sin pagos como aquellas que tienen pagos parciales, y cuánto dinero falta por recaudar en cada una?

El objetivo fue identificar las reservas cuyo valor económico no había sido cubierto completamente mediante los pagos registrados.

Esta consulta permitió ampliar el análisis realizado anteriormente sobre las reservas sin pagos, incluyendo también aquellas que contaban con abonos parciales.

6.2.2. Código generado mediante inteligencia artificial

ChatGPT generó una consulta que relaciona las tablas reserva, cliente, habitacion y pago.

El código utiliza una subconsulta para calcular los pagos acumulados por reserva mediante SUM, mientras que la función COALESCE permite asignar un valor de cero cuando una reserva no tiene pagos asociados.

Posteriormente, se calcula el saldo pendiente restando el dinero pagado al valor total de la estancia, obtenido mediante la multiplicación de las noches por el precio correspondiente a la habitación.

La condición WHERE permite seleccionar únicamente las reservas confirmadas cuyo saldo pendiente sea superior a cero.

Código SQL generado por IA: Primera consulta del archivo 14_validacion_nl2sql.sql, conservada en su versión original.

Prompt enviado:

Actúa como experto en PostgreSQL 18. Tengo un sistema de reservas hoteleras con las siguientes tablas: cliente(id_cliente, nombre, correo, telefono, nacionalidad), habitacion(id_habitacion, numero, tipo, precio_noche, estado), reserva(id_reserva, id_cliente, id_habitacion, fecha_entrada, fecha_salida, estado_reserva) y pago(id_pago, id_reserva, fecha_pago, monto, metodo). Genera una consulta SQL que responda: ¿Cuáles son las reservas confirmadas que tienen saldo pendiente de pago, considerando reservas sin pagos y con pagos parciales, y cuánto dinero falta por recaudar en cada una? El valor de cada

reserva se calcula multiplicando noches por precio_noche. Una reserva puede tener varios pagos. Muestra identificador de reserva, cliente, habitación, valor total, total pagado y saldo pendiente. Utiliza únicamente SELECT, no modifiques la base de datos y explica brevemente tu solución.

Respuesta IA:

```
SELECT
    r.id_reserva,
    c.nombre AS cliente,
    h.numero AS habitacion,
    (r.fecha_salida - r.fecha_entrada) * h.precio_noche AS valor_total,
    COALESCE(p.total_pagado, 0) AS total_pagado,
    (
        (r.fecha_salida - r.fecha_entrada) * h.precio_noche
        - COALESCE(p.total_pagado, 0)
    ) AS saldo_pendiente
FROM reserva r
INNER JOIN cliente c
    ON r.id_cliente = c.id_cliente
INNER JOIN habitacion h
    ON r.id_habitacion = h.id_habitacion
LEFT JOIN (
    SELECT
        id_reserva,
        SUM(monto) AS total_pagado
```

```
    FROM pago
    GROUP BY id_reserva
) p
    ON r.id_reserva = p.id_reserva
WHERE LOWER(r.estado_reserva) = 'confirmada'
    AND (
        (r.fecha_salida - r.fecha_entrada) * h.precio_noche
        - COALESCE(p.total_pagado, 0)
    ) > 0
ORDER BY saldo_pendiente DESC;
```

6.2.3. Ejecución y validación de resultados

La consulta se ejecutó correctamente en PostgreSQL 18 y devolvió ocho reservas confirmadas con saldo pendiente.

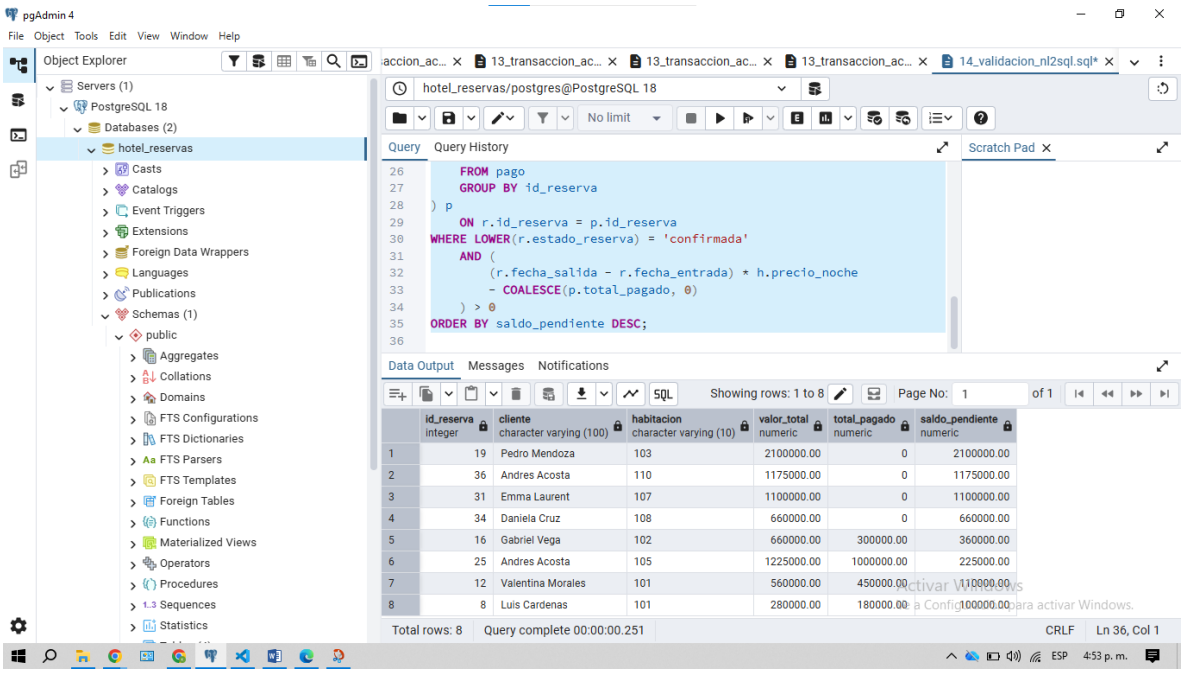
De los ocho registros identificados, cuatro correspondían a reservas sin pagos y cuatro a reservas que tenían pagos parciales.

Entre los resultados obtenidos se identificó la reserva de Pedro Mendoza, correspondiente a la habitación 103, con un valor total de $2.100.000 y sin pagos registrados, por lo que su saldo pendiente era de $2.100.000.

También se identificó una reserva de Gabriel Vega, cuyo valor total era de $660.000, con pagos acumulados de $300.000 y un saldo pendiente de $360.000.

Estos resultados permitieron comprobar que la consulta distinguía correctamente entre reservas sin pagos y reservas con abonos parciales.

Figura. Ejecución y resultados de la consulta NL2SQL de reservas con saldo pendiente.



6.2.4. Evaluación de la consulta

La consulta se ejecutó sin errores y los resultados observados respondieron a la pregunta planteada.

Se verificó que las reservas mostradas estuvieran confirmadas y que los saldos pendientes correspondieran a la diferencia entre el valor de la estancia y los pagos registrados.

Resultado de la evaluación: Consulta válida según las verificaciones realizadas. No fue necesario modificar el código original generado por la inteligencia artificial.

6.3. Segunda pregunta NL2SQL: Porcentaje de ingresos por nacionalidad

6.3.1. Planteamiento de la pregunta

La segunda pregunta formulada a ChatGPT fue:

¿Qué porcentaje de los ingresos efectivamente recaudados por el hotel corresponde a los clientes de cada nacionalidad?

El objetivo fue analizar la participación económica de los clientes según su nacionalidad, utilizando los pagos registrados y no el valor estimado de las reservas.

Esta consulta permite obtener información útil sobre la distribución de los ingresos entre los diferentes grupos de clientes registrados en el sistema.

6.3.2. Código generado mediante inteligencia artificial

ChatGPT generó una consulta que relaciona las tablas pago, reserva y cliente mediante operaciones INNER JOIN.

Para obtener la información solicitada, se utilizó COUNT para contabilizar los pagos de cada nacionalidad y SUM para calcular el dinero recaudado.

Adicionalmente, se empleó una función de ventana mediante SUM(SUM(p.monto)) OVER (), que permite obtener el ingreso total general a partir de los valores agrupados.

El porcentaje correspondiente a cada nacionalidad se calculó dividiendo sus ingresos acumulados entre el total general y multiplicando el resultado por cien.

La función ROUND permitió presentar los porcentajes con dos cifras decimales y NULLIF se utilizó para evitar posibles divisiones entre cero.

Código SQL generado por IA: Segunda consulta del archivo 14_validacion_nl2sql.sql, conservada sin modificaciones.

Prompt enviado:

Actúa como experto en PostgreSQL 18.

Tengo un sistema de reservas hoteleras con las siguientes tablas:

- cliente(id_cliente, nombre, correo, telefono, nacionalidad)

- habitacion(id_habitacion, numero, tipo, precio_noche, estado)

- reserva(id_reserva, id_cliente, id_habitacion, fecha_entrada, fecha_salida, estado_reserva)

- pago(id_pago, id_reserva, fecha_pago, monto, metodo)

Genera una consulta SQL que responda la siguiente pregunta de negocio:

¿Qué porcentaje de los ingresos efectivamente recaudados por el hotel corresponde a los clientes de cada nacionalidad?

Ten en cuenta las siguientes condiciones:

- Los ingresos deben calcularse utilizando los montos registrados en la tabla pago, no el valor estimado de las reservas.

- Un cliente puede tener varias reservas y una reserva puede tener varios pagos.

- Cada pago debe contabilizarse una sola vez.

- Agrupa los resultados por nacionalidad.

- Muestra la nacionalidad, la cantidad de pagos, el ingreso total recaudado y el porcentaje que representa frente al total general.

- Presenta los porcentajes redondeados a dos decimales.

- Ordena los resultados desde la nacionalidad con mayores ingresos hasta la de menores ingresos.

Utiliza únicamente sentencias SELECT, sin modificar la estructura ni los registros de la base de datos.

Entrega el código SQL completo, compatible con PostgreSQL 18, y explica brevemente cómo funciona.

Respuesta IA:

```
SELECT

    c.nacionalidad,

    COUNT(p.id_pago) AS cantidad_pagos,

    SUM(p.monto) AS ingreso_total_recaudado,

    ROUND(

        100.0 * SUM(p.monto) /

        NULLIF(SUM(SUM(p.monto)) OVER (), 0),

        2

    ) AS porcentaje_ingresos
```

```
FROM pago p

INNER JOIN reserva r

    ON p.id_reserva = r.id_reserva

INNER JOIN cliente c

    ON r.id_cliente = c.id_cliente

GROUP BY c.nacionalidad

ORDER BY ingreso_total_recaudado DESC;
```

6.3.3. Ejecución y validación de resultados

La consulta se ejecutó correctamente en PostgreSQL 18 y devolvió nueve nacionalidades con ingresos registrados.

Los resultados visibles incluyeron clientes de España, Brasil, México, Estados Unidos, Venezuela, Panamá, Argentina y Chile, además de Colombia.

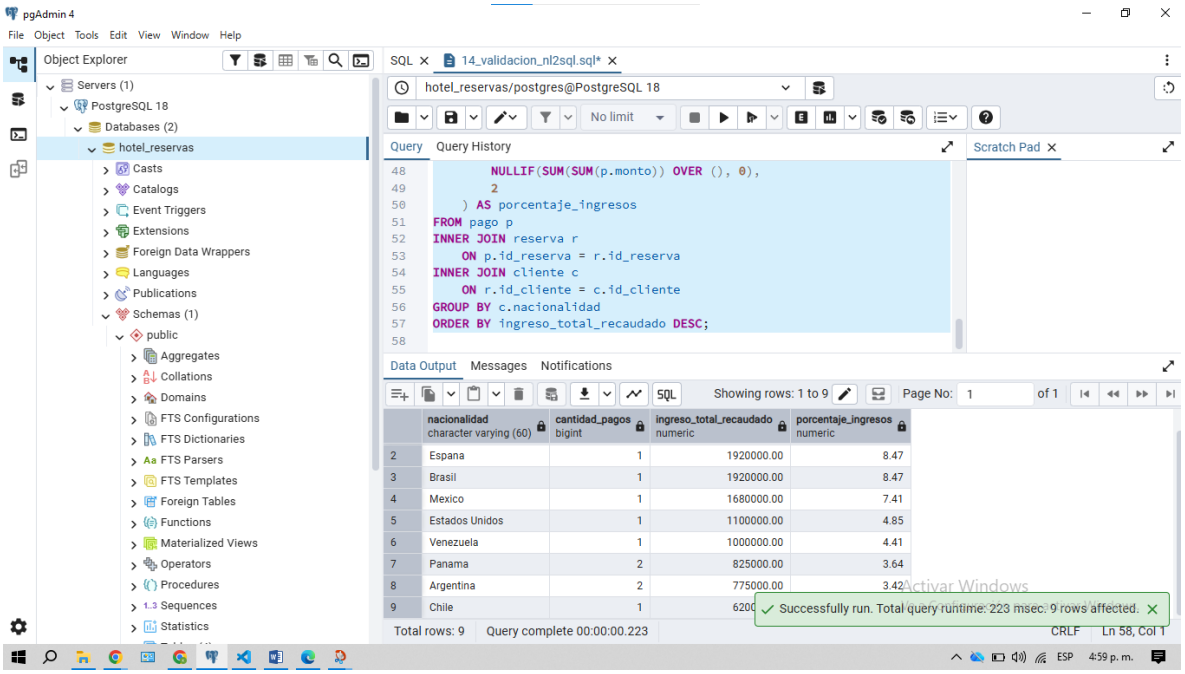
Por ejemplo, los clientes de España y Brasil registraron ingresos de $1.920.000 cada uno, equivalentes aproximadamente al 8,47 % del total general para cada nacionalidad.

Asimismo, México registró ingresos de $1.680.000, correspondientes al 7,41 % del total recaudado.

La consulta utilizó como base los montos existentes en la tabla pago, cuyos treinta registros representaban ingresos generales por $22.665.000.

Los resultados obtenidos permitieron observar la distribución de ingresos según la nacionalidad, sin calcular los porcentajes sobre el valor estimado de las estancias.

Figura. Ejecución y resultados de la consulta NL2SQL de ingresos por nacionalidad.



6.3.4. Evaluación de la consulta

La consulta se ejecutó sin errores de sintaxis y presentó los resultados agrupados por nacionalidad, incluyendo la cantidad de pagos, el ingreso total y el porcentaje correspondiente.

Los valores visibles fueron coherentes con los montos registrados en la base de datos y con el objetivo de la pregunta.

Resultado de la evaluación: Consulta válida en su ejecución y en los resultados revisados. No se identificaron errores que requirieran modificar el código original de la inteligencia artificial.

6.4. Tercera pregunta NL2SQL: Días libres entre reservas consecutivas

6.4.1. Planteamiento de la pregunta

La tercera pregunta formulada a ChatGPT fue:

¿Cuántos días libres existen entre reservas confirmadas consecutivas de una misma habitación?

El objetivo fue identificar los periodos transcurridos entre la finalización de una reserva y el comienzo de la siguiente, considerando únicamente las reservas confirmadas.

Esta información puede resultar útil para analizar los intervalos sin reservas programadas entre diferentes estancias de una misma habitación.

6.4.2. Código generado mediante inteligencia artificial

La inteligencia artificial generó una consulta que utiliza una expresión de tabla común mediante WITH, denominada reservas_confirmadas.

Dentro de esta expresión se empleó la función de ventana LAG, que permite recuperar información de la reserva anterior dentro de un conjunto ordenado.

La instrucción PARTITION BY separa las reservas según su habitación, mientras que ORDER BY establece su secuencia cronológica.

Posteriormente, la consulta calcula los días libres mediante la diferencia entre la fecha de entrada de la reserva siguiente y la fecha de salida de la reserva anterior.

También se utilizó TO_CHAR para presentar las fechas en el formato DD/MM/AAAA y una condición para excluir los registros que no tenían una reserva anterior con la cual compararse.

Código SQL generado por IA: Tercera consulta del archivo 14_validacion_nl2sql.sql, conservada sin modificaciones.

Figura. Interacción con ChatGPT para generar la consulta de días libres entre reservas consecutivas.

Prompt enviado:

Actúa como experto en PostgreSQL 18.

Tengo un sistema de reservas hoteleras con las siguientes tablas:

- cliente(id_cliente, nombre, correo, telefono, nacionalidad)

- habitacion(id_habitacion, numero, tipo, precio_noche, estado)

- reserva(id_reserva, id_cliente, id_habitacion, fecha_entrada, fecha_salida, estado_reserva)

- pago(id_pago, id_reserva, fecha_pago, monto, metodo)

Genera una consulta SQL que responda la siguiente pregunta de negocio:

¿Cuántos días libres existen entre reservas confirmadas consecutivas de una misma habitación?

Ten en cuenta las siguientes condiciones:

- Considera únicamente las reservas cuyo estado sea Confirmada.

- Una habitación puede tener múltiples reservas en diferentes periodos.

- Compara cada reserva con la reserva confirmada inmediatamente anterior de la misma habitación, según las fechas de entrada.

- Calcula los días libres entre la fecha de salida de la reserva anterior y la fecha de entrada de la siguiente reserva.

- Si una reserva comienza exactamente el día en que termina la anterior, el resultado debe ser cero días libres.

- No incluyas reservas canceladas ni relaciones entre habitaciones diferentes.

- No muestres la primera reserva de cada habitación si no existe una reserva anterior con la cual compararla.

- Muestra el identificador y número de habitación, el identificador de la reserva anterior, su fecha de salida, el identificador de la siguiente reserva, su fecha de entrada y la cantidad de días libres.

- Presenta las fechas en formato DD/MM/AAAA mediante TO_CHAR.

- Ordena los resultados por número de habitación y fecha de entrada de la siguiente reserva.

Utiliza únicamente consultas SELECT, sin modificar la estructura ni los registros de la base de datos.

Entrega el código SQL completo, compatible con PostgreSQL 18, y explica brevemente cómo funciona.

Respuesta IA:

```
WITH reservas_confirmadas AS (
    SELECT
        r.id_habitacion,
        h.numero AS numero_habitacion,
        r.id_reserva AS id_reserva_siguiente,
        r.fecha_entrada AS fecha_entrada_siguiente,

        LAG(r.id_reserva) OVER (
            PARTITION BY r.id_habitacion
            ORDER BY r.fecha_entrada, r.id_reserva
        ) AS id_reserva_anterior,

        LAG(r.fecha_salida) OVER (
            PARTITION BY r.id_habitacion
            ORDER BY r.fecha_entrada, r.id_reserva
        ) AS fecha_salida_anterior

    FROM reserva r
    INNER JOIN habitacion h
        ON r.id_habitacion = h.id_habitacion
    WHERE r.estado_reserva = 'Confirmada'
```

```
)
SELECT
    id_habitacion,
    numero_habitacion,
    id_reserva_anterior,
    TO_CHAR(
        fecha_salida_anterior, 'DD/MM/YYYY'
    ) AS fecha_salida_anterior,
    id_reserva_siguiente,
    TO_CHAR(
        fecha_entrada_siguiente, 'DD/MM/YYYY'
    ) AS fecha_entrada_siguiente,
    (
        fecha_entrada_siguiente - fecha_salida_anterior
    ) AS dias_libres
FROM reservas_confirmadas
WHERE id_reserva_anterior IS NOT NULL
ORDER BY
    numero_habitacion,
    fecha_entrada_siguiente,
    id_reserva_siguiente;
```

6.4.3. Ejecución y validación de resultados

La consulta se ejecutó correctamente en PostgreSQL 18 y devolvió diecinueve registros correspondientes a comparaciones entre reservas confirmadas consecutivas.

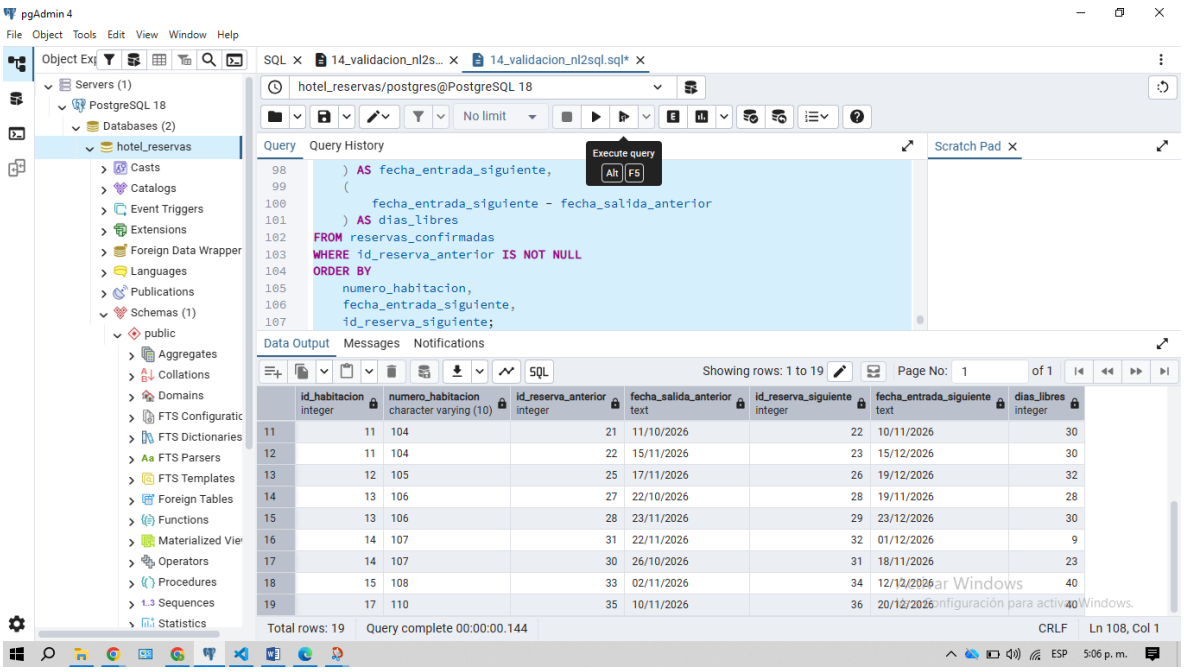
Entre los resultados observados se identificó la habitación 110, cuya reserva anterior finalizó el 10/11/2026 y cuya siguiente reserva comenzó el 20/12/2026.

PostgreSQL calculó una diferencia de cuarenta días entre ambas fechas.

También se observaron registros correspondientes a otras habitaciones, en los cuales la consulta permitió relacionar la fecha de finalización de una estancia con el comienzo de la siguiente.

El resultado de diecinueve comparaciones fue coherente con las veintiocho reservas confirmadas distribuidas entre nueve habitaciones que tenían reservas registradas.

Figura. Ejecución y resultados de la consulta NL2SQL de días libres entre reservas consecutivas.



	id_habitacion integer	numero_habitacion character varying (10)	id_reserva_anterior integer	fecha_salida_anterior text	id_reserva_siguiente integer	fecha_entrada_siguiente text	dias_libres integer
11	11	104	21	11/10/2026	22	10/11/2026	30
12	11	104	22	15/11/2026	23	15/12/2026	30
13	12	105	25	17/11/2026	26	19/12/2026	32
14	13	106	27	22/10/2026	28	19/11/2026	28
15	13	106	28	23/11/2026	29	23/12/2026	30
16	14	107	31	22/11/2026	32	01/12/2026	9
17	14	107	30	26/10/2026	31	18/11/2026	23
18	15	108	33	02/11/2026	34	12/12/2026	40
19	17	110	35	10/11/2026	36	20/12/2026	40

6.4.4. Evaluación de la consulta

La consulta se ejecutó sin errores y los resultados observados mostraron comparaciones cronológicas entre reservas de una misma habitación.

Se comprobó que la función LAG permitió obtener los datos de las reservas anteriores y calcular los intervalos entre estancias.

Resultado de la evaluación: Consulta válida en su ejecución y en los ejemplos revisados. No se identificó una corrección necesaria en el código original, aunque no se realizó una comprobación independiente de todos los intervalos obtenidos.

6.5. Tabla de auditoría de las consultas generadas por IA

Con el propósito de documentar la evaluación realizada, se elaboró una tabla de auditoría que relaciona las preguntas formuladas, el código generado por ChatGPT, los resultados de la validación y las correcciones aplicadas.

Pregunta en lenguaje natural	SQL generado por la IA	¿Válido?	Corrección aplicada
¿Qué reservas confirmadas tienen saldo pendiente y cuánto falta por recaudar?	Consulta con LEFT JOIN, SUM, COALESCE y cálculo del saldo. Código completo en 14_validacion_nl2sql.sql, Pregunta 1.	Sí. Se ejecutó correctamente y devolvió 8 reservas con saldo pendiente.	Ninguna. Los resultados revisados fueron coherentes.
¿Qué porcentaje de ingresos corresponde a cada nacionalidad?	Consulta con INNER JOIN, SUM, GROUP BY y función de ventana OVER(). Código completo en 14_validacion_nl2sql.sql, Pregunta 2.	Sí, según la ejecución y resultados visibles. Se obtuvieron 9 nacionalidades.	Ninguna. No se identificaron errores en los resultados revisados.
¿Cuántos días libres existen entre reservas confirmadas consecutivas de una habitación?	Consulta con WITH, LAG, PARTITION BY y cálculo de diferencias entre fechas. Código completo en 14_validacion_nl2sql.sql, Pregunta 3.	Sí, según la ejecución y ejemplos revisados. Se obtuvieron 19 comparaciones.	Ninguna. No se identificaron errores en las comparaciones examinadas.

Los tres códigos generados por la inteligencia artificial se conservaron en el archivo SQL del proyecto, permitiendo consultar sus versiones originales y verificar las operaciones empleadas.

Durante las ejecuciones no se presentaron errores de sintaxis. Las verificaciones realizadas permitieron observar que los resultados respondían a las preguntas formuladas, dentro del alcance de los registros examinados.

No fue necesario modificar los códigos originales, debido a que no se identificaron errores que justificaran una corrección.

6.6. Controles de seguridad aplicados

Durante la validación de las consultas NL2SQL se establecieron controles para proteger la integridad de la base de datos.

En primer lugar, se solicitó a ChatGPT generar exclusivamente consultas SELECT, evitando instrucciones que modificaran la estructura o los registros del sistema.

Adicionalmente, las consultas se ejecutaron mediante transacciones de solo lectura, utilizando BEGIN TRANSACTION READ ONLY, lo que permitió restringir operaciones de modificación de las tablas durante las pruebas.

Al finalizar cada ejecución se utilizó ROLLBACK para cerrar la transacción de prueba.

Este procedimiento permitió evaluar el código generado por la inteligencia artificial sin realizar modificaciones sobre los registros sintéticos del hotel.

También se mantuvo la intervención humana durante la revisión de las consultas, considerando que una sentencia SQL puede ejecutarse correctamente y, aun así, producir resultados que no respondan adecuadamente a la pregunta planteada.

Por esta razón, la validación contempló tanto el funcionamiento técnico como la interpretación de los resultados visibles.

6.7. Control de versiones y evidencias

Las tres consultas generadas mediante inteligencia artificial se organizaron en el archivo 14_validacion_nl2sql.sql, manteniendo identificadas las preguntas de negocio y los códigos originales correspondientes.

Este archivo forma parte de los scripts desarrollados para el proyecto y debe conservarse en el repositorio GitHub junto con los archivos de las partes anteriores.

Asimismo, se conservaron evidencias de las interacciones con ChatGPT y de las ejecuciones realizadas en PostgreSQL mediante pgAdmin.

6.8. Conclusión de la Parte 6

La utilización de inteligencia artificial para generar consultas SQL permitió explorar la aplicación del enfoque NL2SQL dentro del Sistema de Gestión de Reservas para un Hotel.

Las tres preguntas formuladas abordaron necesidades de información relacionadas con los saldos pendientes, la distribución de ingresos por nacionalidad y los intervalos entre reservas consecutivas.

Los códigos generados por ChatGPT fueron ejecutados en PostgreSQL 18, obteniendo resultados coherentes con las verificaciones realizadas sobre el conjunto de datos sintéticos.

Adicionalmente, se aplicaron controles de seguridad mediante consultas de solo lectura, evitando modificaciones sobre la información almacenada.

El desarrollo de esta actividad permitió reconocer que la inteligencia artificial puede apoyar la construcción de consultas SQL, pero que sus respuestas deben ser revisadas por una persona para verificar su compatibilidad con el modelo físico y la coherencia de los resultados.

De esta manera, se desarrollaron las actividades correspondientes a la Parte 6, integrando el uso de inteligencia artificial, la validación de consultas SQL y la aplicación de controles de seguridad sobre la base de datos del sistema hotelero.